Telegram Bot API

Fonte: https://core.telegram.org/bots/api?utm_source=chatgpt.com#sendwidget · Extraído em 10 de novembro de 2026

> The Bot API is an HTTP-based interface created for developers keen on building bots for Telegram.
> To learn how to create and set up a bot, please consult our **Introduction to Bots** and **Bot FAQ**.

Recent changes

> Subscribe to @BotNews to be the first to know about the latest updates and join the discussion in @BotTalk

November 9, 2026

Bot API 10.4

Widgets

- Added the class Widget and the field *widget* to the classes Message and ExternalReplyInfo for messages with HTML widgets.
- Added the method sendWidget, allowing bots to send messages with HTML widgets. Currently, in the production environment, widgets can be sent only to private chats, and sent widgets automatically have protected content and therefore can't be forwarded.
- Added the method sendSharedWidget, allowing bots to send again shared widgets created by the bot.
- Added the class WidgetQuery and the field *widget_query* to the class Update for queries sent to the bot by widgets in messages sent by the bot or via the bot.
- Added the class SharedWidgetQuery and the field *shared_widget_query* to the class Update for queries sent to the bot from forwarded messages with shared widgets.
- Added the classes WidgetQueryType, WidgetQueryTypeData, WidgetQueryTypeSignData, WidgetQueryTypeSentTransaction and WidgetQueryWallet, describing the type of a query from a widget. A query can contain data for the bot together with the TON wallet card of the user, data signed by a TON wallet of the user, or a notification about a transaction sent from a TON wallet of the user.
- Added the method answerWidgetQuery, allowing bots to answer queries from widgets.
- Added the method editMessageWidgetData, allowing bots to change data of a widget in a message.
- Added the method editSharedWidgetData, allowing bots to change public data of a shared widget.
- Added the classes InputWidgetMessageContent and InputSharedWidgetMessageContent, allowing bots to send widgets in response to inline queries. Currently, widgets can't be sent in

response to inline queries in the production environment.
- Widget messages can't be copied using the methods copyMessage and copyMessages.
- The right *can_send_other_messages* in the classes ChatPermissions and ChatMemberRestricted now also allows sending widgets.

Ephemeral messages

- Allowed bots to send ephemeral replies to callback queries in any chat except channel direct messages chats by specifying *callback_query_id* without *receiver_user_id* in the parameter *ephemeral_message_parameters* of the methods sendMessage, sendRichMessage, sendAnimation, sendAudio, sendContact, sendDocument, sendLivePhoto, sendLocation, sendPhoto, sendSticker, sendVenue, sendVideo, sendVideoNote and sendVoice.
- The field *receiver_user_id* of the class EphemeralMessageParameters and the parameter *chat_id* of the mentioned methods are optional now.

Quotes from rich messages

- Added the field *rich_texts* to the class TextQuote for quotes chosen from a RichMessage.
- Added the field *quote_rich_texts* to the class ReplyParameters, allowing bots to quote rich messages.

TON wallet

- Added the classes TonWalletCard, TonWalletCardSticker, TonNft and TonNftAttribute, and the field *ton_wallet_card* to the class ChatFullInfo.
- Added the class TonWalletTransfer and the field *ton_wallet_transfer* to the class Message for service messages about transfers with a TON wallet.
- Renamed the class StoryAreaPosition to AreaPosition, because it is now also used to describe the position of stickers on TON wallet cards.

Chat share themes

- Added the classes ChatShareTheme, ChatShareThemeEmoji and ChatShareThemeUniqueGift, and the field *share_theme* to the class ChatFullInfo, describing the theme of the screen for sharing a link to a user or a chat.
- Added the classes EmojiType, EmojiTypeEmoji and EmojiTypeCustomEmoji.
- Added the classes ChatShareThemeBackdrop and ChatShareThemeBackdrops and the method getChatShareThemeBackdrops.

Gifts

- Added the method getUniqueGift, allowing bots to get information about a unique gift by its name.
- Added the method deleteBusinessAccountGift, allowing bots to delete regular gifts owned by a managed business account without receiving Telegram Stars for them.

General

- Allowed users with a Telegram Premium subscription to assign secondary usernames ending with “agent”, “ai”, “app”, “game”, or “gay” to up to 2 owned bots. Such usernames can be suggested in the field *suggested_username* of the class KeyboardButtonRequestManagedBot.

- Added the class RichTextTonAddress representing an address in the TON blockchain in a RichMessage.
- Added the type “ton_address” to the class MessageEntity for addresses in the TON blockchain.
- Added the type “bank_card_number” to the class MessageEntity for bank card numbers.
- Added the parameters *custom_description_parse_mode* and *custom_description_entities* to the methods verifyUser and verifyChat.
- Added the class PaidMessagesWaived and the field *paid_messages_waived* to the class Message for service messages about the sender using a link which allows sending messages without paying Telegram Stars.
- Added the ability to create links to topics in private chats with the bot in the format `https://t.me/<bot_username>/<message_thread_id>`. Such a link opens the topic only for the user in the private chat with whom the topic exists.

August 24, 2026

Bot API 10.3

Rich Messages

- Added the class RichMessageButton representing a button in a RichMessage.
- Added the class RichTextButton.
- Added the classes RichBlockButtons and InputRichBlockButtons.
- Added the field *is_compact* to the classes RichBlockTable and InputRichBlockTable.
- Added the classes RichBlockExpandableBlockQuotation and InputRichBlockExpandableBlockQuotation representing a block quotation, which can be expanded or collapsed back.
- Added the classes RichBlockDocument and InputRichBlockDocument, containing a file.
- Added support for links of the form `tg://document?id=` for general file uploads in rich messages.

Ephemeral messages

- Added the class EphemeralMessageParameters and replaced the parameters *receiver_user_id* and *callback_query_id* in the methods sendMessage, sendAnimation, sendAudio, sendDocument, sendLivePhoto, sendPhoto, sendSticker, sendVideo, sendVideoNote, sendVoice, sendContact, sendLocation and sendVenue with the parameter *ephemeral_message_parameters*.
- Added the parameter *ephemeral_message_parameters* to the method sendRichMessage.
- Added the field *replace_callback_query_message* to the class EphemeralMessageParameters, which allows bots to show an ephemeral message in place of the original message.
- Supported upload of new files in editEphemeralMessageMedia.
- Added the parameter *show_caption_above_media* to the method editEphemeralMessageCaption.
- Added the parameter *rich_message* to the method editEphemeralMessageText.
- Added the field *can_send_welcome_messages* to the classes ChatAdministratorRights and ChatMemberAdministrator.
- Added the parameter *can_send_welcome_messages* to the method promoteChatMember.

Reply markup

- Added the class DisabledButton and the field *disabled* to the class InlineKeyboardButton.

- Added the field *force_reply* to the classes InlineKeyboardMarkup and ReplyKeyboardMarkup.

General

- Added the parameters *can_stop* and *keep_on_stop* to the methods sendMessageDraft and sendRichMessageDraft.
- Added updates about user stopping message generation, represented by the class MessageGenerationStopped and the field *stopped_message_generation* in the class Update.
- Added the class CommunityChatJoined and the field *community_chat_joined* to the class Message for service messages about join of a chat from a community.
- Added the fields *text*, *entities* and *is_private* to the class UniqueGiftInfo.

July 14, 2026

Bot API 10.2

Rich Messages

- Added the class InputRichMessageMedia and the field *media* to the class InputRichMessage, allowing bots to explicitly specify media used in *markdown* or *html* formatting when sending a rich message.
- Added the class InputMediaVoiceNote, representing a voice message to be sent.
- Added the class InputRichBlockListItem, which represents an item in a list to be sent.
- Added the classes InputRichBlockParagraph, InputRichBlockSectionHeading, InputRichBlockPreformatted, InputRichBlockFooter, InputRichBlockDivider, InputRichBlockMathematicalExpression, InputRichBlockAnchor, InputRichBlockList, InputRichBlockBlockQuotation, InputRichBlockPullQuotation, InputRichBlockCollage, InputRichBlockSlideshow, InputRichBlockTable, InputRichBlockDetails, InputRichBlockMap, InputRichBlockAnimation, InputRichBlockAudio, InputRichBlockPhoto, InputRichBlockVideo, InputRichBlockVoiceNote and InputRichBlockThinking, which represent different types of blocks available to format an outgoing rich message.
- Added the field *blocks* to the class InputRichMessage, allowing bots to specify rich message formatting via block entities.

Ephemeral Messages

- Introduced support for Ephemeral Messages, allowing bots to send group messages and receive commands that are visible only to a specific user and the bot.
- Added the field *is_ephemeral* to the class BotCommand.
- Added the field *receiver_user* to the class Message.
- Added the field *ephemeral_message_id* to the class Message.
- Added the parameters *receiver_user_id* and *callback_query_id* to the methods sendMessage, sendAnimation, sendAudio, sendDocument, sendLivePhoto, sendPhoto, sendSticker, sendVideo, sendVideoNote, sendVoice, sendContact, sendLocation, sendVenue.
- Added the field *ephemeral_message_id* to the class ReplyParameters, allowing bots to reply to ephemeral messages.
- Marked the field *message_id* in the class ReplyParameters as optional if the field *ephemeral_message_id* is present.
- Added the methods editEphemeralMessageText, editEphemeralMessageMedia, editEphemeralMessageCaption, and editEphemeralMessageReplyMarkup allowing bots to edit

ephemeral messages.
- Added the method deleteEphemeralMessage allowing bots to delete ephemeral messages.

Communities

- Introduced initial support for **Communities** - several supergroups, channels, and bots linked together around a shared topic or audience.
- Added the class Community which represents a community.
- Added the class CommunityChatAdded and the field *community_chat_added* to the class Message.
- Added the class CommunityChatRemoved and the field *community_chat_removed* to the class Message.
- Added the field *community* to the class ChatFullInfo.

General

- Added updates about changes to a user payment subscription, represented by the class BotSubscriptionUpdated and the field *subscription* in the class Update.
- Hardened the security of Mini Apps by disallowing the usage of Mini App methods from origins different from the original Mini App domain. The protection will be automatically enabled for all Mini Apps on July 20, 2026. You can opt-out from the protection through the @BotFather Mini App. If you do so, you acknowledge that it is the responsibility of the bot to ensure that the Mini App has no links to untrusted sites.

See earlier changes »

Authorizing your bot

Each bot is given a unique authentication token when it is created. The token looks something like `123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11`, but we'll use simply **<token>** in this document instead. You can learn about obtaining tokens and generating new ones in this document.

Making requests

All queries to the Telegram Bot API must be served over HTTPS and need to be presented in this form: `https://api.telegram.org/bot<token>/METHOD_NAME`. Like this for example:

```
https://api.telegram.org/bot123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11/getMe
```

We support **GET** and **POST** HTTP methods. We support four ways of passing parameters in Bot API requests:

- URL query string
- application/x-www-form-urlencoded
- application/json (except for uploading files)
- multipart/form-data (use to upload files)

The response contains a JSON object, which always has a Boolean field 'ok' and may have an optional String field 'description' with a human-readable description of the result. If 'ok' equals *True*, the request was successful and the result of the query can be found in the 'result' field. In case of an

unsuccessful request, 'ok' equals *False* and the error is explained in the 'description'. An Integer 'error_code' field is also returned, but its contents are subject to change in the future. Some errors may also have an optional field 'parameters' of the type ResponseParameters, which can help to automatically handle the error.

- All methods in the Bot API are case-insensitive.
- All queries must be made using UTF-8.

Making requests when getting updates

If you're using **webhooks**, you can perform a request to the Bot API while sending an answer to the webhook. Use either *application/json* or *application/x-www-form-urlencoded* or *multipart/form-data* response content type for passing parameters. Specify the method to be invoked in the *method* parameter of the request. It's not possible to know that such a request was successful or get its result.

> Please see our FAQ for examples.

Using a Local Bot API Server

The Bot API server source code is available at telegram-bot-api. You can run it locally and send the requests to your own server instead of `https://api.telegram.org`. If you switch to a local Bot API server, your bot will be able to:

- Download files without a size limit.
- Upload files up to 2000 MB.
- Upload files using their local path and the file URI scheme.
- Use an HTTP URL for the webhook.
- Use any local IP address for the webhook.
- Use any port for the webhook.
- Set *max_webhook_connections* up to 100000.
- Receive the absolute local path as a value of the *file_path* field without the need to download the file after a getFile request.

Do I need a Local Bot API Server

The majority of bots will be OK with the default configuration, running on our servers. But if you feel that you need one of these features, you're welcome to switch to your own at any time.

Getting updates

There are two mutually exclusive ways of receiving updates for your bot - the getUpdates method on one hand and webhooks on the other. Incoming updates are stored on the server until the bot receives them either way, but they will not be kept longer than 24 hours.

Regardless of which option you choose, you will receive JSON-serialized Update objects as a result.

Update

This object represents an incoming update.
At most **one** of the optional fields can be present in any given update.

Field	Type	Description
update_id	Integer	The update's unique identifier. Update identifiers start from a certain positive number and increase sequentially. This identifier becomes especially handy if you're using webhooks, since it allows you to ignore repeated updates or to restore the correct update sequence, should they get out of order. If there are no new updates for at least a week, then identifier of the next update will be chosen randomly instead of sequentially.
message	Message	*Optional*. New incoming message of any kind - text, photo, sticker, etc.
edited_message	Message	*Optional*. New version of a message that is known to the bot and was edited. This update may at times be triggered by changes to message fields that are either unavailable or not actively used by your bot.
channel_post	Message	*Optional*. New incoming channel post of any kind - text, photo, sticker, etc.
edited_channel_post	Message	*Optional*. New version of a channel post that is known to the bot and was edited. This update may at times be triggered by changes to message fields that are either unavailable or not actively used by your bot.
business_connection	BusinessConnection	*Optional*. The bot was connected to or disconnected from a business account, or a user edited an existing connection with the bot
business_message	Message	*Optional*. New message from a connected business account
edited_business_message	Message	*Optional*. New version of a message from a connected business account
deleted_business_messages	BusinessMessagesDeleted	*Optional*. Messages were deleted from a connected business account
guest_message	Message	*Optional*. New guest message. The bot can use the field *Message.guest_query_id* and the method answerGuestQuery to send a message in response.

Field	Type	Description
message_reaction	MessageReactionUpdated	*Optional*. A reaction to a message was changed by a user. The bot must be an administrator in the chat and must explicitly specify `"message_reaction"` in the list of *allowed_updates* to receive these updates. The update isn't received for reactions set by bots.
message_reaction_count	MessageReactionCountUpdated	*Optional*. Reactions to a message with anonymous reactions were changed. The bot must be an administrator in the chat and must explicitly specify `"message_reaction_count"` in the list of *allowed_updates* to receive these updates. The updates are grouped and can be sent with delay up to a few minutes.
inline_query	InlineQuery	*Optional*. New incoming inline query
chosen_inline_result	ChosenInlineResult	*Optional*. The result of an inline query that was chosen by a user and sent to their chat partner. Please see our documentation on the feedback collecting for details on how to enable these updates for your bot.
callback_query	CallbackQuery	*Optional*. New incoming callback query
widget_query	WidgetQuery	*Optional*. New incoming query from a widget in a message sent by the bot or via the bot
shared_widget_query	SharedWidgetQuery	*Optional*. New incoming query from a forwarded message with a shared widget
shipping_query	ShippingQuery	*Optional*. New incoming shipping query. Only for invoices with flexible price.
pre_checkout_query	PreCheckoutQuery	*Optional*. New incoming pre-checkout query. Contains full information about checkout.
purchased_paid_media	PaidMediaPurchased	*Optional*. A user purchased paid media with a non-empty payload sent by the bot in a non-channel chat
poll	Poll	*Optional*. New poll state. Bots receive only updates about manually stopped polls and polls, which are sent by the bot.
poll_answer	PollAnswer	*Optional*. A user changed their answer in a non-anonymous poll. Bots receive new votes only in polls that were sent by the bot itself.
my_chat_member	ChatMemberUpdated	*Optional*. The bot's chat member status was updated in a chat. For private chats, this update is received only when the bot is blocked or unblocked by the user.

Field	Type	Description
chat_member	ChatMemberUpdated	*Optional*. A chat member's status was updated in a chat. The bot must be an administrator in the chat and must explicitly specify `"chat_member"` in the list of *allowed_updates* to receive these updates.
chat_join_request	ChatJoinRequest	*Optional*. A request to join the chat has been sent. The bot must have the *can_invite_users* administrator right in the chat to receive these updates.
chat_boost	ChatBoostUpdated	*Optional*. A chat boost was added or changed. The bot must be an administrator in the chat to receive these updates.
removed_chat_boost	ChatBoostRemoved	*Optional*. A boost was removed from a chat. The bot must be an administrator in the chat to receive these updates.
managed_bot	ManagedBotUpdated	*Optional*. A new bot was created to be managed by the bot, or token or owner of a managed bot was changed
subscription	BotSubscriptionUpdated	*Optional*. User payment subscription has changed
stopped_message_generation	MessageGenerationStopped	*Optional*. A user asked the bot to stop the generation of a message

getUpdates

Use this method to receive incoming updates using long polling (wiki). Returns an Array of Update objects.

Parameter	Type	Required	Description
offset	Integer	Optional	Identifier of the first update to be returned. Must be greater by one than the highest among the identifiers of previously received updates. By default, updates starting with the earliest unconfirmed update are returned. An update is considered confirmed as soon as getUpdates is called with an *offset* higher than its *update_id*. The negative offset can be specified to retrieve updates starting from *-offset* update from the end of the updates queue. All previous updates will be forgotten.
limit	Integer	Optional	Limits the number of updates to be retrieved. Values between 1-100 are accepted. Defaults to 100.
timeout	Integer	Optional	Timeout in seconds for long polling. Defaults to 0, i.e. usual short polling. Should be positive, short polling should be used for testing purposes only.

Parameter	Type	Required	Description
allowed_updates	Array of String	Optional	A JSON-serialized list of the update types you want your bot to receive. For example, specify `["message", "edited_channel_post", "callback_query"]` to only receive updates of these types. See Update for a complete list of available update types. Specify an empty list to receive all update types except *chat_member*, *message_reaction*, and *message_reaction_count* (default). If not specified, the previous setting will be used. Please note that this parameter doesn't affect updates created before the call to getUpdates, so unwanted updates may be received for a short period of time.

> **Notes**
> **1.** This method will not work if an outgoing webhook is set up.
> **2.** In order to avoid getting duplicate updates, recalculate *offset* after each server response.

setWebhook

Use this method to specify a URL and receive incoming updates via an outgoing webhook. Whenever there is an update for the bot, we will send an HTTPS POST request to the specified URL, containing a JSON-serialized Update. In case of an unsuccessful request (a request with response HTTP status code different from `2XY`), we will repeat the request and give up after a reasonable amount of attempts. Returns *True* on success.

If you'd like to make sure that the webhook was set by you, you can specify secret data in the parameter *secret_token*. If specified, the request will contain a header “X-Telegram-Bot-Api-Secret-Token” with the secret token as content.

Parameter	Type	Required	Description
url	String	Yes	HTTPS URL to send updates to. Use an empty string to remove webhook integration.
certificate	InputFile	Optional	Upload your public key certificate so that the root certificate in use can be checked. See our self-signed guide for details.
ip_address	String	Optional	The fixed IP address which will be used to send webhook requests instead of the IP address resolved through DNS
max_connections	Integer	Optional	The maximum allowed number of simultaneous HTTPS connections to the webhook for update delivery, 1-100. Defaults to *40*. Use lower values to limit the load on your bot's server, and higher values to increase your bot's throughput.

Parameter	Type	Required	Description
allowed_updates	Array of String	Optional	A JSON-serialized list of the update types you want your bot to receive. For example, specify `["message", "edited_channel_post", "callback_query"]` to only receive updates of these types. See Update for a complete list of available update types. Specify an empty list to receive all update types except *chat_member*, *message_reaction*, and *message_reaction_count* (default). If not specified, the previous setting will be used. Please note that this parameter doesn't affect updates created before the call to the setWebhook, so unwanted updates may be received for a short period of time.
drop_pending_updates	Boolean	Optional	Pass *True* to drop all pending updates
secret_token	String	Optional	A secret token to be sent in a header “X-Telegram-Bot-Api-Secret-Token” in every webhook request, 1-256 characters. Only characters `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. The header is useful to ensure that the request comes from a webhook set by you.

> **Notes**
> **1.** You will not be able to receive updates using getUpdates for as long as an outgoing webhook is set up.
> **2.** To use a self-signed certificate, you need to upload your public key certificate using *certificate* parameter. Please upload as InputFile, sending a String will not work.
> **3.** Ports currently supported *for webhooks*: **443, 80, 88, 8443**.
>
> If you're having any trouble setting up webhooks, please check out this amazing guide to webhooks.

deleteWebhook

Use this method to remove webhook integration if you decide to switch back to getUpdates. Returns *True* on success.

Parameter	Type	Required	Description
drop_pending_updates	Boolean	Optional	Pass *True* to drop all pending updates

getWebhookInfo

Use this method to get current webhook status. Requires no parameters. On success, returns a WebhookInfo object. If the bot is using getUpdates, will return an object with the *url* field empty.

WebhookInfo

Describes the current status of a webhook.

Field	Type	Description
url	String	Webhook URL, may be empty if webhook is not set up
has_custom_certificate	Boolean	*True*, if a custom certificate was provided for webhook certificate checks
pending_update_count	Integer	Number of updates awaiting delivery
ip_address	String	*Optional*. Currently used webhook IP address
last_error_date	Integer	*Optional*. Unix time for the most recent error that happened when trying to deliver an update via webhook
last_error_message	String	*Optional*. Error message in human-readable format for the most recent error that happened when trying to deliver an update via webhook
last_synchronization_error_date	Integer	*Optional*. Unix time of the most recent error that happened when trying to synchronize available updates with Telegram datacenters
max_connections	Integer	*Optional*. The maximum allowed number of simultaneous HTTPS connections to the webhook for update delivery
allowed_updates	Array of String	*Optional*. A list of update types the bot is subscribed to. Defaults to all update types except *chat_member*, *message_reaction*, and *message_reaction_count*.

Available types

All types used in the Bot API responses are represented as JSON-objects.

It is safe to use 32-bit signed integers for storing all **Integer** fields unless otherwise noted.

> **Optional** fields may be not returned when irrelevant.

User

This object represents a Telegram user or bot.

Field	Type	Description
id	Integer	Unique identifier for this user or bot. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier.
is_bot	Boolean	*True*, if this user is a bot
first_name	String	User's or bot's first name
last_name	String	*Optional*. User's or bot's last name

Field	Type	Description
username	String	*Optional*. User's or bot's username
language_code	String	*Optional*. IETF language tag of the user's language
is_premium	True	*Optional*. *True*, if this user is a Telegram Premium user
added_to_attachment_menu	True	*Optional*. *True*, if this user added the bot to the attachment menu
can_join_groups	Boolean	*Optional*. *True*, if the bot can be invited to groups. Returned only in getMe.
can_read_all_group_messages	Boolean	*Optional*. *True*, if privacy mode is disabled for the bot. Returned only in getMe.
supports_guest_queries	Boolean	*Optional*. *True*, if the bot supports guest queries from chats it is not a member of. Returned only in getMe.
supports_inline_queries	Boolean	*Optional*. *True*, if the bot supports inline queries. Returned only in getMe.
can_connect_to_business	Boolean	*Optional*. *True*, if the bot can be connected to a user account to manage it. Returned only in getMe.
has_main_web_app	Boolean	*Optional*. *True*, if the bot has a main Web App. Returned only in getMe.
has_topics_enabled	Boolean	*Optional*. *True*, if the bot has forum topic mode enabled in private chats. Returned only in getMe.
allows_users_to_create_topics	Boolean	*Optional*. *True*, if the bot allows users to create and delete topics in private chats. Returned only in getMe.
can_manage_bots	Boolean	*Optional*. *True*, if other bots can be created to be controlled by the bot. Returned only in getMe.
supports_join_request_queries	Boolean	*Optional*. *True*, if the bot supports join request queries and can be assigned to process them. Returned only in getMe.

Chat

This object represents a chat.

Field	Type	Description
id	Integer	Unique identifier for this chat. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
type	String	Type of the chat, can be either “private”, “group”, “supergroup” or “channel”
title	String	*Optional*. Title, for supergroups, channels and group chats
username	String	*Optional*. Username, for private chats, supergroups and channels if available
first_name	String	*Optional*. First name of the other party in a private chat

Field	Type	Description
last_name	String	*Optional*. Last name of the other party in a private chat
is_forum	True	*Optional*. *True*, if the supergroup chat is a forum (has topics enabled)
is_direct_messages	True	*Optional*. *True*, if the chat is the direct messages chat of a channel

ChatFullInfo

This object contains full information about a chat.

Field	Type	Description
id	Integer	Unique identifier for this chat. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
type	String	Type of the chat, can be either “private”, “group”, “supergroup” or “channel”
title	String	*Optional*. Title, for supergroups, channels and group chats
username	String	*Optional*. Username, for private chats, supergroups and channels if available
first_name	String	*Optional*. First name of the other party in a private chat
last_name	String	*Optional*. Last name of the other party in a private chat
is_forum	True	*Optional*. *True*, if the supergroup chat is a forum (has topics enabled)
is_direct_messages	True	*Optional*. *True*, if the chat is the direct messages chat of a channel
accent_color_id	Integer	Identifier of the accent color for the chat name and backgrounds of the chat photo, reply header, and link preview. See accent colors for more details.
max_reaction_count	Integer	The maximum number of reactions that can be set on a message in the chat
photo	ChatPhoto	*Optional*. Chat photo
active_usernames	Array of String	*Optional*. If non-empty, the list of all active chat usernames; for private chats, supergroups and channels

Field	Type	Description
birthdate	Birthdate	*Optional*. For private chats, the date of birth of the user
business_intro	BusinessIntro	*Optional*. For private chats with business accounts, the intro of the business
business_location	BusinessLocation	*Optional*. For private chats with business accounts, the location of the business
business_opening_hours	BusinessOpeningHours	*Optional*. For private chats with business accounts, the opening hours of the business
personal_chat	Chat	*Optional*. For private chats, the personal channel of the user
parent_chat	Chat	*Optional*. Information about the corresponding channel chat; for direct messages chats only
available_reactions	Array of ReactionType	*Optional*. List of available reactions allowed in the chat. If omitted, then all emoji reactions are allowed.
background_custom_emoji_id	String	*Optional*. Custom emoji identifier of the emoji chosen by the chat for the reply header and link preview background
profile_accent_color_id	Integer	*Optional*. Identifier of the accent color for the chat's profile background. See profile accent colors for more details.
profile_background_custom_emoji_id	String	*Optional*. Custom emoji identifier of the emoji chosen by the chat for its profile background
emoji_status_custom_emoji_id	String	*Optional*. Custom emoji identifier of the emoji status of the chat or the other party in a private chat
emoji_status_expiration_date	Integer	*Optional*. Expiration date of the emoji status of the chat or the other party in a private chat, in Unix time, if any
bio	String	*Optional*. Bio of the other party in a private chat
has_private_forwards	True	*Optional*. *True*, if privacy settings of the other party in the private chat allows to use `tg://user?id=<user_id>` links only in chats with the user
has_restricted_voice_and_video_messages	True	*Optional*. *True*, if the privacy settings of the other party restrict sending voice and video note messages in the private chat

Field	Type	Description
join_to_send_messages	True	*Optional*. *True*, if users need to join the supergroup before they can send messages
join_by_request	True	*Optional*. *True*, if all users directly joining the supergroup without using an invite link need to be approved by supergroup administrators
description	String	*Optional*. Description, for groups, supergroups and channel chats
invite_link	String	*Optional*. Primary invite link, for groups, supergroups and channel chats
pinned_message	Message	*Optional*. The most recent pinned message (by sending date)
permissions	ChatPermissions	*Optional*. Default chat member permissions, for groups and supergroups
accepted_gift_types	AcceptedGiftTypes	Information about types of gifts that are accepted by the chat or by the corresponding user for private chats
can_send_paid_media	True	*Optional*. *True*, if paid media messages can be sent or forwarded to the channel chat. The field is available only for channel chats.
slow_mode_delay	Integer	*Optional*. For supergroups, the minimum allowed delay between consecutive messages sent by each unprivileged user; in seconds
unrestrict_boost_count	Integer	*Optional*. For supergroups, the minimum number of boosts that a non-administrator user needs to add in order to ignore slow mode and chat permissions
message_auto_delete_time	Integer	*Optional*. The time after which all messages sent to the chat will be automatically deleted; in seconds
has_aggressive_anti_spam_enabled	True	*Optional*. *True*, if aggressive anti-spam checks are enabled in the supergroup. The field is only available to chat administrators.
has_hidden_members	True	*Optional*. *True*, if non-administrators can only get the list of bots and administrators in the chat
has_protected_content	True	*Optional*. *True*, if messages from the chat can't be forwarded to other chats

Field	Type	Description
has_visible_history	True	*Optional*. *True*, if new chat members will have access to old messages; available only to chat administrators
sticker_set_name	String	*Optional*. For supergroups, name of the group sticker set
can_set_sticker_set	True	*Optional*. *True*, if the bot can change the group sticker set
custom_emoji_sticker_set_name	String	*Optional*. For supergroups, the name of the group's custom emoji sticker set. Custom emoji from this set can be used by all users and bots in the group.
linked_chat_id	Integer	*Optional*. Unique identifier for the linked chat, i.e. the discussion group identifier for a channel and vice versa; for supergroups and channel chats. This identifier may be greater than 32 bits and some programming languages may have difficulty/silent defects in interpreting it. But it is smaller than 52 bits, so a signed 64 bit integer or double-precision float type are safe for storing this identifier.
location	ChatLocation	*Optional*. For supergroups, the location to which the supergroup is connected
rating	UserRating	*Optional*. For private chats, the rating of the user if any
first_profile_audio	Audio	*Optional*. For private chats, the first audio added to the profile of the user
unique_gift_colors	UniqueGiftColors	*Optional*. The color scheme based on a unique gift that must be used for the chat's name, message replies and link previews
paid_message_star_count	Integer	*Optional*. The number of Telegram Stars a general user has to pay to send a message to the chat
guard_bot	User	*Optional*. The bot that processes join request queries in the chat. The field is only available to chat administrators.
community	Community	*Optional*. The Community to which the chat belongs
ton_wallet_card	TonWalletCard	*Optional*. For private chats, the TON wallet card of the user, which is shown on their profile page

Field	Type	Description
share_theme	ChatShareTheme	*Optional*. For private chats, supergroups and channels, the theme of the screen for sharing a link to the user or the chat; omitted if the default theme is used

Message

This object represents a message.

Field	Type	Description
message_id	Integer	Unique message identifier inside this chat; 0 for ephemeral messages. In specific instances (e.g., a message containing a video sent to a big chat), the server might automatically schedule a message instead of sending it immediately. In such cases, this field will be 0 and the relevant message will be unusable until it is actually sent.
message_thread_id	Integer	*Optional*. Unique identifier of a message thread or forum topic to which the message belongs; for supergroups and private chats only
direct_messages_topic	DirectMessagesTopic	*Optional*. Information about the direct messages chat topic that contains the message
from	User	*Optional*. Sender of the message; may be empty for messages sent to channels. For backward compatibility, if the message was sent on behalf of a chat, the field contains a fake sender user in non-channel chats.
sender_chat	Chat	*Optional*. Sender of the message when sent on behalf of a chat. For example, the supergroup itself for messages sent by its anonymous administrators or a linked channel for messages automatically forwarded to the channel's discussion group. For backward compatibility, if the message was sent on behalf of a chat, the field *from* contains a fake sender user in non-channel chats.

Field	Type	Description
sender_boost_count	Integer	*Optional*. If the sender of the message boosted the chat, the number of boosts added by the user
sender_business_bot	User	*Optional*. The bot that actually sent the message on behalf of the business account. Available only for outgoing messages sent on behalf of the connected business account.
sender_tag	String	*Optional*. Tag or custom title of the sender of the message; for supergroups only
receiver_user	User	*Optional*. For ephemeral messages, the user who received the message
ephemeral_message_id	Integer	*Optional*. For ephemeral messages, identifier of the ephemeral message inside this chat. The identifier may be reused for another ephemeral message after the message is deleted or expires.
date	Integer	Date the message was sent in Unix time. It is always a positive number, representing a valid date.
guest_query_id	String	*Optional*. The unique identifier for the guest query. Use this identifier with the method answerGuestQuery to send a response message. If non-empty, the message belongs to the chat where the guest bot was summoned, which may not coincide with other existing bot chats sharing the same identifier.
business_connection_id	String	*Optional*. Unique identifier of the business connection from which the message was received. If non-empty, the message belongs to a chat of the corresponding business account that is independent from any potential bot chat which might share the same identifier.
chat	Chat	Chat the message belongs to
forward_origin	MessageOrigin	*Optional*. Information about the original message for forwarded messages

Field	Type	Description
is_topic_message	True	*Optional*. *True*, if the message is sent to a topic in a forum supergroup or a private chat with the bot
is_automatic_forward	True	*Optional*. *True*, if the message is a channel post that was automatically forwarded to the connected discussion group
reply_to_message	Message	*Optional*. For replies in the same chat and message thread, the original message. Note that the Message object in this field will not contain further *reply_to_message* fields even if it itself is a reply. If the message is a reply to an ephemeral message, then this field may be omitted.
external_reply	ExternalReplyInfo	*Optional*. Information about the message that is being replied to, which may come from another chat or forum topic
quote	TextQuote	*Optional*. For replies that quote part of the original message, the quoted part of the message
reply_to_story	Story	*Optional*. For replies to a story, the original story
reply_to_checklist_task_id	Integer	*Optional*. Identifier of the specific checklist task that is being replied to
reply_to_poll_option_id	String	*Optional*. Persistent identifier of the specific poll option that is being replied to
via_bot	User	*Optional*. Bot through which the message was sent
guest_bot_caller_user	User	*Optional*. For a message sent by a guest bot, this is the user whose original message triggered the bot's response
guest_bot_caller_chat	Chat	*Optional*. For a message sent by a guest bot, this is the chat whose original message triggered the bot's response
edit_date	Integer	*Optional*. Date the message was last edited in Unix time

Field	Type	Description
has_protected_content	True	*Optional*. *True*, if the message can't be forwarded
is_from_offline	True	*Optional*. *True*, if the message was sent by an implicit action, for example, as an away or a greeting business message, or as a scheduled message
is_paid_post	True	*Optional*. *True*, if the message is a paid post. Note that such posts must not be deleted for 24 hours to receive the payment and can't be edited.
media_group_id	String	*Optional*. The unique identifier inside this chat of a media message group this message belongs to
author_signature	String	*Optional*. Signature of the post author for messages in channels, or the custom title of an anonymous group administrator
paid_star_count	Integer	*Optional*. The number of Telegram Stars that were paid by the sender of the message to send it
text	String	*Optional*. For text messages, the actual UTF-8 text of the message
entities	Array of MessageEntity	*Optional*. For text messages, special entities like usernames, URLs, bot commands, etc. that appear in the text
link_preview_options	LinkPreviewOptions	*Optional*. Options used for link preview generation for the message, if it is a text message and link preview options were changed
suggested_post_info	SuggestedPostInfo	*Optional*. Information about suggested post parameters if the message is a suggested post in a channel direct messages chat. If the message is an approved or declined suggested post, then it can't be edited.
effect_id	String	*Optional*. Unique identifier of the message effect added to the message
rich_message	RichMessage	*Optional*. Message is a rich formatted message

Field	Type	Description
animation	Animation	*Optional*. Message is an animation, information about the animation. For backward compatibility, when this field is set, the *document* field will also be set.
audio	Audio	*Optional*. Message is an audio file, information about the file
document	Document	*Optional*. Message is a general file, information about the file
live_photo	LivePhoto	*Optional*. Message is a live photo, information about the live photo. For backward compatibility, when this field is set, the *photo* field will also be set.
paid_media	PaidMediaInfo	*Optional*. Message contains paid media; information about the paid media
photo	Array of PhotoSize	*Optional*. Message is a photo, available sizes of the photo
sticker	Sticker	*Optional*. Message is a sticker, information about the sticker
story	Story	*Optional*. Message is a forwarded story
video	Video	*Optional*. Message is a video, information about the video
video_note	VideoNote	*Optional*. Message is a video note, information about the video message
voice	Voice	*Optional*. Message is a voice message, information about the file
widget	Widget	*Optional*. Message is an HTML widget, information about the widget
caption	String	*Optional*. Caption for the animation, audio, document, live photo, paid media, photo, video, voice, or widget
caption_entities	Array of MessageEntity	*Optional*. For messages with a caption, special entities like usernames, URLs, bot commands, etc. that appear in the caption

Field	Type	Description
show_caption_above_media	True	*Optional*. *True*, if the caption must be shown above the message media or the widget
has_media_spoiler	True	*Optional*. *True*, if the message media is covered by a spoiler animation
checklist	Checklist	*Optional*. Message is a checklist
contact	Contact	*Optional*. Message is a shared contact, information about the contact
dice	Dice	*Optional*. Message is a dice with random value
game	Game	*Optional*. Message is a game, information about the game. More about games »
poll	Poll	*Optional*. Message is a native poll, information about the poll
venue	Venue	*Optional*. Message is a venue, information about the venue. For backward compatibility, when this field is set, the *location* field will also be set.
location	Location	*Optional*. Message is a shared location, information about the location
new_chat_members	Array of User	*Optional*. New members that were added to the group or supergroup and information about them (the bot itself may be one of these members)
left_chat_member	User	*Optional*. A member was removed from the group, information about them (this member may be the bot itself)
chat_owner_left	ChatOwnerLeft	*Optional*. Service message: chat owner has left
chat_owner_changed	ChatOwnerChanged	*Optional*. Service message: chat owner has changed
new_chat_title	String	*Optional*. A chat title was changed to this value
new_chat_photo	Array of PhotoSize	*Optional*. A chat photo was change to this value

Field	Type	Description
delete_chat_photo	True	*Optional*. Service message: the chat photo was deleted
group_chat_created	True	*Optional*. Service message: the group has been created
supergroup_chat_created	True	*Optional*. Service message: the supergroup has been created. This field can't be received in a message coming through updates, because bot can't be a member of a supergroup when it is created. It can only be found in reply_to_message if someone replies to a very first message in a directly created supergroup.
channel_chat_created	True	*Optional*. Service message: the channel has been created. This field can't be received in a message coming through updates, because bot can't be a member of a channel when it is created. It can only be found in reply_to_message if someone replies to a very first message in a channel.
message_auto_delete_timer_changed	MessageAutoDeleteTimerChanged	*Optional*. Service message: auto-delete timer settings changed in the chat
migrate_to_chat_id	Integer	*Optional*. The group has been migrated to a supergroup with the specified identifier. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
migrate_from_chat_id	Integer	*Optional*. The supergroup has been migrated from a group with the specified identifier. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.

Field	Type	Description
pinned_message	MaybeInaccessibleMessage	*Optional*. Specified message was pinned. Note that the Message object in this field will not contain further *reply_to_message* fields even if it itself is a reply.
invoice	Invoice	*Optional*. Message is an invoice for a payment, information about the invoice. More about payments »
successful_payment	SuccessfulPayment	*Optional*. Message is a service message about a successful payment, information about the payment. More about payments »
refunded_payment	RefundedPayment	*Optional*. Message is a service message about a refunded payment, information about the payment. More about payments »
users_shared	UsersShared	*Optional*. Service message: users were shared with the bot
chat_shared	ChatShared	*Optional*. Service message: a chat was shared with the bot
gift	GiftInfo	*Optional*. Service message: a regular gift was sent or received
unique_gift	UniqueGiftInfo	*Optional*. Service message: a unique gift was sent or received
gift_upgrade_sent	GiftInfo	*Optional*. Service message: upgrade of a gift was purchased after the gift was sent
connected_website	String	*Optional*. The domain name of the website on which the user has logged in. More about Telegram Login »
write_access_allowed	WriteAccessAllowed	*Optional*. Service message: the user allowed the bot to write messages after adding it to the attachment or side menu, launching a Web App from a link, or accepting an explicit request from a Web App sent by the method requestWriteAccess
passport_data	PassportData	*Optional*. Telegram Passport data
proximity_alert_triggered	ProximityAlertTriggered	*Optional*. Service message: a user in the chat triggered another user's proximity alert while sharing Live Location

Field	Type	Description
boost_added	ChatBoostAdded	*Optional*. Service message: user boosted the chat
chat_background_set	ChatBackground	*Optional*. Service message: chat background set
checklist_tasks_done	ChecklistTasksDone	*Optional*. Service message: some tasks in a checklist were marked as done or not done
checklist_tasks_added	ChecklistTasksAdded	*Optional*. Service message: tasks were added to a checklist
community_chat_added	CommunityChatAdded	*Optional*. Service message: chat or bot added to a Community
community_chat_joined	CommunityChatJoined	*Optional*. Service message: chat was joined by a user from a Community
community_chat_removed	CommunityChatRemoved	*Optional*. Service message: chat or bot removed from a Community
direct_message_price_changed	DirectMessagePriceChanged	*Optional*. Service message: the price for paid messages in the corresponding direct messages chat of a channel has changed
forum_topic_created	ForumTopicCreated	*Optional*. Service message: forum topic created
forum_topic_edited	ForumTopicEdited	*Optional*. Service message: forum topic edited
forum_topic_closed	ForumTopicClosed	*Optional*. Service message: forum topic closed
forum_topic_reopened	ForumTopicReopened	*Optional*. Service message: forum topic reopened
general_forum_topic_hidden	GeneralForumTopicHidden	*Optional*. Service message: the 'General' forum topic hidden
general_forum_topic_unhidden	GeneralForumTopicUnhidden	*Optional*. Service message: the 'General' forum topic unhidden
giveaway_created	GiveawayCreated	*Optional*. Service message: a scheduled giveaway was created
giveaway	Giveaway	*Optional*. The message is a scheduled giveaway message
giveaway_winners	GiveawayWinners	*Optional*. A giveaway with public winners was completed

Field	Type	Description
giveaway_completed	GiveawayCompleted	*Optional*. Service message: a giveaway without public winners was completed
managed_bot_created	ManagedBotCreated	*Optional*. Service message: user created a bot that will be managed by the current bot
paid_message_price_changed	PaidMessagePriceChanged	*Optional*. Service message: the price for paid messages has changed in the chat
paid_messages_waived	PaidMessagesWaived	*Optional*. Service message: the sender of the message used a link of the other user, which allows sending messages to the user without paying Telegram Stars
poll_option_added	PollOptionAdded	*Optional*. Service message: answer option was added to a poll
poll_option_deleted	PollOptionDeleted	*Optional*. Service message: answer option was deleted from a poll
suggested_post_approved	SuggestedPostApproved	*Optional*. Service message: a suggested post was approved
suggested_post_approval_failed	SuggestedPostApprovalFailed	*Optional*. Service message: approval of a suggested post has failed
suggested_post_declined	SuggestedPostDeclined	*Optional*. Service message: a suggested post was declined
suggested_post_paid	SuggestedPostPaid	*Optional*. Service message: payment for a suggested post was received
suggested_post_refunded	SuggestedPostRefunded	*Optional*. Service message: payment for a suggested post was refunded
ton_wallet_transfer	TonWalletTransfer	*Optional*. Service message: a transfer with a TON wallet
video_chat_scheduled	VideoChatScheduled	*Optional*. Service message: video chat scheduled
video_chat_started	VideoChatStarted	*Optional*. Service message: video chat started
video_chat_ended	VideoChatEnded	*Optional*. Service message: video chat ended
video_chat_participants_invited	VideoChatParticipantsInvited	*Optional*. Service message: new participants invited to a video chat
web_app_data	WebAppData	*Optional*. Service message: data sent by a Web App

Field	Type	Description
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message. `login_url` buttons are represented as ordinary `url` buttons.

MessageId

This object represents a unique message identifier.

Field	Type	Description
message_id	Integer	Unique message identifier. In specific instances (e.g., message containing a video sent to a big chat), the server might automatically schedule a message instead of sending it immediately. In such cases, this field will be 0 and the relevant message will be unusable until it is actually sent.

InaccessibleMessage

This object describes a message that was deleted or is otherwise inaccessible to the bot.

Field	Type	Description
chat	Chat	Chat the message belonged to
message_id	Integer	Unique message identifier inside the chat
date	Integer	Always 0. The field can be used to differentiate regular and inaccessible messages.

MaybeInaccessibleMessage

This object describes a message that can be inaccessible to the bot. It can be one of

- Message
- InaccessibleMessage

MessageEntity

This object represents one special entity in a text message. For example, hashtags, usernames, URLs, etc.

Field	Type	Description
type	String	Type of the entity. Currently, can be “mention” (`@username`), “hashtag” (`#hashtag` or `#hashtag@chatusername`), “cashtag” (`$USD` or `$USD@chatusername`), “bot_command” (`/start@jobs_bot`), “url” (`https://telegram.org`), “email” (`do-not-reply@telegram.org`), “phone_number” (`+1-212-555-0123`), “bank_card_number” (bank card number), “ton_address” (address in the TON blockchain), “bold” (**bold text**), “italic” (*italic text*), “underline” (underlined text), “strikethrough” (strikethrough text), “spoiler” (spoiler message), “blockquote” (block quotation), “expandable_blockquote” (collapsed-by-default block quotation), “code” (monowidth string), “pre” (monowidth block), “text_link” (for clickable text URLs), “text_mention” (for users without usernames), “custom_emoji” (for inline custom emoji stickers), or “date_time” (for formatted date and time).
offset	Integer	Offset in UTF-16 code units to the start of the entity
length	Integer	Length of the entity in UTF-16 code units
url	String	*Optional*. For “text_link” only, URL that will be opened after user taps on the text
user	User	*Optional*. For “text_mention” only, the mentioned user
language	String	*Optional*. For “pre” only, the programming language of the entity text
custom_emoji_id	String	*Optional*. For “custom_emoji” only, unique identifier of the custom emoji. Use getCustomEmojiStickers to get full information about the sticker.
unix_time	Integer	*Optional*. For “date_time” only, the Unix time associated with the entity
date_time_format	String	*Optional*. For “date_time” only, the string that defines the formatting of the date and time. See date-time entity formatting for more details.

TextQuote

This object contains information about the quoted part of a message that is replied to by the given message.

Field	Type	Description
text	String	Text of the quoted part of a message that is replied to by the given message; may be empty if the quote was chosen from a rich message
entities	Array of MessageEntity	*Optional*. Special entities that appear in the quote. Currently, only *bold*, *italic*, *underline*, *strikethrough*, *spoiler*, *custom_emoji*, and *date_time* entities are kept in quotes.
rich_texts	Array of RichText	*Optional*. Rich text of the quote if it was chosen from a rich message. May contain only plain text, RichTextBold, RichTextItalic, RichTextUnderline, RichTextStrikethrough, RichTextSpoiler, RichTextSubscript, RichTextSuperscript, RichTextDateTime, RichTextCustomEmoji, RichTextMathematicalExpression, and arrays of them.
position	Integer	Approximate quote position in the original message in UTF-16 code units as specified by the sender. For quotes from rich messages, it is the position in the text formed by joining plain texts of all quotable texts of the message, separated by line feeds.

Field	Type	Description
is_manual	True	*Optional*. *True*, if the quote was chosen manually by the message sender. Otherwise, the quote was added automatically by the server.

ExternalReplyInfo

This object contains information about a message that is being replied to, which may come from another chat or forum topic.

Field	Type	Description
origin	MessageOrigin	Origin of the message replied to by the given message
chat	Chat	*Optional*. Chat the original message belongs to. Available only if the chat is a supergroup or a channel.
message_id	Integer	*Optional*. Unique message identifier inside the original chat. Available only if the original chat is a supergroup or a channel.
link_preview_options	LinkPreviewOptions	*Optional*. Options used for link preview generation for the original message, if it is a text message
animation	Animation	*Optional*. Message is an animation, information about the animation
audio	Audio	*Optional*. Message is an audio file, information about the file
document	Document	*Optional*. Message is a general file, information about the file
live_photo	LivePhoto	*Optional*. Message is a live photo, information about the live photo
paid_media	PaidMediaInfo	*Optional*. Message contains paid media; information about the paid media
photo	Array of PhotoSize	*Optional*. Message is a photo, available sizes of the photo
sticker	Sticker	*Optional*. Message is a sticker, information about the sticker
story	Story	*Optional*. Message is a forwarded story
video	Video	*Optional*. Message is a video, information about the video
video_note	VideoNote	*Optional*. Message is a video note, information about the video message
voice	Voice	*Optional*. Message is a voice message, information about the file
widget	Widget	*Optional*. Message is an HTML widget, information about the widget
has_media_spoiler	True	*Optional*. *True*, if the message media is covered by a spoiler animation
checklist	Checklist	*Optional*. Message is a checklist
contact	Contact	*Optional*. Message is a shared contact, information about the contact

Field	Type	Description
dice	Dice	*Optional*. Message is a dice with random value
game	Game	*Optional*. Message is a game, information about the game. More about games »
giveaway	Giveaway	*Optional*. Message is a scheduled giveaway, information about the giveaway
giveaway_winners	GiveawayWinners	*Optional*. A giveaway with public winners was completed
invoice	Invoice	*Optional*. Message is an invoice for a payment, information about the invoice. More about payments »
location	Location	*Optional*. Message is a shared location, information about the location
poll	Poll	*Optional*. Message is a native poll, information about the poll
venue	Venue	*Optional*. Message is a venue, information about the venue

ReplyParameters

Describes reply parameters for the message that is being sent.

Field	Type	Description
message_id	Integer	*Optional*. Identifier of the message that will be replied to in the current chat, or in the chat *chat_id* if it is specified. Required if *ephemeral_message_id* isn't specified.
chat_id	Integer or String	*Optional*. If the message to be replied to is from a different chat, unique identifier for the chat or username of the bot, supergroup, or channel in the format `@username`. Not supported for messages sent on behalf of a business account, messages from channel direct messages chats and ephemeral messages.
ephemeral_message_id	Integer	*Optional*. Identifier of the incoming ephemeral message that will be replied to in the current chat. A reply to an ephemeral message must itself be an ephemeral message. An ephemeral message may only be replied to within 15 seconds of being sent. Required if *message_id* isn't specified.
allow_sending_without_reply	Boolean	*Optional*. Pass *True* if the message should be sent even if the specified message to be replied to is not found. Always *False* for replies in another chat or forum topic, and sent ephemeral messages. Always *True* for messages sent on behalf of a business account.
quote	String	*Optional*. Quoted part of the message to be replied to; 0-1024 characters after entities parsing. The quote must be an exact substring of the message to be replied to, including *bold*, *italic*, *underline*, *strikethrough*, *spoiler*, *custom_emoji*, and *date_time* entities. The message will fail to send if the quote isn't found in the original message. Ignored for ephemeral messages.

Field	Type	Description
quote_parse_mode	String	*Optional*. Mode for parsing entities in the quote. See formatting options for more details.
quote_entities	Array of MessageEntity	*Optional*. A JSON-serialized list of special entities that appear in the quote. It can be specified instead of *quote_parse_mode*.
quote_rich_texts	Array of RichText	*Optional*. A JSON-serialized list of quoted parts of the rich message to be replied to. Must contain one element for each quoted text of the message in the order of their appearance. Each of the following is a separate quoted text: the text of a paragraph, a heading, a preformatted block, a footer, or a mathematical expression; the text and the credit of a block quotation or a pull quotation; the summary of a details block; the caption and each cell of a table; the text and the credit of a caption of a media block, a collage, a slideshow, or a map. Texts of blocks nested in lists, block quotations, details, collages, and slideshows are separate quoted texts too. The first and the last elements can contain only a part of the corresponding text; all other elements must contain the whole text. Formatting of the quoted parts must be kept, and rich texts must be nested in the same way as in the message received from the server. Mathematical expressions must be quoted entirely. Buttons in RichBlockButtons, anchors and dividers can't be quoted. Up to 32 quoted texts with total length up to 1024 characters can be specified. Only plain text, RichTextBold, RichTextItalic, RichTextUnderline, RichTextStrikethrough, RichTextSpoiler, RichTextSubscript, RichTextSuperscript, RichTextDateTime, RichTextCustomEmoji, RichTextMathematicalExpression, and arrays of them are kept in the quote; other rich text types are replaced with their text. If non-empty, then *quote*, *quote_parse_mode* and *quote_entities* are ignored.
quote_position	Integer	*Optional*. Position of the quote in the original message in UTF-16 code units. For quotes from rich messages, it is the position in the text formed by joining plain texts of all quotable texts of the message, separated by line feeds.
checklist_task_id	Integer	*Optional*. Identifier of the specific checklist task to be replied to
poll_option_id	String	*Optional*. Persistent identifier of the specific poll option to be replied to

EphemeralMessageParameters

Field	Type	Description
receiver_user_id	Integer	*Optional*. Identifier of the user who will receive the message; required unless the message is sent as an ephemeral reply to a callback query. If omitted, then the parameter *chat_id* of the method is ignored. If specified together with *callback_query_id*, then it must be the identifier of the user who pressed the button. Must be omitted for messages sent in chats where the bot isn't a member, unless the message is a reply to an ephemeral message. It is not guaranteed that the user will receive the message, especially if they are offline. See here for more details.
callback_query_id	String	*Optional*. Identifier of the callback query which triggered the message, if any. If specified and *receiver_user_id* is omitted, then the message is sent as an ephemeral reply to the callback query and will be shown only to the user who pressed the button. Ephemeral replies can't be sent to callback queries from messages in channel direct messages chats. The bot must still answer the callback query using answerCallbackQuery.
replace_callback_query_message	Boolean	*Optional*. Pass *True* if the ephemeral message must be shown in place of the original message. Must be *False* for callback queries from ephemeral messages, which must be edited using regular *editEphemeralMessage...* methods.

MessageOrigin

This object describes the origin of a message. It can be one of

- MessageOriginUser
- MessageOriginHiddenUser
- MessageOriginChat
- MessageOriginChannel

MessageOriginUser

The message was originally sent by a known user.

Field	Type	Description
type	String	Type of the message origin, always “user”
date	Integer	Date the message was sent originally in Unix time
sender_user	User	User that sent the message originally

MessageOriginHiddenUser

The message was originally sent by an unknown user.

Field	Type	Description
type	String	Type of the message origin, always “hidden_user”
date	Integer	Date the message was sent originally in Unix time
sender_user_name	String	Name of the user that sent the message originally

MessageOriginChat

The message was originally sent on behalf of a chat to a group chat.

Field	Type	Description
type	String	Type of the message origin, always “chat”
date	Integer	Date the message was sent originally in Unix time
sender_chat	Chat	Chat that sent the message originally
author_signature	String	*Optional*. For messages originally sent by an anonymous chat administrator, original message author signature

MessageOriginChannel

The message was originally sent to a channel chat.

Field	Type	Description
type	String	Type of the message origin, always “channel”
date	Integer	Date the message was sent originally in Unix time
chat	Chat	Channel chat to which the message was originally sent
message_id	Integer	Unique message identifier inside the chat
author_signature	String	*Optional*. Signature of the original post author

PhotoSize

This object represents one size of a photo or a file / sticker thumbnail.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
width	Integer	Photo width
height	Integer	Photo height
file_size	Integer	*Optional*. File size in bytes

Animation

This object represents an animation file (GIF or H.264/MPEG-4 AVC video without sound).

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
width	Integer	Video width as defined by the sender
height	Integer	Video height as defined by the sender
duration	Integer	Duration of the video in seconds as defined by the sender
thumbnail	PhotoSize	*Optional*. Animation thumbnail as defined by the sender
file_name	String	*Optional*. Original animation filename as defined by the sender
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

Audio

This object represents an audio file to be treated as music by the Telegram clients.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
duration	Integer	Duration of the audio in seconds as defined by the sender
performer	String	*Optional*. Performer of the audio as defined by the sender or by audio tags
title	String	*Optional*. Title of the audio as defined by the sender or by audio tags
file_name	String	*Optional*. Original filename as defined by the sender
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.
thumbnail	PhotoSize	*Optional*. Thumbnail of the album cover to which the music file belongs

Document

This object represents a general file (as opposed to photos, voice messages and audio files).

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
thumbnail	PhotoSize	*Optional*. Document thumbnail as defined by the sender
file_name	String	*Optional*. Original filename as defined by the sender
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

LivePhoto

This object represents a live photo.

Field	Type	Description
photo	Array of PhotoSize	*Optional*. Available sizes of the corresponding static photo
file_id	String	Identifier for the video file which can be used to download or reuse the file
file_unique_id	String	Unique identifier for the video file which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
width	Integer	Video width as defined by the sender
height	Integer	Video height as defined by the sender
duration	Integer	Duration of the video in seconds as defined by the sender
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

Story

This object represents a story.

Field	Type	Description
chat	Chat	Chat that posted the story

Field	Type	Description
id	Integer	Unique identifier for the story in the chat

VideoQuality

This object represents a video file of a specific quality.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
width	Integer	Video width
height	Integer	Video height
codec	String	Codec that was used to encode the video, for example, “h264”, “h265”, or “av01”
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

Video

This object represents a video file.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
width	Integer	Video width as defined by the sender
height	Integer	Video height as defined by the sender
duration	Integer	Duration of the video in seconds as defined by the sender
thumbnail	PhotoSize	*Optional*. Video thumbnail
cover	Array of PhotoSize	*Optional*. Available sizes of the cover of the video in the message
start_timestamp	Integer	*Optional*. Timestamp in seconds from which the video will play in the message
qualities	Array of VideoQuality	*Optional*. List of available qualities of the video
file_name	String	*Optional*. Original filename as defined by the sender

Field	Type	Description
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

VideoNote

This object represents a video message.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
length	Integer	Video width and height (diameter of the video message) as defined by the sender
duration	Integer	Duration of the video in seconds as defined by the sender
thumbnail	PhotoSize	*Optional*. Video thumbnail
file_size	Integer	*Optional*. File size in bytes

Voice

This object represents a voice note.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
duration	Integer	Duration of the audio in seconds as defined by the sender
mime_type	String	*Optional*. MIME type of the file as defined by the sender
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.

Widget

This object describes an HTML widget.

Field	Type	Description
id	String	*Optional*. Unique identifier of the shared widget, which can be used to send the widget again using the method sendSharedWidget; omitted if the widget isn't shared
width	Integer	*Optional*. Width of the widget in logical pixels; omitted if the widget must occupy the whole width of the message bubble
height	Integer	*Optional*. Height of the widget in logical pixels; omitted if the height of the widget isn't fixed
html	String	HTML code of the widget
public_data	String	*Optional*. Public data of the widget, which is available wherever the widget is shown. All messages with a shared widget have the same public data.
chat_data	String	*Optional*. Data of the widget, which is specific to the message

PaidMediaInfo

Describes the paid media added to a message.

Field	Type	Description
star_count	Integer	The number of Telegram Stars that must be paid to buy access to the media
paid_media	Array of PaidMedia	Information about the paid media

PaidMedia

This object describes paid media. Currently, it can be one of

- PaidMediaLivePhoto
- PaidMediaPhoto
- PaidMediaPreview
- PaidMediaVideo

PaidMediaLivePhoto

The paid media is a live photo.

Field	Type	Description
type	String	Type of the paid media, always “live_photo”
live_photo	LivePhoto	The photo

PaidMediaPhoto

The paid media is a photo.

Field	Type	Description
type	String	Type of the paid media, always “photo”
photo	Array of PhotoSize	The photo

PaidMediaPreview

The paid media isn't available before the payment.

Field	Type	Description
type	String	Type of the paid media, always “preview”
width	Integer	*Optional*. Media width as defined by the sender
height	Integer	*Optional*. Media height as defined by the sender
duration	Integer	*Optional*. Duration of the media in seconds as defined by the sender

PaidMediaVideo

The paid media is a video.

Field	Type	Description
type	String	Type of the paid media, always “video”
video	Video	The video

Contact

This object represents a phone contact.

Field	Type	Description
phone_number	String	Contact's phone number
first_name	String	Contact's first name
last_name	String	*Optional*. Contact's last name
user_id	Integer	*Optional*. Contact's user identifier in Telegram. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier.
vcard	String	*Optional*. Additional data about the contact in the form of a vCard

Dice

This object represents an animated emoji that displays a random value.

Field	Type	Description
emoji	String	Emoji on which the dice throw animation is based
value	Integer	Value of the dice, 1-6 for “🎲”, “🎯” and “🎳” base emoji, 1-5 for “🏀” and “⚽” base emoji, 1-64 for “🎰” base emoji

Link

Represents an HTTP link.

Field	Type	Description
url	String	URL of the link

PollMedia

At most **one** of the optional fields can be present in any given object.

Field	Type	Description
animation	Animation	*Optional*. Media is an animation, information about the animation
audio	Audio	*Optional*. Media is an audio file, information about the file; currently, can't be received in a poll option
document	Document	*Optional*. Media is a general file, information about the file; currently, can't be received in a poll option
link	Link	*Optional*. The HTTP link attached to the poll option
live_photo	LivePhoto	*Optional*. Media is a live photo, information about the live photo
location	Location	*Optional*. Media is a shared location, information about the location
photo	Array of PhotoSize	*Optional*. Media is a photo, available sizes of the photo
sticker	Sticker	*Optional*. Media is a sticker, information about the sticker; currently, for poll options only
venue	Venue	*Optional*. Media is a venue, information about the venue
video	Video	*Optional*. Media is a video, information about the video

InputPollMedia

This object represents the content of a poll description or a quiz explanation to be sent. It should be one of

- InputMediaAnimation
- InputMediaAudio
- InputMediaDocument
- InputMediaLivePhoto

- InputMediaLocation
- InputMediaPhoto
- InputMediaVenue
- InputMediaVideo

InputPollOptionMedia

This object represents the content of a poll option to be sent. It should be one of

- InputMediaAnimation
- InputMediaLink
- InputMediaLivePhoto
- InputMediaLocation
- InputMediaPhoto
- InputMediaSticker
- InputMediaVenue
- InputMediaVideo

PollOption

This object contains information about one answer option in a poll.

Field	Type	Description
persistent_id	String	Unique identifier of the option, persistent on option addition and deletion
text	String	Option text, 1-100 characters
text_entities	Array of MessageEntity	*Optional*. Special entities that appear in the option *text*. Currently, only custom emoji entities are allowed in poll option texts
media	PollMedia	*Optional*. Media added to the poll option
voter_count	Integer	Number of users who voted for this option; may be 0 if unknown
added_by_user	User	*Optional*. User who added the option; omitted if the option wasn't added by a user after poll creation
added_by_chat	Chat	*Optional*. Chat that added the option; omitted if the option wasn't added by a chat after poll creation
addition_date	Integer	*Optional*. Point in time (Unix timestamp) when the option was added; omitted if the option existed in the original poll

InputPollOption

This object contains information about one answer option in a poll to be sent.

Field	Type	Description
text	String	Option text, 1-100 characters
text_parse_mode	String	*Optional*. Mode for parsing entities in the text. See formatting options for more details. Currently, only custom emoji entities are allowed.

Field	Type	Description
text_entities	Array of MessageEntity	*Optional*. A JSON-serialized list of special entities that appear in the poll option text. It can be specified instead of *text_parse_mode*.
media	InputPollOptionMedia	*Optional*. Media added to the poll option

PollAnswer

This object represents an answer of a user in a non-anonymous poll.

Field	Type	Description
poll_id	String	Unique poll identifier
voter_chat	Chat	*Optional*. The chat that changed the answer to the poll, if the voter is anonymous
user	User	*Optional*. The user that changed the answer to the poll, if the voter isn't anonymous
option_ids	Array of Integer	0-based identifiers of chosen answer options. May be empty if the vote was retracted.
option_persistent_ids	Array of String	Persistent identifiers of the chosen answer options. May be empty if the vote was retracted.

Poll

This object contains information about a poll.

Field	Type	Description
id	String	Unique poll identifier
question	String	Poll question, 1-300 characters
question_entities	Array of MessageEntity	*Optional*. Special entities that appear in the *question*. Currently, only custom emoji entities are allowed in poll questions
options	Array of PollOption	List of poll options
total_voter_count	Integer	Total number of users that voted in the poll
is_closed	Boolean	*True*, if the poll is closed
is_anonymous	Boolean	*True*, if the poll is anonymous
type	String	Poll type, currently can be “regular” or “quiz”
allows_multiple_answers	Boolean	*True*, if the poll allows multiple answers
allows_revoting	Boolean	*True*, if the poll allows to change the chosen answer options

Field	Type	Description
members_only	Boolean	*True* if voting is limited to users who have been members of the chat where the poll was originally sent for more than 24 hours
country_codes	Array of String	*Optional*. A list of two-letter ISO 3166-1 alpha-2 country codes indicating the countries from which users can vote in the poll. The country code “FT” is used for users with anonymous numbers. If omitted, then users from any country can participate in the poll.
correct_option_ids	Array of Integer	*Optional*. Array of 0-based identifiers of the correct answer options. Available only for polls in quiz mode which are closed or were sent (not forwarded) by the bot or to the private chat with the bot.
explanation	String	*Optional*. Text that is shown when a user chooses an incorrect answer or taps on the lamp icon in a quiz-style poll, 0-200 characters
explanation_entities	Array of MessageEntity	*Optional*. Special entities like usernames, URLs, bot commands, etc. that appear in the *explanation*
explanation_media	PollMedia	*Optional*. Media added to the quiz explanation
open_period	Integer	*Optional*. Amount of time in seconds the poll will be active after creation
close_date	Integer	*Optional*. Point in time (Unix timestamp) when the poll will be automatically closed
description	String	*Optional*. Description of the poll; for polls inside the Message object only
description_entities	Array of MessageEntity	*Optional*. Special entities like usernames, URLs, bot commands, etc. that appear in the description
media	PollMedia	*Optional*. Media added to the poll description; for polls inside the Message object only

ChecklistTask

Describes a task in a checklist.

Field	Type	Description
id	Integer	Unique identifier of the task
text	String	Text of the task
text_entities	Array of MessageEntity	*Optional*. Special entities that appear in the task text
completed_by_user	User	*Optional*. User that completed the task; omitted if the task wasn't completed by a user
completed_by_chat	Chat	*Optional*. Chat that completed the task; omitted if the task wasn't completed by a chat

Field	Type	Description
completion_date	Integer	*Optional*. Point in time (Unix timestamp) when the task was completed; 0 if the task wasn't completed

Checklist

Describes a checklist.

Field	Type	Description
title	String	Title of the checklist
title_entities	Array of MessageEntity	*Optional*. Special entities that appear in the checklist title
tasks	Array of ChecklistTask	List of tasks in the checklist
others_can_add_tasks	True	*Optional*. *True*, if users other than the creator of the list can add tasks to the list
others_can_mark_tasks_as_done	True	*Optional*. *True*, if users other than the creator of the list can mark tasks as done or not done

InputChecklistTask

Describes a task to add to a checklist.

Field	Type	Description
id	Integer	Unique identifier of the task; must be positive and unique among all task identifiers currently present in the checklist
text	String	Text of the task; 1-100 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the text. See formatting options for more details.
text_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the text, which can be specified instead of parse_mode. Currently, only *bold*, *italic*, *underline*, *strikethrough*, *spoiler*, *custom_emoji*, and *date_time* entities are allowed.

InputChecklist

Describes a checklist to create.

Field	Type	Description
title	String	Title of the checklist; 1-255 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the title. See formatting options for more details.

Field	Type	Description
title_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the title, which can be specified instead of parse_mode. Currently, only *bold*, *italic*, *underline*, *strikethrough*, *spoiler*, *custom_emoji*, and *date_time* entities are allowed.
tasks	Array of InputChecklistTask	List of 1-30 tasks in the checklist
others_can_add_tasks	Boolean	*Optional*. Pass *True* if other users can add tasks to the checklist
others_can_mark_tasks_as_done	Boolean	*Optional*. Pass *True* if other users can mark tasks as done or not done in the checklist

Location

This object represents a point on the map.

Field	Type	Description
latitude	Float	Latitude as defined by the sender
longitude	Float	Longitude as defined by the sender
horizontal_accuracy	Float	*Optional*. The radius of uncertainty for the location, measured in meters; 0-1500
live_period	Integer	*Optional*. Time relative to the message sending date, during which the location can be updated; in seconds. For active live locations only.
heading	Integer	*Optional*. The direction in which user is moving, in degrees; 1-360. For active live locations only.
proximity_alert_radius	Integer	*Optional*. The maximum distance for proximity alerts about approaching another chat member, in meters. For sent live locations only.

Venue

This object represents a venue.

Field	Type	Description
location	Location	Venue location. Can't be a live location.
title	String	Name of the venue
address	String	Address of the venue
foursquare_id	String	*Optional*. Foursquare identifier of the venue
foursquare_type	String	*Optional*. Foursquare type of the venue. (For example, “arts_entertainment/default”, “arts_entertainment/aquarium” or “food/icecream”.)

Field	Type	Description
google_place_id	String	*Optional*. Google Places identifier of the venue
google_place_type	String	*Optional*. Google Places type of the venue. (See supported types.)

WebAppData

Describes data sent from a Web App to the bot.

Field	Type	Description
data	String	The data. Be aware that a bad client can send arbitrary data in this field.
button_text	String	Text of the *web_app* keyboard button from which the Web App was opened. Be aware that a bad client can send arbitrary data in this field.

ProximityAlertTriggered

This object represents the content of a service message, sent whenever a user in the chat triggers a proximity alert set by another user.

Field	Type	Description
traveler	User	User that triggered the alert
watcher	User	User that set the alert
distance	Integer	The distance between the users

MessageAutoDeleteTimerChanged

This object represents a service message about a change in auto-delete timer settings.

Field	Type	Description
message_auto_delete_time	Integer	New auto-delete time for messages in the chat; in seconds

ManagedBotCreated

This object contains information about the bot that was created to be managed by the current bot.

Field	Type	Description
bot	User	Information about the bot. The bot's token can be fetched using the method getManagedBotToken.

ManagedBotUpdated

This object contains information about the creation, token update, or owner update of a bot that is managed by the current bot.

Field	Type	Description
user	User	User that created the bot
bot	User	Information about the bot. Token of the bot can be fetched using the method getManagedBotToken.

BotSubscriptionUpdated

This object contains information about changes to a user payment subscription toward the current bot.

Field	Type	Description
user	User	User who subscribed for payments toward the bot
invoice_payload	String	Bot-specified invoice payload
state	String	The new state of the subscription. Currently, it can be one of “canceled” if the user canceled the subscription, “active” if the user re-enabled a previously canceled subscription, or “failed” if payment for the subscription failed.

MessageGenerationStopped

This object describes an update about a user stopping message generation.

Field	Type	Description
chat	Chat	Chat in which the message is generated
message_thread_id	Integer	*Optional*. Unique identifier of the message thread in which the message is generated
draft_id	Integer	Unique identifier of the message draft which was stopped

PollOptionAdded

Describes a service message about an option added to a poll.

Field	Type	Description
poll_message	MaybeInaccessibleMessage	*Optional*. Message containing the poll to which the option was added, if known. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
option_persistent_id	String	Unique identifier of the added option
option_text	String	Option text
option_text_entities	Array of MessageEntity	*Optional*. Special entities that appear in the *option_text*

PollOptionDeleted

Describes a service message about an option deleted from a poll.

Field	Type	Description
poll_message	MaybeInaccessibleMessage	*Optional*. Message containing the poll from which the option was deleted, if known. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
option_persistent_id	String	Unique identifier of the deleted option
option_text	String	Option text
option_text_entities	Array of MessageEntity	*Optional*. Special entities that appear in the *option_text*

ChatBoostAdded

This object represents a service message about a user boosting a chat.

Field	Type	Description
boost_count	Integer	Number of boosts added by the user

BackgroundFill

This object describes the way a background is filled based on the selected colors. Currently, it can be one of

- BackgroundFillSolid
- BackgroundFillGradient
- BackgroundFillFreeformGradient

BackgroundFillSolid

The background is filled using the selected color.

Field	Type	Description
type	String	Type of the background fill, always “solid”
color	Integer	The color of the background fill in the RGB24 format

BackgroundFillGradient

The background is a gradient fill.

Field	Type	Description
type	String	Type of the background fill, always “gradient”
top_color	Integer	Top color of the gradient in the RGB24 format

Field	Type	Description
bottom_color	Integer	Bottom color of the gradient in the RGB24 format
rotation_angle	Integer	Clockwise rotation angle of the background fill in degrees; 0-359

BackgroundFillFreeformGradient

The background is a freeform gradient that rotates after every message in the chat.

Field	Type	Description
type	String	Type of the background fill, always “freeform_gradient”
colors	Array of Integer	A list of the 3 or 4 base colors that are used to generate the freeform gradient in the RGB24 format

BackgroundType

This object describes the type of a background. Currently, it can be one of

- BackgroundTypeFill
- BackgroundTypeWallpaper
- BackgroundTypePattern
- BackgroundTypeChatTheme

BackgroundTypeFill

The background is automatically filled based on the selected colors.

Field	Type	Description
type	String	Type of the background, always “fill”
fill	BackgroundFill	The background fill
dark_theme_dimming	Integer	Dimming of the background in dark themes, as a percentage; 0-100

BackgroundTypeWallpaper

The background is a wallpaper in the JPEG format.

Field	Type	Description
type	String	Type of the background, always “wallpaper”
document	Document	Document with the wallpaper
dark_theme_dimming	Integer	Dimming of the background in dark themes, as a percentage; 0-100
is_blurred	True	*Optional*. *True*, if the wallpaper is downscaled to fit in a 450x450 square and then box-blurred with radius 12
is_moving	True	*Optional*. *True*, if the background moves slightly when the device is tilted

BackgroundTypePattern

The background is a .PNG or .TGV (gzipped subset of SVG with MIME type “application/x-tgwallpattern”) pattern to be combined with the background fill chosen by the user.

Field	Type	Description
type	String	Type of the background, always “pattern”
document	Document	Document with the pattern
fill	BackgroundFill	The background fill that is combined with the pattern
intensity	Integer	Intensity of the pattern when it is shown above the filled background; 0-100
is_inverted	True	*Optional*. *True*, if the background fill must be applied only to the pattern itself. All other pixels are black in this case. For dark themes only.
is_moving	True	*Optional*. *True*, if the background moves slightly when the device is tilted

BackgroundTypeChatTheme

The background is taken directly from a built-in chat theme.

Field	Type	Description
type	String	Type of the background, always “chat_theme”
theme_name	String	Name of the chat theme, which is usually an emoji

ChatBackground

This object represents a chat background.

Field	Type	Description
type	BackgroundType	Type of the background

ChecklistTasksDone

Describes a service message about checklist tasks marked as done or not done.

Field	Type	Description
checklist_message	Message	*Optional*. Message containing the checklist whose tasks were marked as done or not done. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
marked_as_done_task_ids	Array of Integer	*Optional*. Identifiers of the tasks that were marked as done
marked_as_not_done_task_ids	Array of Integer	*Optional*. Identifiers of the tasks that were marked as not done

ChecklistTasksAdded

Describes a service message about tasks added to a checklist.

Field	Type	Description
checklist_message	Message	*Optional*. Message containing the checklist to which the tasks were added. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
tasks	Array of ChecklistTask	List of tasks added to the checklist

CommunityChatAdded

Describes a service message about a chat or a bot being added to a community.

Field	Type	Description
community	Community	The new community to which the chat or the bot belongs

CommunityChatJoined

Describes a service message about a chat being joined by a user from a community.

Field	Type	Description
community	Community	The community from which the chat was joined

CommunityChatRemoved

Describes a service message about a chat or a bot being removed from a community. Currently holds no information.

ForumTopicCreated

This object represents a service message about a new forum topic created in the chat.

Field	Type	Description
name	String	Name of the topic
icon_color	Integer	Color of the topic icon in RGB format
icon_custom_emoji_id	String	*Optional*. Unique identifier of the custom emoji shown as the topic icon
is_name_implicit	True	*Optional*. *True*, if the name of the topic wasn't specified explicitly by its creator and likely needs to be changed by the bot

ForumTopicClosed

This object represents a service message about a forum topic closed in the chat. Currently holds no information.

ForumTopicEdited

This object represents a service message about an edited forum topic.

Field	Type	Description
name	String	*Optional*. New name of the topic, if it was edited
icon_custom_emoji_id	String	*Optional*. New identifier of the custom emoji shown as the topic icon, if it was edited; an empty string if the icon was removed

ForumTopicReopened

This object represents a service message about a forum topic reopened in the chat. Currently holds no information.

GeneralForumTopicHidden

This object represents a service message about General forum topic hidden in the chat. Currently holds no information.

GeneralForumTopicUnhidden

This object represents a service message about General forum topic unhidden in the chat. Currently holds no information.

SharedUser

This object contains information about a user that was shared with the bot using a KeyboardButtonRequestUsers button.

Field	Type	Description
user_id	Integer	Identifier of the shared user. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so 64-bit integers or double-precision float types are safe for storing these identifiers. The bot may not have access to the user and could be unable to use this identifier, unless the user is already known to the bot by some other means.
first_name	String	*Optional*. First name of the user, if the name was requested by the bot
last_name	String	*Optional*. Last name of the user, if the name was requested by the bot
username	String	*Optional*. Username of the user, if the username was requested by the bot
photo	Array of PhotoSize	*Optional*. Available sizes of the chat photo, if the photo was requested by the bot

UsersShared

This object contains information about the users whose identifiers were shared with the bot using a KeyboardButtonRequestUsers button.

Field	Type	Description
request_id	Integer	Identifier of the request
users	Array of SharedUser	Information about users shared with the bot

ChatShared

This object contains information about a chat that was shared with the bot using a KeyboardButtonRequestChat button.

Field	Type	Description
request_id	Integer	Identifier of the request
chat_id	Integer	Identifier of the shared chat. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier. The bot may not have access to the chat and could be unable to use this identifier, unless the chat is already known to the bot by some other means.
title	String	*Optional*. Title of the chat, if the title was requested by the bot
username	String	*Optional*. Username of the chat, if the username was requested by the bot and available
photo	Array of PhotoSize	*Optional*. Available sizes of the chat photo, if the photo was requested by the bot

WriteAccessAllowed

This object represents a service message about a user allowing a bot to write messages after adding it to the attachment menu, launching a Web App from a link, or accepting an explicit request from a Web App sent by the method requestWriteAccess.

Field	Type	Description
from_request	Boolean	*Optional*. *True*, if the access was granted after the user accepted an explicit request from a Web App sent by the method requestWriteAccess
web_app_name	String	*Optional*. Name of the Web App, if the access was granted when the Web App was launched from a link
from_attachment_menu	Boolean	*Optional*. *True*, if the access was granted when the bot was added to the attachment or side menu

VideoChatScheduled

This object represents a service message about a video chat scheduled in the chat.

Field	Type	Description
start_date	Integer	Point in time (Unix timestamp) when the video chat is supposed to be started by a chat administrator

VideoChatStarted

This object represents a service message about a video chat started in the chat. Currently holds no information.

VideoChatEnded

This object represents a service message about a video chat ended in the chat.

Field	Type	Description
duration	Integer	Video chat duration in seconds

VideoChatParticipantsInvited

This object represents a service message about new members invited to a video chat.

Field	Type	Description
users	Array of User	New members that were invited to the video chat

PaidMessagePriceChanged

Describes a service message about a change in the price of paid messages within a chat.

Field	Type	Description
paid_message_star_count	Integer	The new number of Telegram Stars that must be paid by non-administrator users of the supergroup chat for each sent message

DirectMessagePriceChanged

Describes a service message about a change in the price of direct messages sent to a channel chat.

Field	Type	Description
are_direct_messages_enabled	Boolean	*True*, if direct messages are enabled for the channel chat; *False* otherwise
direct_message_star_count	Integer	*Optional*. The new number of Telegram Stars that must be paid by users for each direct message sent to the channel. Does not apply to users who have been exempted by administrators. Defaults to 0.

SuggestedPostApproved

Describes a service message about the approval of a suggested post.

Field	Type	Description
suggested_post_message	Message	*Optional*. Message containing the suggested post. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
price	SuggestedPostPrice	*Optional*. Amount paid for the post
send_date	Integer	Date when the post will be published

SuggestedPostApprovalFailed

Describes a service message about the failed approval of a suggested post. Currently, only caused by insufficient user funds at the time of approval.

Field	Type	Description
suggested_post_message	Message	*Optional*. Message containing the suggested post whose approval has failed. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
price	SuggestedPostPrice	Expected price of the post

SuggestedPostDeclined

Describes a service message about the rejection of a suggested post.

Field	Type	Description
suggested_post_message	Message	*Optional*. Message containing the suggested post. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
comment	String	*Optional*. Comment with which the post was declined

SuggestedPostPaid

Describes a service message about a successful payment for a suggested post.

Field	Type	Description
suggested_post_message	Message	*Optional*. Message containing the suggested post. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
currency	String	Currency in which the payment was made. Currently, one of “XTR” for Telegram Stars or “TON” for TON grams.
amount	Integer	*Optional*. The amount of the currency that was received by the channel in nanograms; for payments in TON grams only
star_amount	StarAmount	*Optional*. The amount of Telegram Stars that was received by the channel; for payments in Telegram Stars only

SuggestedPostRefunded

Describes a service message about a payment refund for a suggested post.

Field	Type	Description
suggested_post_message	Message	*Optional*. Message containing the suggested post. Note that the Message object in this field will not contain the *reply_to_message* field even if it itself is a reply.
reason	String	Reason for the refund. Currently, one of “post_deleted” if the post was deleted within 24 hours of being posted or removed from scheduled messages without being posted, or “payment_refunded” if the payer refunded their payment.

PaidMessagesWaived

Describes a service message about the sender of the message using a link of the other user, which allows sending messages to the user without paying Telegram Stars. Currently holds no information.

TonWalletTransfer

Describes a service message about a transfer with a TON wallet.

Field	Type	Description
transaction_id	String	Unique identifier of the transaction; may be empty if the transfer hasn't been sent yet
peer_address	String	Address of the transfer peer in the TON blockchain
amount	String	The amount of transferred cryptocurrency, in the *smallest units* of the cryptocurrency, as a decimal number; negative for outgoing transfers
comment	String	*Optional*. Comment added to the transfer; omitted if the comment is encrypted

GiveawayCreated

This object represents a service message about the creation of a scheduled giveaway.

Field	Type	Description
prize_star_count	Integer	*Optional*. The number of Telegram Stars to be split between giveaway winners; for Telegram Star giveaways only

Giveaway

This object represents a message about a scheduled giveaway.

Field	Type	Description
chats	Array of Chat	The list of chats which the user must join to participate in the giveaway

Field	Type	Description
winners_selection_date	Integer	Point in time (Unix timestamp) when winners of the giveaway will be selected
winner_count	Integer	The number of users which are supposed to be selected as winners of the giveaway
only_new_members	True	*Optional*. *True*, if only users who join the chats after the giveaway started should be eligible to win
has_public_winners	True	*Optional*. *True*, if the list of giveaway winners will be visible to everyone
prize_description	String	*Optional*. Description of additional giveaway prize
country_codes	Array of String	*Optional*. A list of two-letter ISO 3166-1 alpha-2 country codes indicating the countries from which eligible users for the giveaway must come. If empty, then all users can participate in the giveaway. Users with a phone number that was bought on Fragment can always participate in giveaways.
prize_star_count	Integer	*Optional*. The number of Telegram Stars to be split between giveaway winners; for Telegram Star giveaways only
premium_subscription_month_count	Integer	*Optional*. The number of months the Telegram Premium subscription won from the giveaway will be active for; for Telegram Premium giveaways only

GiveawayWinners

This object represents a message about the completion of a giveaway with public winners.

Field	Type	Description
chat	Chat	The chat that created the giveaway
giveaway_message_id	Integer	Identifier of the message with the giveaway in the chat
winners_selection_date	Integer	Point in time (Unix timestamp) when winners of the giveaway were selected
winner_count	Integer	Total number of winners in the giveaway
winners	Array of User	List of up to 100 winners of the giveaway
additional_chat_count	Integer	*Optional*. The number of other chats the user had to join in order to be eligible for the giveaway
prize_star_count	Integer	*Optional*. The number of Telegram Stars that were split between giveaway winners; for Telegram Star giveaways only
premium_subscription_month_count	Integer	*Optional*. The number of months the Telegram Premium subscription won from the giveaway will be active for; for Telegram Premium giveaways only

Field	Type	Description
unclaimed_prize_count	Integer	*Optional*. Number of undistributed prizes
only_new_members	True	*Optional*. *True*, if only users who had joined the chats after the giveaway started were eligible to win
was_refunded	True	*Optional*. *True*, if the giveaway was canceled because the payment for it was refunded
prize_description	String	*Optional*. Description of additional giveaway prize

GiveawayCompleted

This object represents a service message about the completion of a giveaway without public winners.

Field	Type	Description
winner_count	Integer	Number of winners in the giveaway
unclaimed_prize_count	Integer	*Optional*. Number of undistributed prizes
giveaway_message	Message	*Optional*. Message with the giveaway that was completed, if it wasn't deleted
is_star_giveaway	True	*Optional*. *True*, if the giveaway is a Telegram Star giveaway. Otherwise, currently, the giveaway is a Telegram Premium giveaway.

LinkPreviewOptions

Describes the options used for link preview generation.

Field	Type	Description
is_disabled	Boolean	*Optional*. *True*, if the link preview is disabled
url	String	*Optional*. URL to use for the link preview. If empty, then the first URL found in the message text will be used.
prefer_small_media	Boolean	*Optional*. *True*, if the media in the link preview is supposed to be shrunk; ignored if the URL isn't explicitly specified or media size change isn't supported for the preview
prefer_large_media	Boolean	*Optional*. *True*, if the media in the link preview is supposed to be enlarged; ignored if the URL isn't explicitly specified or media size change isn't supported for the preview
show_above_text	Boolean	*Optional*. *True*, if the link preview must be shown above the message text; otherwise, the link preview will be shown below the message text

SuggestedPostPrice

Describes the price of a suggested post.

Field	Type	Description
currency	String	Currency in which the post will be paid. Currently, must be one of “XTR” for Telegram Stars or “TON” for TON grams.
amount	Integer	The amount of the currency that will be paid for the post in the *smallest units* of the currency, i.e. Telegram Stars or nanograms. Currently, price in Telegram Stars must be between 5 and 100000, and price in nanograms must be between 10000000 and 10000000000000.

SuggestedPostInfo

Contains information about a suggested post.

Field	Type	Description
state	String	State of the suggested post. Currently, it can be one of “pending”, “approved”, “declined”.
price	SuggestedPostPrice	*Optional*. Proposed price of the post. If the field is omitted, then the post is unpaid.
send_date	Integer	*Optional*. Proposed send date of the post. If the field is omitted, then the post can be published at any time within 30 days at the sole discretion of the user or administrator who approves it.

SuggestedPostParameters

Contains parameters of a post that is being suggested by the bot.

Field	Type	Description
price	SuggestedPostPrice	*Optional*. Proposed price for the post. If the field is omitted, then the post is unpaid.
send_date	Integer	*Optional*. Proposed send date of the post. If specified, then the date must be between 300 second and 2678400 seconds (30 days) in the future. If the field is omitted, then the post can be published at any time within 30 days at the sole discretion of the user who approves it.

DirectMessagesTopic

Describes a topic of a direct messages chat.

Field	Type	Description
topic_id	Integer	Unique identifier of the topic. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier.
user	User	*Optional*. Information about the user that created the topic. Currently, it is always present.

UserProfilePhotos

This object represent a user's profile pictures.

Field	Type	Description
total_count	Integer	Total number of profile pictures the target user has
photos	Array of Array of PhotoSize	Requested profile pictures (in up to 4 sizes each)

UserProfileAudios

This object represents the audios displayed on a user's profile.

Field	Type	Description
total_count	Integer	Total number of profile audios for the target user
audios	Array of Audio	Requested profile audios

File

This object represents a file ready to be downloaded. The file can be downloaded via the link `https://api.telegram.org/file/bot<token>/<file_path>`. It is guaranteed that the link will be valid for at least 1 hour. When the link expires, a new one can be requested by calling getFile.

> The maximum file size to download is 20 MB

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
file_size	Integer	*Optional*. File size in bytes. It can be bigger than 2^31 and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this value.
file_path	String	*Optional*. File path. Use `https://api.telegram.org/file/bot<token>/<file_path>` to get the file.

WebAppInfo

Describes a Web App.

Field	Type	Description
url	String	An HTTPS URL of a Web App to be opened with additional data as specified in Initializing Web Apps

ReplyKeyboardMarkup

This object represents a custom keyboard with reply options (see Introduction to bots for details and examples). Not supported in channels and for messages sent on behalf of a business account.

Field	Type	Description
keyboard	Array of Array of KeyboardButton	Array of button rows, each represented by an Array of KeyboardButton objects
is_persistent	Boolean	*Optional*. Requests clients to always show the keyboard when the regular keyboard is hidden. Defaults to *False*, in which case the custom keyboard can be hidden and opened with a keyboard icon.
resize_keyboard	Boolean	*Optional*. Requests clients to resize the keyboard vertically for optimal fit (e.g., make the keyboard smaller if there are just two rows of buttons). Defaults to *False*, in which case the custom keyboard is always of the same height as the app's standard keyboard.
one_time_keyboard	Boolean	*Optional*. Requests clients to hide the keyboard as soon as it's been used. The keyboard will still be available, but clients will automatically display the usual letter-keyboard in the chat - the user can press a special button in the input field to see the custom keyboard again. Defaults to *False*.
input_field_placeholder	String	*Optional*. The placeholder to be shown in the input field when the keyboard is active; 1-64 characters
selective	Boolean	*Optional*. Use this parameter if you want to show the keyboard to specific users only. Targets: 1) users that are @mentioned in the *text* of the Message object; 2) if the bot's message is a reply to a message in the same chat and forum topic, sender of the original message. *Example:* A user requests to change the bot's language, bot replies to the request with a keyboard to select the new language. Other users in the group don't see the keyboard.
force_reply	Boolean	*Optional*. Pass *True* if the reply interface must be shown to the user, as if they had manually selected the bot's message and tapped 'Reply'

KeyboardButton

This object represents one button of the reply keyboard. At most one of the fields other than *text*, *icon_custom_emoji_id*, and *style* must be used to specify the type of the button. For simple text buttons, *String* can be used instead of this object to specify the button text.

Field	Type	Description
text	String	Text of the button. If none of the fields other than *text*, *icon_custom_emoji_id*, and *style* are used, it will be sent as a message when the button is pressed.

Field	Type	Description
icon_custom_emoji_id	String	*Optional*. Unique identifier of the custom emoji shown before the text of the button. Can only be used by bots that purchased additional usernames on Fragment or in the messages directly sent by the bot to private, group and supergroup chats if the owner of the bot has a Telegram Premium subscription.
style	String	*Optional*. Style of the button. Must be one of “danger” (red), “success” (green) or “primary” (blue). If omitted, then an app-specific style is used.
request_users	KeyboardButtonRequestUsers	*Optional*. If specified, pressing the button will open a list of suitable users. Identifiers of selected users will be sent to the bot in a “users_shared” service message. Available in private chats only.
request_chat	KeyboardButtonRequestChat	*Optional*. If specified, pressing the button will open a list of suitable chats. Tapping on a chat will send its identifier to the bot in a “chat_shared” service message. Available in private chats only.
request_managed_bot	KeyboardButtonRequestManagedBot	*Optional*. If specified, pressing the button will ask the user to create and share a bot that will be managed by the current bot. Available for bots that enabled management of other bots in the @BotFather Mini App. Available in private chats only.
request_contact	Boolean	*Optional*. If *True*, the user's phone number will be sent as a contact when the button is pressed. Available in private chats only.
request_location	Boolean	*Optional*. If *True*, the user's current location will be sent when the button is pressed. Available in private chats only.
request_poll	KeyboardButtonPollType	*Optional*. If specified, the user will be asked to create a poll and send it to the bot when the button is pressed. Available in private chats only.
web_app	WebAppInfo	*Optional*. If specified, the described Web App will be launched when the button is pressed. The Web App will be able to send a “web_app_data” service message. Available in private chats only.

KeyboardButtonRequestUsers

This object defines the criteria used to request suitable users. Information about the selected users will be shared with the bot when the corresponding button is pressed. More about requesting users »

Field	Type	Description
request_id	Integer	Signed 32-bit identifier of the request that will be received back in the UsersShared object. Must be unique within the message.
user_is_bot	Boolean	*Optional*. Pass *True* to request bots, pass *False* to request regular users. If not specified, no additional restrictions are applied.
user_is_premium	Boolean	*Optional*. Pass *True* to request premium users, pass *False* to request non-premium users. If not specified, no additional restrictions are applied.
max_quantity	Integer	*Optional*. The maximum number of users to be selected; 1-10. Defaults to 1.
request_name	Boolean	*Optional*. Pass *True* to request the users' first and last names
request_username	Boolean	*Optional*. Pass *True* to request the users' usernames
request_photo	Boolean	*Optional*. Pass *True* to request the users' photos

KeyboardButtonRequestChat

This object defines the criteria used to request a suitable chat. Information about the selected chat will be shared with the bot when the corresponding button is pressed. The bot will be granted requested rights in the chat if appropriate. More about requesting chats ».

Field	Type	Description
request_id	Integer	Signed 32-bit identifier of the request, which will be received back in the ChatShared object. Must be unique within the message.
chat_is_channel	Boolean	Pass *True* to request a channel chat, pass *False* to request a group or a supergroup chat
chat_is_forum	Boolean	*Optional*. Pass *True* to request a forum supergroup, pass *False* to request a non-forum chat. If not specified, no additional restrictions are applied.
chat_has_username	Boolean	*Optional*. Pass *True* to request a supergroup or a channel with a username, pass *False* to request a chat without a username. If not specified, no additional restrictions are applied.
chat_is_created	Boolean	*Optional*. Pass *True* to request a chat owned by the user. Otherwise, no additional restrictions are applied.
user_administrator_rights	ChatAdministratorRights	*Optional*. A JSON-serialized object listing the required administrator rights of the user in the chat. The rights must be a superset of *bot_administrator_rights*. If not specified, no additional restrictions are applied.
bot_administrator_rights	ChatAdministratorRights	*Optional*. A JSON-serialized object listing the required administrator rights of the bot in the chat. The rights must be a subset of *user_administrator_rights*. If not specified, no additional restrictions are applied.

Field	Type	Description
bot_is_member	Boolean	*Optional*. Pass *True* to request a chat with the bot as a member. Otherwise, no additional restrictions are applied.
request_title	Boolean	*Optional*. Pass *True* to request the chat's title
request_username	Boolean	*Optional*. Pass *True* to request the chat's username
request_photo	Boolean	*Optional*. Pass *True* to request the chat's photo

KeyboardButtonRequestManagedBot

This object defines the parameters for the creation of a managed bot. Information about the created bot will be shared with the bot using the update *managed_bot* and a Message with the field *managed_bot_created*.

Field	Type	Description
request_id	Integer	Signed 32-bit identifier of the request. Must be unique within the message.
suggested_name	String	*Optional*. Suggested name for the bot
suggested_username	String	*Optional*. Suggested username for the bot. The username must end with “bot”. If the user creating the bot has a Telegram Premium subscription, then the username can also end with “agent”, “ai”, “app”, “game”, or “gay”; in this case, the username will be assigned to the bot as a secondary username, and a regular username ending with “bot” will be assigned to the bot automatically.

KeyboardButtonPollType

This object represents type of a poll, which is allowed to be created and sent when the corresponding button is pressed.

Field	Type	Description
type	String	*Optional*. If *quiz* is passed, the user will be allowed to create only polls in the quiz mode. If *regular* is passed, only regular polls will be allowed. Otherwise, the user will be allowed to create a poll of any type.

ReplyKeyboardRemove

Upon receiving a message with this object, Telegram clients will remove the current custom keyboard and display the default letter-keyboard. By default, custom keyboards are displayed until a new keyboard is sent by a bot. An exception is made for one-time keyboards that are hidden immediately after the user presses a button (see ReplyKeyboardMarkup). Not supported in channels and for messages sent on behalf of a business account.

Field	Type	Description
remove_keyboard	True	Requests clients to remove the custom keyboard (user will not be able to summon this keyboard; if you want to hide the keyboard from sight but keep it accessible, use *one_time_keyboard* in ReplyKeyboardMarkup)
selective	Boolean	*Optional*. Use this parameter if you want to remove the keyboard for specific users only. Targets: 1) users that are @mentioned in the *text* of the Message object; 2) if the bot's message is a reply to a message in the same chat and forum topic, sender of the original message. *Example:* A user votes in a poll, bot returns confirmation message in reply to the vote and removes the keyboard for that user, while still showing the keyboard with poll options to users who haven't voted yet.

InlineKeyboardMarkup

This object represents an inline keyboard that appears right next to the message it belongs to.

Field	Type	Description
inline_keyboard	Array of Array of InlineKeyboardButton	Array of button rows, each represented by an Array of InlineKeyboardButton objects
force_reply	Boolean	*Optional*. Pass *True* if the reply interface must be shown to the user, as if they had manually selected the bot's message and tapped 'Reply'. The value of the field can't be changed when the inline keyboard is edited.

InlineKeyboardButton

This object represents one button of an inline keyboard. Exactly one of the fields other than *text*, *icon_custom_emoji_id*, and *style* must be used to specify the type of the button.

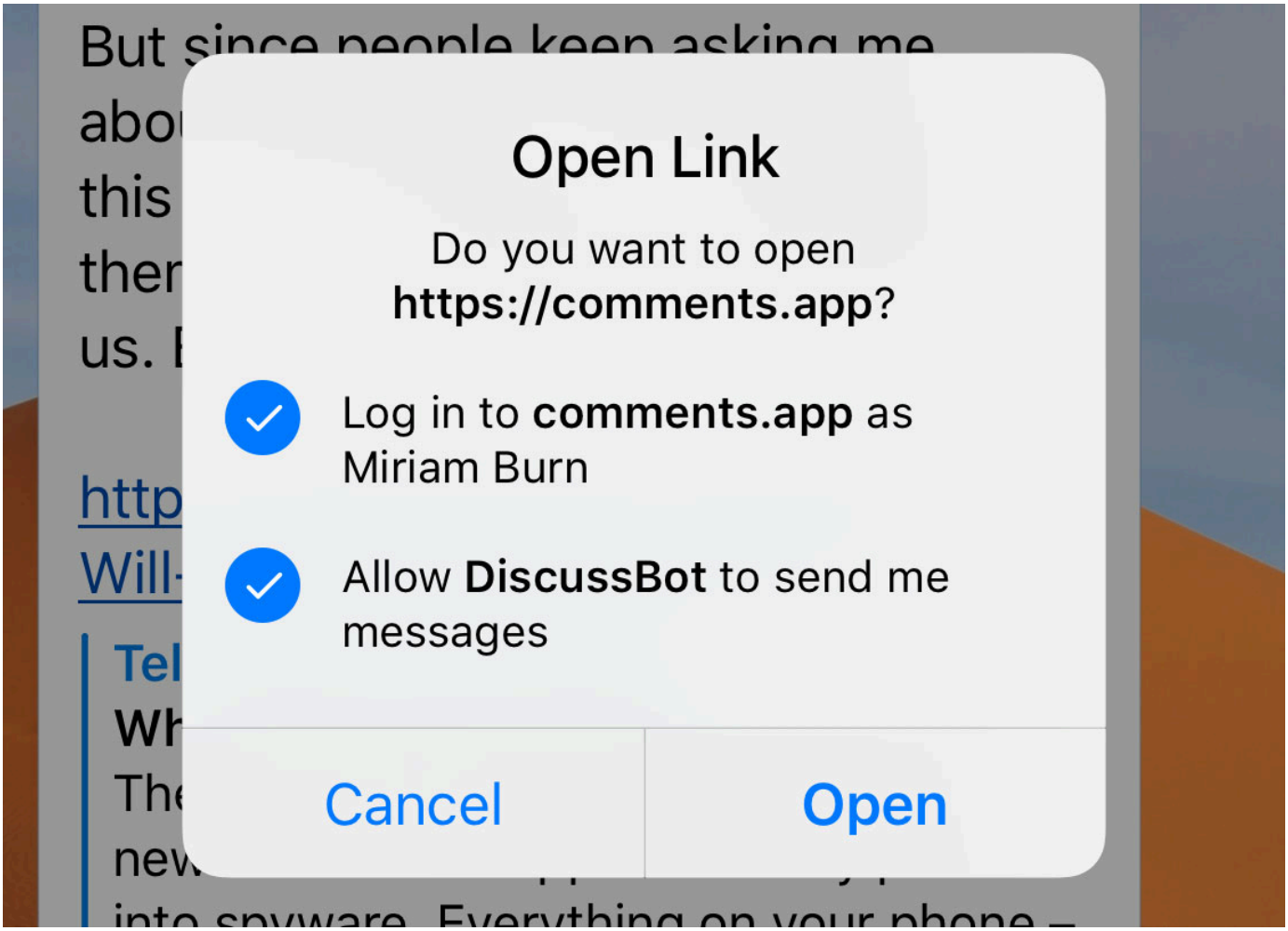
Field	Type	Description
text	String	Label text on the button
icon_custom_emoji_id	String	*Optional*. Unique identifier of the custom emoji shown before the text of the button. Can only be used by bots that purchased additional usernames on Fragment or in the messages directly sent by the bot to private, group and supergroup chats if the owner of the bot has a Telegram Premium subscription.
style	String	*Optional*. Style of the button. Must be one of “danger” (red), “success” (green) or “primary” (blue). If omitted, then an app-specific style is used.

Field	Type	Description
url	String	*Optional*. HTTP or tg:// URL to be opened when the button is pressed. Links `tg://user?id=<user_id>` can be used to mention a user by their identifier without using a username, if this is allowed by their privacy settings.
callback_data	String	*Optional*. Data to be sent in a callback query to the bot when the button is pressed, 1-64 bytes
web_app	WebAppInfo	*Optional*. Description of the Web App that will be launched when the user presses the button. The Web App will be able to send an arbitrary message on behalf of the user using the method answerWebAppQuery. Available only in private chats between a user and the bot. Not supported for messages sent on behalf of a business account and ephemeral messages.
login_url	LoginUrl	*Optional*. An HTTPS URL used to automatically authorize the user. Can be used as a replacement for the Telegram Login Widget. Not supported for ephemeral messages.
switch_inline_query	String	*Optional*. If set, pressing the button will prompt the user to select one of their chats, open that chat and insert the bot's username and the specified inline query in the input field. May be empty, in which case just the bot's username will be inserted. Not supported for messages sent in channel direct messages chats and on behalf of a business account.
switch_inline_query_current_chat	String	*Optional*. If set, pressing the button will insert the bot's username and the specified inline query in the current chat's input field. May be empty, in which case only the bot's username will be inserted. This offers a quick way for the user to open your bot in inline mode in the same chat - good for selecting something from multiple options. Not supported in channels and for messages sent in channel direct messages chats and on behalf of a business account.

Field	Type	Description
switch_inline_query_chosen_chat	SwitchInlineQueryChosenChat	*Optional*. If set, pressing the button will prompt the user to select one of their chats of the specified type, open that chat and insert the bot's username and the specified inline query in the input field. Not supported for messages sent in channel direct messages chats and on behalf of a business account.
copy_text	CopyTextButton	*Optional*. Description of the button that copies the specified text to the clipboard
callback_game	CallbackGame	*Optional*. Description of the game that will be launched when the user presses the button. **NOTE:** This type of button **must** always be the first button in the first row.
pay	Boolean	*Optional*. Specify *True*, to send a Pay button. Substrings "⭐" and "XTR" in the buttons's text will be replaced with a Telegram Star icon. **NOTE:** This type of button **must** always be the first button in the first row and can only be used in invoice messages.
disabled	DisabledButton	*Optional*. If set, then the button is disabled and does nothing

LoginUrl

This object represents a parameter of the inline keyboard button used to automatically authorize a user. It serves as a great replacement for the Telegram Login Widget when the user is coming from Telegram. All the user needs to do is tap/click a button and confirm that they want to log in:



> Sample bot: @DiscussBot

Field	Type	Description
url	String	An HTTPS URL to be opened with user authorization data added to the query string when the button is pressed. If the user refuses to provide authorization data, the original URL without information about the user will be opened. The data added is the same as described in Receiving authorization data. **NOTE:** You **must** always check the hash of the received data to verify the authentication and the integrity of the data as described in Checking authorization.
forward_text	String	*Optional*. New text of the button in forwarded messages
bot_username	String	*Optional*. Username of a bot, which will be used for user authorization; not supported in RichMessageButton. See Setting up a bot for more details. If not specified, the current bot's username will be assumed. The *url*'s domain must be the same as the domain linked with the bot. See Linking your domain to the bot for more details.
request_write_access	Boolean	*Optional*. Pass *True* to request the permission for your bot to send messages to the user

SwitchInlineQueryChosenChat

This object represents an inline button that switches the current user to inline mode in a chosen chat, with an optional default inline query.

Field	Type	Description
query	String	*Optional*. The default inline query to be inserted in the input field. If left empty, only the bot's username will be inserted.
allow_user_chats	Boolean	*Optional*. *True*, if private chats with users can be chosen
allow_bot_chats	Boolean	*Optional*. *True*, if private chats with bots can be chosen
allow_group_chats	Boolean	*Optional*. *True*, if group and supergroup chats can be chosen
allow_channel_chats	Boolean	*Optional*. *True*, if channel chats can be chosen

CopyTextButton

This object represents an inline keyboard button that copies specified text to the clipboard.

Field	Type	Description
text	String	The text to be copied to the clipboard; 1-256 characters

DisabledButton

This object represents a disabled button which does nothing. Currently holds no information.

CallbackQuery

This object represents an incoming callback query from a callback button in an inline keyboard. If the button that originated the query was attached to a message sent by the bot, the field *message* will be present. If the button was attached to a message sent via the bot (in inline mode), the field *inline_message_id* will be present. Exactly one of the fields *data* or *game_short_name* will be present.

Field	Type	Description
id	String	Unique identifier for this query
from	User	Sender
message	MaybeInaccessibleMessage	*Optional*. Message sent by the bot with the callback button that originated the query
inline_message_id	String	*Optional*. Identifier of the message sent via the bot in inline mode, that originated the query
chat_instance	String	Global identifier, uniquely corresponding to the chat to which the message with the callback button was sent. Useful for high scores in games.

Field	Type	Description
data	String	*Optional*. Data associated with the callback button. Be aware that the message originated the query can contain no callback buttons with this data.
game_short_name	String	*Optional*. Short name of a Game to be returned, serves as the unique identifier for the game

> **NOTE:** After the user presses a callback button, Telegram clients will display a progress bar until you call answerCallbackQuery. It is, therefore, necessary to react by calling answerCallbackQuery even if no notification to the user is needed (e.g., without specifying any of the optional parameters).

WidgetQueryType

This object describes the type of a query from a widget. Currently, it can be one of

- WidgetQueryTypeData
- WidgetQueryTypeSignData
- WidgetQueryTypeSentTransaction

WidgetQueryTypeData

The query contains data for the bot.

Field	Type	Description
type	String	Type of the query, always “data”
wallet_card	TonWalletCard	*Optional*. TON wallet card of the user who sent the query, if it was sent to the bot along with the query

WidgetQueryTypeSignData

The query contains data signed by a TON wallet of the user who sent the query. The data is signed for the domain `bot<bot_id>`, where `<bot_id>` is the identifier of the bot.

Field	Type	Description
type	String	Type of the query, always “sign_data”
wallet	WidgetQueryWallet	The wallet that signed the data
sign_data	String	JSON-serialized signed data

WidgetQueryTypeSentTransaction

The query notifies the bot about a transaction sent from a TON wallet of the user who sent the query.

Field	Type	Description
type	String	Type of the query, always “sent_transaction”
wallet	WidgetQueryWallet	The wallet from which the transaction was sent
msg_hash	String	Value of *msg_hash* of the transaction

WidgetQueryWallet

This object describes a TON wallet of the user, which is used in a query from a widget.

Field	Type	Description
address	String	Address of the wallet
amount	String	Balance of the wallet, in the *smallest units* of the cryptocurrency, as a decimal number
public_key	String	*Optional*. Hex-encoded public key of the wallet; omitted if unknown
state_init	String	*Optional*. Base64-encoded serialized StateInit of the wallet contract; omitted if and only if *public_key* is omitted

WidgetQuery

This object represents an incoming query from a widget in a message sent by the bot or via the bot. Use answerWidgetQuery to answer the query. Each user can send at most 5 queries per 5 seconds to the bot from all its widgets.

Field	Type	Description
id	String	Unique identifier for this query
from	User	Sender
chat	Chat	*Optional*. Chat with the message containing the widget; omitted if the widget is in an inline message
message_id	Integer	*Optional*. Identifier of the message containing the widget; omitted if the widget is in an inline message
inline_message_id	String	*Optional*. Identifier of the inline message containing the widget
chat_instance	String	Global identifier, uniquely corresponding to the chat to which the message with the widget was sent
chat_type	String	*Optional*. For widgets in inline messages, type of the chat to which the message was sent. Can be either “sender” for a private chat with the inline query sender, “private”, “group”, “supergroup”, or “channel”.
type	WidgetQueryType	Type of the query
data	String	Data of the query, 1-64 bytes
widget_id	String	*Optional*. Unique identifier of the shared widget, which sent the query; omitted if the widget isn't shared

SharedWidgetQuery

This object represents an incoming query from a forwarded message with a shared widget created by the bot. Use answerWidgetQuery to answer the query. Each user can send at most 5 queries per 5 seconds to the bot from all its widgets.

Field	Type	Description
id	String	Unique identifier for this query
from	User	Sender
type	WidgetQueryType	Type of the query
data	String	Data of the query, 1-64 bytes
widget_id	String	Unique identifier of the shared widget, which sent the query

ForceReply

Upon receiving a message with this object, Telegram clients will display a reply interface to the user (act as if the user has selected the bot's message and tapped 'Reply'). This can be extremely useful if you want to create user-friendly step-by-step interfaces without having to sacrifice privacy mode. Not supported in channels and for messages sent on behalf of a user account.

Field	Type	Description
force_reply	True	Shows reply interface to the user, as if they had manually selected the bot's message and tapped 'Reply'
input_field_placeholder	String	*Optional*. The placeholder to be shown in the input field when the reply is active; 1-64 characters
selective	Boolean	*Optional*. Use this parameter if you want to force reply from specific users only. Targets: 1) users that are @mentioned in the *text* of the Message object; 2) if the bot's message is a reply to a message in the same chat and forum topic, sender of the original message.

> **Example:** A poll bot for groups runs in privacy mode (only receives commands, replies to its messages and mentions). There could be two ways to create a new poll:
>
> - Explain the user how to send a command with parameters (e.g. /newpoll question answer1 answer2). May be appealing for hardcore users but lacks modern day polish.
> - Guide the user through a step-by-step process. 'Please send me your question', 'Cool, now let's add the first answer option', 'Great. Keep adding answer options, then send /done when you're ready'.
>
> The last option is definitely more attractive. And if you use ForceReply in your bot's questions, it will receive the user's answers even if it only receives replies, commands and mentions - without any extra work for the user.

Community

Represents a community (a group of chats).

Field	Type	Description
id	Integer	Unique identifier for this community. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
name	String	Name of the community

ChatPhoto

This object represents a chat photo.

Field	Type	Description
small_file_id	String	File identifier of small (160x160) chat photo. This file_id can be used only for photo download and only for as long as the photo is not changed.
small_file_unique_id	String	Unique file identifier of small (160x160) chat photo, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
big_file_id	String	File identifier of big (640x640) chat photo. This file_id can be used only for photo download and only for as long as the photo is not changed.
big_file_unique_id	String	Unique file identifier of big (640x640) chat photo, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.

ChatInviteLink

Represents an invite link for a chat.

Field	Type	Description
invite_link	String	The invite link. If the link was created by another chat administrator, then the second part of the link will be replaced with “…”.
creator	User	Creator of the link
creates_join_request	Boolean	*True*, if users joining the chat via the link need to be approved by chat administrators
is_primary	Boolean	*True*, if the link is primary
is_revoked	Boolean	*True*, if the link is revoked
name	String	*Optional*. Invite link name
expire_date	Integer	*Optional*. Point in time (Unix timestamp) when the link will expire or has been expired
member_limit	Integer	*Optional*. The maximum number of users that can be members of the chat simultaneously after joining the chat via this invite link; 1-99999

Field	Type	Description
pending_join_request_count	Integer	*Optional*. Number of pending join requests created using this link
subscription_period	Integer	*Optional*. The number of seconds the subscription will be active for before the next payment
subscription_price	Integer	*Optional*. The amount of Telegram Stars a user must pay initially and after each subsequent subscription period to be a member of the chat using the link

ChatAdministratorRights

Represents the rights of an administrator in a chat.

Field	Type	Description
is_anonymous	Boolean	*True*, if the user's presence in the chat is hidden
can_manage_chat	Boolean	*True*, if the administrator can access the chat event log, get boost list, see hidden supergroup and channel members, report spam messages, ignore slow mode, and send messages to the chat without paying Telegram Stars. Implied by any other administrator privilege.
can_delete_messages	Boolean	*True*, if the administrator can delete messages of other users
can_manage_video_chats	Boolean	*True*, if the administrator can manage video chats
can_restrict_members	Boolean	*True*, if the administrator can restrict, ban or unban chat members, or access supergroup statistics
can_promote_members	Boolean	*True*, if the administrator can add new administrators with a subset of their own privileges or demote administrators that they have promoted, directly or indirectly (promoted by administrators that were appointed by the user)
can_change_info	Boolean	*True*, if the user is allowed to change the chat title, photo and other settings
can_invite_users	Boolean	*True*, if the user is allowed to invite new users to the chat
can_post_stories	Boolean	*True*, if the administrator can post stories to the chat
can_edit_stories	Boolean	*True*, if the administrator can edit stories posted by other users, post stories to the chat page, pin chat stories, and access the chat's story archive
can_delete_stories	Boolean	*True*, if the administrator can delete stories posted by other users
can_post_messages	Boolean	*Optional*. *True*, if the administrator can post messages in the channel, approve suggested posts, or access channel statistics; for channels only
can_edit_messages	Boolean	*Optional*. *True*, if the administrator can edit messages of other users and can pin messages; for channels only

Field	Type	Description
can_pin_messages	Boolean	*Optional*. *True*, if the user is allowed to pin messages; for groups and supergroups only
can_manage_topics	Boolean	*Optional*. *True*, if the user is allowed to create, rename, close, and reopen forum topics; for supergroups only
can_manage_direct_messages	Boolean	*Optional*. *True*, if the administrator can manage direct messages of the channel and decline suggested posts; for channels only
can_manage_tags	Boolean	*Optional*. *True*, if the administrator can edit the tags of regular members; for groups and supergroups only
can_send_welcome_messages	Boolean	*True*, if the administrator can manage chat welcome messages or directly send them in the case of bots

ChatMemberUpdated

This object represents changes in the status of a chat member.

Field	Type	Description
chat	Chat	Chat the user belongs to
from	User	Performer of the action, which resulted in the change
date	Integer	Date the change was done in Unix time
old_chat_member	ChatMember	Previous information about the chat member
new_chat_member	ChatMember	New information about the chat member
invite_link	ChatInviteLink	*Optional*. Chat invite link, which was used by the user to join the chat; for joining by invite link events only
via_join_request	Boolean	*Optional*. *True*, if the user joined the chat after sending a direct join request without using an invite link and being approved by an administrator
via_chat_folder_invite_link	Boolean	*Optional*. *True*, if the user joined the chat via a chat folder invite link

ChatMember

This object contains information about one member of a chat. Currently, the following 6 types of chat members are supported:

- ChatMemberOwner
- ChatMemberAdministrator
- ChatMemberMember
- ChatMemberRestricted
- ChatMemberLeft
- ChatMemberBanned

ChatMemberOwner

Represents a chat member that owns the chat and has all administrator privileges.

Field	Type	Description
status	String	The member's status in the chat, always “creator”
user	User	Information about the user
is_anonymous	Boolean	*True*, if the user's presence in the chat is hidden
custom_title	String	*Optional*. Custom title for this user

ChatMemberAdministrator

Represents a chat member that has some additional privileges.

Field	Type	Description
status	String	The member's status in the chat, always “administrator”
user	User	Information about the user
can_be_edited	Boolean	*True*, if the bot is allowed to edit administrator privileges of that user
is_anonymous	Boolean	*True*, if the user's presence in the chat is hidden
can_manage_chat	Boolean	*True*, if the administrator can access the chat event log, get boost list, see hidden supergroup and channel members, report spam messages, ignore slow mode, and send messages to the chat without paying Telegram Stars. Implied by any other administrator privilege.
can_delete_messages	Boolean	*True*, if the administrator can delete messages of other users
can_manage_video_chats	Boolean	*True*, if the administrator can manage video chats
can_restrict_members	Boolean	*True*, if the administrator can restrict, ban or unban chat members, or access supergroup statistics
can_promote_members	Boolean	*True*, if the administrator can add new administrators with a subset of their own privileges or demote administrators that they have promoted, directly or indirectly (promoted by administrators that were appointed by the user)
can_change_info	Boolean	*True*, if the user is allowed to change the chat title, photo and other settings
can_invite_users	Boolean	*True*, if the user is allowed to invite new users to the chat
can_post_stories	Boolean	*True*, if the administrator can post stories to the chat
can_edit_stories	Boolean	*True*, if the administrator can edit stories posted by other users, post stories to the chat page, pin chat stories, and access the chat's story archive
can_delete_stories	Boolean	*True*, if the administrator can delete stories posted by other users

Field	Type	Description
can_post_messages	Boolean	*Optional*. *True*, if the administrator can post messages in the channel, approve suggested posts, or access channel statistics; for channels only
can_edit_messages	Boolean	*Optional*. *True*, if the administrator can edit messages of other users and can pin messages; for channels only
can_pin_messages	Boolean	*Optional*. *True*, if the user is allowed to pin messages; for groups and supergroups only
can_manage_topics	Boolean	*Optional*. *True*, if the user is allowed to create, rename, close, and reopen forum topics; for supergroups only
can_manage_direct_messages	Boolean	*Optional*. *True*, if the administrator can manage direct messages of the channel and decline suggested posts; for channels only
can_manage_tags	Boolean	*Optional*. *True*, if the administrator can edit the tags of regular members; for groups and supergroups only
can_send_welcome_messages	Boolean	*True*, if the administrator can manage chat welcome messages or directly send them in the case of bots
custom_title	String	*Optional*. Custom title for this user

ChatMemberMember

Represents a chat member that has no additional privileges or restrictions.

Field	Type	Description
status	String	The member's status in the chat, always “member”
tag	String	*Optional*. Tag of the member
user	User	Information about the user
until_date	Integer	*Optional*. Date when the user's subscription will expire; Unix time

ChatMemberRestricted

Represents a chat member that is under certain restrictions in the chat. Supergroups only.

Field	Type	Description
status	String	The member's status in the chat, always “restricted”
tag	String	*Optional*. Tag of the member
user	User	Information about the user
is_member	Boolean	*True*, if the user is a member of the chat at the moment of the request
can_send_messages	Boolean	*True*, if the user is allowed to send text messages, rich messages, contacts, giveaways, giveaway winners, invoices, locations and venues

Field	Type	Description
can_send_audios	Boolean	*True*, if the user is allowed to send audios
can_send_documents	Boolean	*True*, if the user is allowed to send documents
can_send_photos	Boolean	*True*, if the user is allowed to send photos
can_send_videos	Boolean	*True*, if the user is allowed to send videos
can_send_video_notes	Boolean	*True*, if the user is allowed to send video notes
can_send_voice_notes	Boolean	*True*, if the user is allowed to send voice notes
can_send_polls	Boolean	*True*, if the user is allowed to send polls and checklists
can_send_other_messages	Boolean	*True*, if the user is allowed to send animations, games, stickers, widgets and use inline bots
can_add_web_page_previews	Boolean	*True*, if the user is allowed to add web page previews to their messages
can_react_to_messages	Boolean	*True*, if the user is allowed to react to messages
can_edit_tag	Boolean	*True*, if the user is allowed to edit their own tag
can_change_info	Boolean	*True*, if the user is allowed to change the chat title, photo and other settings
can_invite_users	Boolean	*True*, if the user is allowed to invite new users to the chat
can_pin_messages	Boolean	*True*, if the user is allowed to pin messages
can_manage_topics	Boolean	*True*, if the user is allowed to create forum topics
until_date	Integer	Date when restrictions will be lifted for this user; Unix time. If 0, then the user is restricted forever.

ChatMemberLeft

Represents a chat member that isn't currently a member of the chat, but may join it themselves.

Field	Type	Description
status	String	The member's status in the chat, always “left”
user	User	Information about the user

ChatMemberBanned

Represents a chat member that was banned in the chat and can't return to the chat or view chat messages.

Field	Type	Description
status	String	The member's status in the chat, always “kicked”

Field	Type	Description
user	User	Information about the user
until_date	Integer	Date when restrictions will be lifted for this user; Unix time. If 0, then the user is banned forever.

ChatJoinRequest

Represents a join request sent to a chat.

Field	Type	Description
chat	Chat	Chat to which the request was sent
from	User	User that sent the join request
user_chat_id	Integer	Identifier of a private chat with the user who sent the join request. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier. The bot can use this identifier for 5 minutes to send messages until the join request is processed, assuming no other administrator contacted the user.
date	Integer	Date the request was sent in Unix time
bio	String	*Optional*. Bio of the user
invite_link	ChatInviteLink	*Optional*. Chat invite link that was used by the user to send the join request
query_id	String	*Optional*. Identifier of the join request query; for bots assigned to process join requests only. If present, then the bot must call sendChatJoinRequestWebApp or directly call answerChatJoinRequestQuery within 10 seconds.

ChatPermissions

Describes actions that a non-administrator user is allowed to take in a chat.

Field	Type	Description
can_send_messages	Boolean	*Optional*. *True*, if the user is allowed to send text messages, rich messages, contacts, giveaways, giveaway winners, invoices, locations and venues
can_send_audios	Boolean	*Optional*. *True*, if the user is allowed to send audios
can_send_documents	Boolean	*Optional*. *True*, if the user is allowed to send documents
can_send_photos	Boolean	*Optional*. *True*, if the user is allowed to send photos
can_send_videos	Boolean	*Optional*. *True*, if the user is allowed to send videos
can_send_video_notes	Boolean	*Optional*. *True*, if the user is allowed to send video notes
can_send_voice_notes	Boolean	*Optional*. *True*, if the user is allowed to send voice notes

Field	Type	Description
can_send_polls	Boolean	*Optional*. *True*, if the user is allowed to send polls and checklists
can_send_other_messages	Boolean	*Optional*. *True*, if the user is allowed to send animations, games, stickers, widgets and use inline bots
can_add_web_page_previews	Boolean	*Optional*. *True*, if the user is allowed to add web page previews to their messages
can_react_to_messages	Boolean	*Optional*. *True*, if the user is allowed to react to messages. If omitted, defaults to the value of *can_send_messages*.
can_edit_tag	Boolean	*Optional*. *True*, if the user is allowed to edit their own tag. If omitted, defaults to the value of *can_pin_messages*.
can_change_info	Boolean	*Optional*. *True*, if the user is allowed to change the chat title, photo and other settings. Ignored in public supergroups.
can_invite_users	Boolean	*Optional*. *True*, if the user is allowed to invite new users to the chat
can_pin_messages	Boolean	*Optional*. *True*, if the user is allowed to pin messages. Ignored in public supergroups.
can_manage_topics	Boolean	*Optional*. *True*, if the user is allowed to create forum topics. If omitted, defaults to the value of can_pin_messages.

Birthdate

Describes the birthdate of a user.

Field	Type	Description
day	Integer	Day of the user's birth; 1-31
month	Integer	Month of the user's birth; 1-12
year	Integer	*Optional*. Year of the user's birth

BusinessIntro

Contains information about the start page settings of a Telegram Business account.

Field	Type	Description
title	String	*Optional*. Title text of the business intro
message	String	*Optional*. Message text of the business intro
sticker	Sticker	*Optional*. Sticker of the business intro

BusinessLocation

Contains information about the location of a Telegram Business account.

Field	Type	Description
address	String	Address of the business
location	Location	*Optional*. Location of the business

BusinessOpeningHoursInterval

Describes an interval of time during which a business is open.

Field	Type	Description
opening_minute	Integer	The minute's sequence number in a week, starting on Monday, marking the start of the time interval during which the business is open; 0 - 7 * 24 * 60
closing_minute	Integer	The minute's sequence number in a week, starting on Monday, marking the end of the time interval during which the business is open; 0 - 8 * 24 * 60

BusinessOpeningHours

Describes the opening hours of a business.

Field	Type	Description
time_zone_name	String	Unique name of the time zone for which the opening hours are defined
opening_hours	Array of BusinessOpeningHoursInterval	List of time intervals describing business opening hours

UserRating

This object describes the rating of a user based on their Telegram Star spendings.

Field	Type	Description
level	Integer	Current level of the user, indicating their reliability when purchasing digital goods and services. A higher level suggests a more trustworthy customer; a negative level is likely reason for concern.
rating	Integer	Numerical value of the user's rating; the higher the rating, the better
current_level_rating	Integer	The rating value required to get the current level
next_level_rating	Integer	*Optional*. The rating value required to get to the next level; omitted if the maximum level was reached

AreaPosition

Describes the position of a rectangle area on a story media or a TON wallet card.

Field	Type	Description
x_percentage	Float	The abscissa of the area's center, as a percentage of the media or the card width
y_percentage	Float	The ordinate of the area's center, as a percentage of the media or the card height
width_percentage	Float	The width of the area's rectangle, as a percentage of the media or the card width
height_percentage	Float	The height of the area's rectangle, as a percentage of the media or the card height
rotation_angle	Float	The clockwise rotation angle of the rectangle, in degrees; 0-360
corner_radius_percentage	Float	The radius of the rectangle corner rounding, as a percentage of the media or the card width

TonNft

This object represents an NFT in the TON blockchain.

Field	Type	Description
address	String	Address of the NFT
collection_address	String	*Optional*. Address of the collection to which the NFT belongs; omitted if unknown
index	String	Index of the NFT among all NFTs from the collection as a decimal number
name	String	Name of the NFT
description	String	Description of the NFT
attributes	Array of TonNftAttribute	*Optional*. Attributes of the NFT
gift	UniqueGift	*Optional*. The unique gift corresponding to the NFT

TonNftAttribute

This object represents an attribute of an NFT in the TON blockchain.

Field	Type	Description
trait_type	String	Trait type of the attribute
value	String	Value of the attribute

TonWalletCardSticker

This object describes a sticker on a TON wallet card.

Field	Type	Description
position	AreaPosition	Position of the sticker on the card
nft	TonNft	The NFT shown as the sticker

TonWalletCard

This object describes a TON wallet card, which is shown on the profile page of a user to everyone.

Field	Type	Description
address	String	Address of the wallet
amount	String	Balance of the wallet, in the *smallest units* of the cryptocurrency, as a decimal number
stickers	Array of TonWalletCardSticker	Stickers placed on the card
background_nft_address	String	*Optional*. Address of the NFT of one of the stickers, whose colors must be used for the card background; omitted if the default background must be used

EmojiType

This object describes an emoji. Currently, it can be one of

- EmojiTypeEmoji
- EmojiTypeCustomEmoji

EmojiTypeEmoji

A regular emoji.

Field	Type	Description
type	String	Type of the emoji, always “emoji”
emoji	String	The emoji

EmojiTypeCustomEmoji

A custom emoji.

Field	Type	Description
type	String	Type of the emoji, always “custom_emoji”
custom_emoji_id	String	Identifier of the custom emoji. Use getCustomEmojiStickers to get full information about the sticker.

ChatShareTheme

This object describes the theme of the screen for sharing a link to a user or a chat. Currently, it can be one of

- ChatShareThemeEmoji
- ChatShareThemeUniqueGift

ChatShareThemeEmoji

The theme with emoji placed on a background.

Field	Type	Description
type	String	Type of the theme, always “emoji”
emojis	Array of EmojiType	The emoji; 0-6 regular or custom emoji
background_backdrop_id	Integer	*Optional*. Identifier of the backdrop used as the background; omitted if the default background is used. Use getChatShareThemeBackdrops to get details about the backdrop.

ChatShareThemeUniqueGift

The theme based on a unique gift owned by the user or the chat.

Field	Type	Description
type	String	Type of the theme, always “unique_gift”
gift	UniqueGift	The gift

ChatShareThemeBackdrops

This object contains a list of backdrops, which can be used as the background of a theme of the screen for sharing a link to a user or a chat.

Field	Type	Description
backdrops	Array of ChatShareThemeBackdrop	The list of backdrops

ChatShareThemeBackdrop

This object describes a backdrop, which can be used as the background of a theme of the screen for sharing a link to a user or a chat.

Field	Type	Description
id	Integer	Unique identifier of the backdrop
name	String	Name of the backdrop

Field	Type	Description
colors	UniqueGiftBackdropColors	Colors of the backdrop

LocationAddress

Describes the physical address of a location.

Field	Type	Description
country_code	String	The two-letter ISO 3166-1 alpha-2 country code of the country where the location is located
state	String	*Optional*. State of the location
city	String	*Optional*. City of the location
street	String	*Optional*. Street address of the location

StoryAreaType

Describes the type of a clickable area on a story. Currently, it can be one of

- StoryAreaTypeLocation
- StoryAreaTypeSuggestedReaction
- StoryAreaTypeLink
- StoryAreaTypeWeather
- StoryAreaTypeUniqueGift

StoryAreaTypeLocation

Describes a story area pointing to a location. Currently, a story can have up to 10 location areas.

Field	Type	Description
type	String	Type of the area, always “location”
latitude	Float	Location latitude in degrees
longitude	Float	Location longitude in degrees
address	LocationAddress	*Optional*. Address of the location

StoryAreaTypeSuggestedReaction

Describes a story area pointing to a suggested reaction. Currently, a story can have up to 5 suggested reaction areas.

Field	Type	Description
type	String	Type of the area, always “suggested_reaction”
reaction_type	ReactionType	Type of the reaction

Field	Type	Description
is_dark	Boolean	*Optional*. Pass *True* if the reaction area has a dark background
is_flipped	Boolean	*Optional*. Pass *True* if reaction area corner is flipped

StoryAreaTypeLink

Describes a story area pointing to an HTTP or tg:// link. Currently, a story can have up to 3 link areas.

Field	Type	Description
type	String	Type of the area, always “link”
url	String	HTTP or tg:// URL to be opened when the area is clicked

StoryAreaTypeWeather

Describes a story area containing weather information. Currently, a story can have up to 3 weather areas.

Field	Type	Description
type	String	Type of the area, always “weather”
temperature	Float	Temperature, in degree Celsius
emoji	String	Emoji representing the weather
background_color	Integer	A color of the area background in the ARGB format

StoryAreaTypeUniqueGift

Describes a story area pointing to a unique gift. Currently, a story can have at most 1 unique gift area.

Field	Type	Description
type	String	Type of the area, always “unique_gift”
name	String	Unique name of the gift

StoryArea

Describes a clickable area on a story media.

Field	Type	Description
position	AreaPosition	Position of the area
type	StoryAreaType	Type of the area

ChatLocation

Represents a location to which a chat is connected.

Field	Type	Description
location	Location	The location to which the supergroup is connected. Can't be a live location.
address	String	Location address; 1-64 characters, as defined by the chat owner

ReactionType

This object describes the type of a reaction. Currently, it can be one of

- ReactionTypeEmoji
- ReactionTypeCustomEmoji
- ReactionTypePaid

ReactionTypeEmoji

The reaction is based on an emoji.

Field	Type	Description
type	String	Type of the reaction, always “emoji”
emoji	String	Reaction emoji. Currently, it can be one of "❤", "👍", "👎", "🔥", "🥰", "👏", "😁", "🤔", "🤯", "😱", "🤬", "😢", "🎉", "🤩", "🤮", "💩", "🙏", "👌", "🕊", "🤡", "🥱", "🥴", "😍", "🐳", "❤‍🔥", "🌚", "🌭", "💯", "🤣", "⚡", "🍌", "🏆", "💔", "🤨", "😐", "🍓", "🍾", "💋", "🖕", "😈", "😴", "😭", "🤓", "👻", "👨‍💻", "👀", "🎃", "🙈", "😇", "😨", "🤝", "✍", "🤗", "🫡", "🎅", "🎄", "☃", "💅", "🤪", "🗿", "🆒", "💘", "🙉", "🦄", "😘", "💊", "🙊", "😎", "👾", "🤷‍♂", "🤷", "🤷‍♀", "😡".

ReactionTypeCustomEmoji

The reaction is based on a custom emoji.

Field	Type	Description
type	String	Type of the reaction, always “custom_emoji”
custom_emoji_id	String	Custom emoji identifier

ReactionTypePaid

The reaction is paid.

Field	Type	Description
type	String	Type of the reaction, always “paid”

ReactionCount

Represents a reaction added to a message along with the number of times it was added.

Field	Type	Description
type	ReactionType	Type of the reaction
total_count	Integer	Number of times the reaction was added

MessageReactionUpdated

This object represents a change of a reaction on a message performed by a user.

Field	Type	Description
chat	Chat	The chat containing the message the user reacted to
message_id	Integer	Unique identifier of the message inside the chat
user	User	*Optional*. The user that changed the reaction, if the user isn't anonymous
actor_chat	Chat	*Optional*. The chat on behalf of which the reaction was changed, if the user is anonymous
date	Integer	Date of the change in Unix time
old_reaction	Array of ReactionType	Previous list of reaction types that were set by the user
new_reaction	Array of ReactionType	New list of reaction types that have been set by the user

MessageReactionCountUpdated

This object represents reaction changes on a message with anonymous reactions.

Field	Type	Description
chat	Chat	The chat containing the message
message_id	Integer	Unique message identifier inside the chat
date	Integer	Date of the change in Unix time
reactions	Array of ReactionCount	List of reactions that are present on the message

ForumTopic

This object represents a forum topic.

Field	Type	Description
message_thread_id	Integer	Unique identifier of the forum topic

Field	Type	Description
name	String	Name of the topic
icon_color	Integer	Color of the topic icon in RGB format
icon_custom_emoji_id	String	*Optional*. Unique identifier of the custom emoji shown as the topic icon
is_name_implicit	True	*Optional*. *True*, if the name of the topic wasn't specified explicitly by its creator and likely needs to be changed by the bot

GiftBackground

This object describes the background of a gift.

Field	Type	Description
center_color	Integer	Center color of the background in RGB format
edge_color	Integer	Edge color of the background in RGB format
text_color	Integer	Text color of the background in RGB format

Gift

This object represents a gift that can be sent by the bot.

Field	Type	Description
id	String	Unique identifier of the gift
sticker	Sticker	The sticker that represents the gift
star_count	Integer	The number of Telegram Stars that must be paid to send the sticker
upgrade_star_count	Integer	*Optional*. The number of Telegram Stars that must be paid to upgrade the gift to a unique one
is_premium	True	*Optional*. *True*, if the gift can only be purchased by Telegram Premium subscribers
has_colors	True	*Optional*. *True*, if the gift can be used (after being upgraded) to customize a user's appearance
total_count	Integer	*Optional*. The total number of gifts of this type that can be sent by all users; for limited gifts only
remaining_count	Integer	*Optional*. The number of remaining gifts of this type that can be sent by all users; for limited gifts only
personal_total_count	Integer	*Optional*. The total number of gifts of this type that can be sent by the bot; for limited gifts only
personal_remaining_count	Integer	*Optional*. The number of remaining gifts of this type that can be sent by the bot; for limited gifts only

Field	Type	Description
background	GiftBackground	*Optional*. Background of the gift
unique_gift_variant_count	Integer	*Optional*. The total number of different unique gifts that can be obtained by upgrading the gift
publisher_chat	Chat	*Optional*. Information about the chat that published the gift

Gifts

This object represent a list of gifts.

Field	Type	Description
gifts	Array of Gift	The list of gifts

UniqueGiftModel

This object describes the model of a unique gift.

Field	Type	Description
name	String	Name of the model
sticker	Sticker	The sticker that represents the unique gift
rarity_per_mille	Integer	The number of unique gifts that receive this model for every 1000 gift upgrades. Always 0 for crafted gifts.
rarity	String	*Optional*. Rarity of the model if it is a crafted model. Currently, can be “uncommon”, “rare”, “epic”, or “legendary”.

UniqueGiftSymbol

This object describes the symbol shown on the pattern of a unique gift.

Field	Type	Description
name	String	Name of the symbol
sticker	Sticker	The sticker that represents the unique gift
rarity_per_mille	Integer	The number of unique gifts that receive this model for every 1000 gifts upgraded

UniqueGiftBackdropColors

This object describes the colors of the backdrop of a unique gift.

Field	Type	Description
center_color	Integer	The color in the center of the backdrop in RGB format

Field	Type	Description
edge_color	Integer	The color on the edges of the backdrop in RGB format
symbol_color	Integer	The color to be applied to the symbol in RGB format
text_color	Integer	The color for the text on the backdrop in RGB format

UniqueGiftBackdrop

This object describes the backdrop of a unique gift.

Field	Type	Description
name	String	Name of the backdrop
colors	UniqueGiftBackdropColors	Colors of the backdrop
rarity_per_mille	Integer	The number of unique gifts that receive this backdrop for every 1000 gifts upgraded

UniqueGiftColors

This object contains information about the color scheme for a user's name, message replies and link previews based on a unique gift.

Field	Type	Description
model_custom_emoji_id	String	Custom emoji identifier of the unique gift's model
symbol_custom_emoji_id	String	Custom emoji identifier of the unique gift's symbol
light_theme_main_color	Integer	Main color used in light themes; RGB format
light_theme_other_colors	Array of Integer	List of 1-3 additional colors used in light themes; RGB format
dark_theme_main_color	Integer	Main color used in dark themes; RGB format
dark_theme_other_colors	Array of Integer	List of 1-3 additional colors used in dark themes; RGB format

UniqueGift

This object describes a unique gift that was upgraded from a regular gift.

Field	Type	Description
gift_id	String	Identifier of the regular gift from which the gift was upgraded
base_name	String	Human-readable name of the regular gift from which this unique gift was upgraded
name	String	Unique name of the gift. This name can be used in `https://t.me/nft/...` links and story areas.

Field	Type	Description
number	Integer	Unique number of the upgraded gift among gifts upgraded from the same regular gift
model	UniqueGiftModel	Model of the gift
symbol	UniqueGiftSymbol	Symbol of the gift
backdrop	UniqueGiftBackdrop	Backdrop of the gift
is_premium	True	*Optional*. *True*, if the original regular gift was exclusively purchaseable by Telegram Premium subscribers
is_burned	True	*Optional*. *True*, if the gift was used to craft another gift and isn't available anymore
is_from_blockchain	True	*Optional*. *True*, if the gift is assigned from the TON blockchain and can't be resold or transferred in Telegram
colors	UniqueGiftColors	*Optional*. The color scheme that can be used by the gift's owner for the chat's name, replies to messages and link previews; for business account gifts and gifts that are currently on sale only
publisher_chat	Chat	*Optional*. Information about the chat that published the gift

GiftInfo

Describes a service message about a regular gift that was sent or received.

Field	Type	Description
gift	Gift	Information about the gift
owned_gift_id	String	*Optional*. Unique identifier of the received gift for the bot; only present for gifts received on behalf of business accounts
convert_star_count	Integer	*Optional*. Number of Telegram Stars that can be claimed by the receiver by converting the gift; omitted if conversion to Telegram Stars is impossible
prepaid_upgrade_star_count	Integer	*Optional*. Number of Telegram Stars that were prepaid for the ability to upgrade the gift
is_upgrade_separate	True	*Optional*. *True*, if the gift's upgrade was purchased after the gift was sent
can_be_upgraded	True	*Optional*. *True*, if the gift can be upgraded to a unique gift
text	String	*Optional*. Text of the message that was added to the gift
entities	Array of MessageEntity	*Optional*. Special entities that appear in the text
is_private	True	*Optional*. *True*, if the sender and gift text are shown only to the gift receiver; otherwise, everyone will be able to see them

Field	Type	Description
unique_gift_number	Integer	*Optional*. Unique number reserved for this gift when upgraded. See the *number* field in UniqueGift.

UniqueGiftInfo

Describes a service message about a unique gift that was sent or received.

Field	Type	Description
gift	UniqueGift	Information about the gift
origin	String	Origin of the gift. Currently, either “upgrade” for gifts upgraded from regular gifts, “transfer” for gifts transferred from other users or channels, “resale” for gifts bought from other users, “gifted_upgrade” for upgrades purchased after the gift was sent, or “offer” for gifts bought or sold through gift purchase offers.
text	String	*Optional*. Text of the message that was added to the gift
entities	Array of MessageEntity	*Optional*. Special entities that appear in the text
is_private	True	*Optional*. *True*, if the sender and gift text are shown only to the gift receiver; otherwise, everyone will be able to see them
last_resale_currency	String	*Optional*. For gifts bought from other users, the currency in which the payment for the gift was done. Currently, one of “XTR” for Telegram Stars or “TON” for TON grams.
last_resale_amount	Integer	*Optional*. For gifts bought from other users, the price paid for the gift in either Telegram Stars or nanograms
owned_gift_id	String	*Optional*. Unique identifier of the received gift for the bot; only present for gifts received on behalf of business accounts
transfer_star_count	Integer	*Optional*. Number of Telegram Stars that must be paid to transfer the gift; omitted if the bot cannot transfer the gift
next_transfer_date	Integer	*Optional*. Point in time (Unix timestamp) when the gift can be transferred. If it is in the past, then the gift can be transferred now.

OwnedGift

This object describes a gift received and owned by a user or a chat. Currently, it can be one of

- OwnedGiftRegular
- OwnedGiftUnique

OwnedGiftRegular

Describes a regular gift owned by a user or a chat.

Field	Type	Description
type	String	Type of the gift, always “regular”
gift	Gift	Information about the regular gift
owned_gift_id	String	*Optional*. Unique identifier of the gift for the bot; for gifts received on behalf of business accounts only
sender_user	User	*Optional*. Sender of the gift if it is a known user
send_date	Integer	Date the gift was sent in Unix time
text	String	*Optional*. Text of the message that was added to the gift
entities	Array of MessageEntity	*Optional*. Special entities that appear in the text
is_private	True	*Optional*. *True*, if the sender and gift text are shown only to the gift receiver; otherwise, everyone will be able to see them
is_saved	True	*Optional*. *True*, if the gift is displayed on the account's profile page; for gifts received on behalf of business accounts only
can_be_upgraded	True	*Optional*. *True*, if the gift can be upgraded to a unique gift; for gifts received on behalf of business accounts only
was_refunded	True	*Optional*. *True*, if the gift was refunded and isn't available anymore
convert_star_count	Integer	*Optional*. Number of Telegram Stars that can be claimed by the receiver instead of the gift; omitted if the gift cannot be converted to Telegram Stars; for gifts received on behalf of business accounts only
prepaid_upgrade_star_count	Integer	*Optional*. Number of Telegram Stars that were paid for the ability to upgrade the gift
is_upgrade_separate	True	*Optional*. *True*, if the gift's upgrade was purchased after the gift was sent; for gifts received on behalf of business accounts only
unique_gift_number	Integer	*Optional*. Unique number reserved for this gift when upgraded. See the *number* field in UniqueGift.

OwnedGiftUnique

Describes a unique gift received and owned by a user or a chat.

Field	Type	Description
type	String	Type of the gift, always “unique”
gift	UniqueGift	Information about the unique gift
owned_gift_id	String	*Optional*. Unique identifier of the received gift for the bot; for gifts received on behalf of business accounts only

Field	Type	Description
sender_user	User	*Optional*. Sender of the gift if it is a known user
send_date	Integer	Date the gift was sent in Unix time
is_saved	True	*Optional*. *True*, if the gift is displayed on the account's profile page; for gifts received on behalf of business accounts only
can_be_transferred	True	*Optional*. *True*, if the gift can be transferred to another owner; for gifts received on behalf of business accounts only
transfer_star_count	Integer	*Optional*. Number of Telegram Stars that must be paid to transfer the gift; omitted if the bot cannot transfer the gift
next_transfer_date	Integer	*Optional*. Point in time (Unix timestamp) when the gift can be transferred. If it is in the past, then the gift can be transferred now.

OwnedGifts

Contains the list of gifts received and owned by a user or a chat.

Field	Type	Description
total_count	Integer	The total number of gifts owned by the user or the chat
gifts	Array of OwnedGift	The list of gifts
next_offset	String	*Optional*. Offset for the next request. If empty, then there are no more results.

BotAccessSettings

This object describes the access settings of a bot.

Field	Type	Description
is_access_restricted	Boolean	*True*, if only selected users can access the bot. The bot's owner can always access it.
added_users	Array of User	*Optional*. The list of other users who have access to the bot if the access is restricted

AcceptedGiftTypes

This object describes the types of gifts that can be gifted to a user or a chat.

Field	Type	Description
unlimited_gifts	Boolean	*True*, if unlimited regular gifts are accepted
limited_gifts	Boolean	*True*, if limited regular gifts are accepted
unique_gifts	Boolean	*True*, if unique gifts or gifts that can be upgraded to unique for free are accepted

Field	Type	Description
premium_subscription	Boolean	*True*, if a Telegram Premium subscription is accepted
gifts_from_channels	Boolean	*True*, if transfers of unique gifts from channels are accepted

StarAmount

Describes an amount of Telegram Stars.

Field	Type	Description
amount	Integer	Integer amount of Telegram Stars, rounded to 0; can be negative
nanostar_amount	Integer	*Optional*. The number of 1/1000000000 shares of Telegram Stars; from -999999999 to 999999999; can be negative if and only if *amount* is non-positive

BotCommand

This object represents a bot command.

Field	Type	Description
command	String	Text of the command; 1-32 characters. Can contain only lowercase English letters, digits and underscores.
description	String	Description of the command; 1-256 characters
is_ephemeral	Boolean	*Optional*. *True*, if the command sends an ephemeral message, which can be seen only by the sender of the message and the bot

BotCommandScope

This object represents the scope to which bot commands are applied. Currently, the following 7 scopes are supported:

- BotCommandScopeDefault
- BotCommandScopeAllPrivateChats
- BotCommandScopeAllGroupChats
- BotCommandScopeAllChatAdministrators
- BotCommandScopeChat
- BotCommandScopeChatAdministrators
- BotCommandScopeChatMember

Determining list of commands

The following algorithm is used to determine the list of commands for a particular user viewing the bot menu. The first list of commands which is set is returned:

Commands in the chat with the bot

- botCommandScopeChat + language_code
- botCommandScopeChat

- botCommandScopeAllPrivateChats + language_code
- botCommandScopeAllPrivateChats
- botCommandScopeDefault + language_code
- botCommandScopeDefault

Commands in group and supergroup chats

- botCommandScopeChatMember + language_code
- botCommandScopeChatMember
- botCommandScopeChatAdministrators + language_code (administrators only)
- botCommandScopeChatAdministrators (administrators only)
- botCommandScopeChat + language_code
- botCommandScopeChat
- botCommandScopeAllChatAdministrators + language_code (administrators only)
- botCommandScopeAllChatAdministrators (administrators only)
- botCommandScopeAllGroupChats + language_code
- botCommandScopeAllGroupChats
- botCommandScopeDefault + language_code
- botCommandScopeDefault

BotCommandScopeDefault

Represents the default scope of bot commands. Default commands are used if no commands with a narrower scope are specified for the user.

Field	Type	Description
type	String	Scope type, must be *default*

BotCommandScopeAllPrivateChats

Represents the scope of bot commands, covering all private chats.

Field	Type	Description
type	String	Scope type, must be *all_private_chats*

BotCommandScopeAllGroupChats

Represents the scope of bot commands, covering all group and supergroup chats.

Field	Type	Description
type	String	Scope type, must be *all_group_chats*

BotCommandScopeAllChatAdministrators

Represents the scope of bot commands, covering all group and supergroup chat administrators.

Field	Type	Description
type	String	Scope type, must be *all_chat_administrators*

BotCommandScopeChat

Represents the scope of bot commands, covering a specific chat.

Field	Type	Description
type	String	Scope type, must be *chat*
chat_id	Integer or String	Unique identifier for the target chat or username of the target supergroup in the format `@username`. Channel direct messages chats and channel chats aren't supported.

BotCommandScopeChatAdministrators

Represents the scope of bot commands, covering all administrators of a specific group or supergroup chat.

Field	Type	Description
type	String	Scope type, must be *chat_administrators*
chat_id	Integer or String	Unique identifier for the target chat or username of the target supergroup in the format `@username`. Channel direct messages chats and channel chats aren't supported.

BotCommandScopeChatMember

Represents the scope of bot commands, covering a specific member of a group or supergroup chat.

Field	Type	Description
type	String	Scope type, must be *chat_member*
chat_id	Integer or String	Unique identifier for the target chat or username of the target supergroup in the format `@username`. Channel direct messages chats and channel chats aren't supported.
user_id	Integer	Unique identifier of the target user

BotName

This object represents the bot's name.

Field	Type	Description
name	String	The bot's name

BotDescription

This object represents the bot's description.

Field	Type	Description
description	String	The bot's description

BotShortDescription

This object represents the bot's short description.

Field	Type	Description
short_description	String	The bot's short description

MenuButton

This object describes the bot's menu button in a private chat. It should be one of

- MenuButtonCommands
- MenuButtonWebApp
- MenuButtonDefault

If a menu button other than MenuButtonDefault is set for a private chat, then it is applied in the chat. Otherwise the default menu button is applied. By default, the menu button opens the list of bot commands.

MenuButtonCommands

Represents a menu button, which opens the bot's list of commands.

Field	Type	Description
type	String	Type of the button, must be *commands*

MenuButtonWebApp

Represents a menu button, which launches a Web App.

Field	Type	Description
type	String	Type of the button, must be *web_app*
text	String	Text on the button
web_app	WebAppInfo	Description of the Web App that will be launched when the user presses the button. The Web App will be able to send an arbitrary message on behalf of the user using the method answerWebAppQuery. Alternatively, a `t.me` link to a Web App of the bot can be specified in the object instead of the Web App's URL, in which case the Web App will be opened as if the user pressed the link.

MenuButtonDefault

Describes that no specific value for the menu button was set.

Field	Type	Description
type	String	Type of the button, must be *default*

ChatBoostSource

This object describes the source of a chat boost. It can be one of

- ChatBoostSourcePremium
- ChatBoostSourceGiftCode
- ChatBoostSourceGiveaway

ChatBoostSourcePremium

The boost was obtained by subscribing to Telegram Premium or by gifting a Telegram Premium subscription to another user.

Field	Type	Description
source	String	Source of the boost, always “premium”
user	User	User that boosted the chat

ChatBoostSourceGiftCode

The boost was obtained by the creation of Telegram Premium gift codes to boost a chat. Each such code boosts the chat 4 times for the duration of the corresponding Telegram Premium subscription.

Field	Type	Description
source	String	Source of the boost, always “gift_code”
user	User	User for which the gift code was created

ChatBoostSourceGiveaway

The boost was obtained by the creation of a Telegram Premium or a Telegram Star giveaway. This boosts the chat 4 times for the duration of the corresponding Telegram Premium subscription for Telegram Premium giveaways and *prize_star_count* / 500 times for one year for Telegram Star giveaways.

Field	Type	Description
source	String	Source of the boost, always “giveaway”
giveaway_message_id	Integer	Identifier of a message in the chat with the giveaway; the message could have been deleted already. May be 0 if the message isn't sent yet.
user	User	*Optional*. User that won the prize in the giveaway if any; for Telegram Premium giveaways only

Field	Type	Description
prize_star_count	Integer	*Optional*. The number of Telegram Stars to be split between giveaway winners; for Telegram Star giveaways only
is_unclaimed	True	*Optional*. *True*, if the giveaway was completed, but there was no user to win the prize

ChatBoost

This object contains information about a chat boost.

Field	Type	Description
boost_id	String	Unique identifier of the boost
add_date	Integer	Point in time (Unix timestamp) when the chat was boosted
expiration_date	Integer	Point in time (Unix timestamp) when the boost will automatically expire, unless the booster's Telegram Premium subscription is prolonged
source	ChatBoostSource	Source of the added boost

ChatBoostUpdated

This object represents a boost added to a chat or changed.

Field	Type	Description
chat	Chat	Chat which was boosted
boost	ChatBoost	Information about the chat boost

ChatBoostRemoved

This object represents a boost removed from a chat.

Field	Type	Description
chat	Chat	Chat which was boosted
boost_id	String	Unique identifier of the boost
remove_date	Integer	Point in time (Unix timestamp) when the boost was removed
source	ChatBoostSource	Source of the removed boost

ChatOwnerLeft

Describes a service message about the chat owner leaving the chat.

Field	Type	Description
new_owner	User	*Optional*. The user who will become the new owner of the chat if the previous owner does not return to the chat

ChatOwnerChanged

Describes a service message about an ownership change in the chat.

Field	Type	Description
new_owner	User	The new owner of the chat

UserChatBoosts

This object represents a list of boosts added to a chat by a user.

Field	Type	Description
boosts	Array of ChatBoost	The list of boosts added to the chat by the user

BusinessBotRights

Represents the rights of a business bot.

Field	Type	Description
can_reply	True	*Optional*. *True*, if the bot can send and edit messages in the private chats that had incoming messages in the last 24 hours
can_read_messages	True	*Optional*. *True*, if the bot can mark incoming private messages as read
can_delete_sent_messages	True	*Optional*. *True*, if the bot can delete messages sent by the bot
can_delete_all_messages	True	*Optional*. *True*, if the bot can delete all private messages in managed chats
can_edit_name	True	*Optional*. *True*, if the bot can edit the first and last name of the business account
can_edit_bio	True	*Optional*. *True*, if the bot can edit the bio of the business account
can_edit_profile_photo	True	*Optional*. *True*, if the bot can edit the profile photo of the business account
can_edit_username	True	*Optional*. *True*, if the bot can edit the username of the business account
can_change_gift_settings	True	*Optional*. *True*, if the bot can change the privacy settings pertaining to gifts for the business account
can_view_gifts_and_stars	True	*Optional*. *True*, if the bot can view gifts and the amount of Telegram Stars owned by the business account
can_convert_gifts_to_stars	True	*Optional*. *True*, if the bot can convert regular gifts owned by the business account to Telegram Stars

Field	Type	Description
can_transfer_and_upgrade_gifts	True	*Optional*. *True*, if the bot can transfer and upgrade gifts owned by the business account
can_transfer_stars	True	*Optional*. *True*, if the bot can transfer Telegram Stars received by the business account to its own account, or use them to upgrade and transfer gifts
can_manage_stories	True	*Optional*. *True*, if the bot can post, edit and delete stories on behalf of the business account

BusinessConnection

Describes the connection of the bot with a business account.

Field	Type	Description
id	String	Unique identifier of the business connection
user	User	Business account user that created the business connection
user_chat_id	Integer	Identifier of a private chat with the user who created the business connection. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a 64-bit integer or double-precision float type are safe for storing this identifier.
date	Integer	Date the connection was established in Unix time
rights	BusinessBotRights	*Optional*. Rights of the business bot
is_enabled	Boolean	*True*, if the connection is active

BusinessMessagesDeleted

This object is received when messages are deleted from a connected business account.

Field	Type	Description
business_connection_id	String	Unique identifier of the business connection
chat	Chat	Information about a chat in the business account. The bot may not have access to the chat or the corresponding user.
message_ids	Array of Integer	The list of identifiers of deleted messages in the chat of the business account

SentWebAppMessage

Describes an inline message sent by a Web App on behalf of a user.

Field	Type	Description
inline_message_id	String	*Optional*. Identifier of the sent inline message. Available only if there is an inline keyboard attached to the message.

SentGuestMessage

Describes an inline message sent by a guest bot.

Field	Type	Description
inline_message_id	String	Identifier of the sent inline message

PreparedInlineMessage

Describes an inline message to be sent by a user of a Mini App.

Field	Type	Description
id	String	Unique identifier of the prepared message
expiration_date	Integer	Expiration date of the prepared message, in Unix time. Expired prepared messages can no longer be used.

PreparedKeyboardButton

Describes a keyboard button to be used by a user of a Mini App.

Field	Type	Description
id	String	Unique identifier of the keyboard button

ResponseParameters

Describes why a request was unsuccessful.

Field	Type	Description
migrate_to_chat_id	Integer	*Optional*. The group has been migrated to a supergroup with the specified identifier. This number may have more than 32 significant bits and some programming languages may have difficulty/silent defects in interpreting it. But it has at most 52 significant bits, so a signed 64-bit integer or double-precision float type are safe for storing this identifier.
retry_after	Integer	*Optional*. In case of exceeding flood control, the number of seconds left to wait before the request can be repeated

InputMedia

This object represents the content of a media message to be sent. It should be one of

- InputMediaAnimation

- InputMediaAudio
- InputMediaDocument
- InputMediaLivePhoto
- InputMediaPhoto
- InputMediaVideo

InputMediaAnimation

Represents an animation file (GIF or H.264/MPEG-4 AVC video without sound) to be sent.

Field	Type	Description
type	String	Type of the media, must be *animation*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
thumbnail	String	*Optional*. Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
caption	String	*Optional*. Caption of the animation to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the animation caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
width	Integer	*Optional*. Animation width
height	Integer	*Optional*. Animation height
duration	Integer	*Optional*. Animation duration in seconds
has_spoiler	Boolean	*Optional*. Pass *True* if the animation needs to be covered with a spoiler animation

InputMediaAudio

Represents an audio file to be treated as music to be sent.

Field	Type	Description
type	String	Type of the media, must be *audio*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
thumbnail	String	*Optional*. Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
caption	String	*Optional*. Caption of the audio to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the audio caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
duration	Integer	*Optional*. Duration of the audio in seconds
performer	String	*Optional*. Performer of the audio
title	String	*Optional*. Title of the audio

InputMediaDocument

Represents a general file to be sent.

Field	Type	Description
type	String	Type of the media, must be *document*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »

Field	Type	Description
thumbnail	String	*Optional*. Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
caption	String	*Optional*. Caption of the document to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the document caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
disable_content_type_detection	Boolean	*Optional*. Disables automatic server-side content type detection for files uploaded using multipart/form-data. Always *True*, if the document is sent as part of an album.

InputMediaLink

Represents an HTTP link to be sent.

Field	Type	Description
type	String	Type of the media, must be *link*
url	String	HTTP URL of the link

InputMediaLivePhoto

Represents a live photo to be sent.

Field	Type	Description
type	String	Type of the media, must be *live_photo*
media	String	Video of the live photo to send. Pass a file_id to send a file that exists on the Telegram servers (recommended) or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files ». Sending live photos by a URL is currently unsupported.

Field	Type	Description
photo	String	The static photo to send. Pass a file_id to send a file that exists on the Telegram servers (recommended) or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files ». Sending live photos by a URL is currently unsupported.
caption	String	*Optional*. Caption of the live photo to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the live photo caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	*Optional*. Pass *True* if the live photo needs to be covered with a spoiler animation

InputMediaLocation

Represents a location to be sent.

Field	Type	Description
type	String	Type of the media, must be *location*
latitude	Float	Latitude of the location
longitude	Float	Longitude of the location
horizontal_accuracy	Float	*Optional*. The radius of uncertainty for the location, measured in meters; 0-1500

InputMediaPhoto

Represents a photo to be sent.

Field	Type	Description
type	String	Type of the media, must be *photo*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
caption	String	*Optional*. Caption of the photo to be sent, 0-1024 characters after entities parsing

Field	Type	Description
parse_mode	String	*Optional*. Mode for parsing entities in the photo caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	*Optional*. Pass *True* if the photo needs to be covered with a spoiler animation

InputMediaSticker

Represents a sticker file to be sent.

Field	Type	Description
type	String	Type of the media, must be *sticker*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a .WEBP sticker from the Internet, or pass “attach://<file_attach_name>” to upload a new .WEBP, .TGS, or .WEBM sticker using multipart/form-data under <file_attach_name> name. More information on Sending Files »
emoji	String	*Optional*. Emoji associated with the sticker; only for just uploaded stickers

InputMediaVenue

Represents a venue to be sent.

Field	Type	Description
type	String	Type of the media, must be *venue*
latitude	Float	Latitude of the location
longitude	Float	Longitude of the location
title	String	Name of the venue
address	String	Address of the venue
foursquare_id	String	*Optional*. Foursquare identifier of the venue
foursquare_type	String	*Optional*. Foursquare type of the venue, if known. (For example, “arts_entertainment/default”, “arts_entertainment/aquarium” or “food/icecream”.)
google_place_id	String	*Optional*. Google Places identifier of the venue
google_place_type	String	*Optional*. Google Places type of the venue. (See supported types.)

InputMediaVideo

Represents a video to be sent.

Field	Type	Description
type	String	Type of the media, must be *video*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
thumbnail	String	*Optional*. Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
cover	String	*Optional*. Cover for the video in the message. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
start_timestamp	Integer	*Optional*. Start timestamp for the video in the message
caption	String	*Optional*. Caption of the video to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the video caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
width	Integer	*Optional*. Video width
height	Integer	*Optional*. Video height
duration	Integer	*Optional*. Video duration in seconds
supports_streaming	Boolean	*Optional*. Pass *True* if the uploaded video is suitable for streaming
has_spoiler	Boolean	*Optional*. Pass *True* if the video needs to be covered with a spoiler animation

InputMediaVoiceNote

Represents a voice message file to be sent.

Field	Type	Description
type	String	Type of the media, must be *voice_note*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass "attach://<file_attach_name>" to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
caption	String	*Optional*. Caption of the voice message to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the voice message caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
duration	Integer	*Optional*. Duration of the voice message in seconds

InputFile

This object represents the contents of a file to be uploaded. Must be posted using multipart/form-data in the usual way that files are uploaded via the browser.

InputPaidMedia

This object describes the paid media to be sent. Currently, it can be one of

- InputPaidMediaLivePhoto
- InputPaidMediaPhoto
- InputPaidMediaVideo

InputPaidMediaLivePhoto

The paid media to send is a live photo.

Field	Type	Description
type	String	Type of the media, must be *live_photo*
media	String	Video of the live photo to send. Pass a file_id to send a file that exists on the Telegram servers (recommended) or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files ». Sending live photos by a URL is currently unsupported.
photo	String	The static photo to send. Pass a file_id to send a file that exists on the Telegram servers (recommended) or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files ». Sending live photos by a URL is currently unsupported.

InputPaidMediaPhoto

The paid media to send is a photo.

Field	Type	Description
type	String	Type of the media, must be *photo*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »

InputPaidMediaVideo

The paid media to send is a video.

Field	Type	Description
type	String	Type of the media, must be *video*
media	String	File to send. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
thumbnail	String	*Optional*. Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
cover	String	*Optional*. Cover for the video in the message. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
start_timestamp	Integer	*Optional*. Start timestamp for the video in the message
width	Integer	*Optional*. Video width
height	Integer	*Optional*. Video height
duration	Integer	*Optional*. Video duration in seconds
supports_streaming	Boolean	*Optional*. Pass *True* if the uploaded video is suitable for streaming

InputProfilePhoto

This object describes a profile photo to set. Currently, it can be one of

- InputProfilePhotoStatic
- InputProfilePhotoAnimated

InputProfilePhotoStatic

A static profile photo in the .JPG format.

Field	Type	Description
type	String	Type of the profile photo, must be *static*
photo	String	The static profile photo. Profile photos can't be reused and can only be uploaded as a new file, so you can pass “attach://<file_attach_name>” if the photo was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »

InputProfilePhotoAnimated

An animated profile photo in the MPEG4 format.

Field	Type	Description
type	String	Type of the profile photo, must be *animated*
animation	String	The animated profile photo. Profile photos can't be reused and can only be uploaded as a new file, so you can pass “attach://<file_attach_name>” if the photo was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
main_frame_timestamp	Float	*Optional*. Timestamp in seconds of the frame that will be used as the static profile photo. Defaults to 0.0.

InputStoryContent

This object describes the content of a story to post. Currently, it can be one of

- InputStoryContentPhoto
- InputStoryContentVideo

InputStoryContentPhoto

Describes a photo to post as a story.

Field	Type	Description
type	String	Type of the content, must be *photo*
photo	String	The photo to post as a story. The photo must be of the size 1080x1920 and must not exceed 10 MB. The photo can't be reused and can only be uploaded as a new file, so you can pass “attach://<file_attach_name>” if the photo was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »

InputStoryContentVideo

Describes a video to post as a story.

Field	Type	Description
type	String	Type of the content, must be *video*
video	String	The video to post as a story. The video must be of the size 720x1280, streamable, encoded with H.265 codec, with key frames added each second in the MPEG4 format, and must not exceed 30 MB. The video can't be reused and can only be uploaded as a new file, so you can pass “attach://<file_attach_name>” if the video was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
duration	Float	*Optional*. Precise duration of the video in seconds; 0-60
cover_frame_timestamp	Float	*Optional*. Timestamp in seconds of the frame that will be used as the static cover for the story. Defaults to 0.0.
is_animation	Boolean	*Optional*. Pass *True* if the video has no sound

Sending files

There are three ways to send files (photos, stickers, audio, media, etc.):

1. If the file is already stored somewhere on the Telegram servers, you don't need to reupload it: each file object has a **file_id** field, simply pass this **file_id** as a parameter instead of uploading. There are **no limits** for files sent this way.
2. Provide Telegram with an HTTP URL for the file to be sent. Telegram will download and send the file. 5 MB max size for photos and 20 MB max for other types of content.
3. Post the file using multipart/form-data in the usual way that files are uploaded via the browser. 10 MB max size for photos, 50 MB for other files.

Sending by file_id

- It is not possible to change the file type when resending by **file_id**. I.e. a video can't be sent as a photo, a photo can't be sent as a document, etc.
- It is not possible to resend thumbnails.
- Resending a photo by **file_id** will send all of its sizes.
- **file_id** is unique for each individual bot and **can't** be transferred from one bot to another.
- **file_id** uniquely identifies a file, but a file can have different valid **file_id**s even for the same bot.

Sending by URL

- When sending by URL the target file must have the correct MIME type (e.g., audio/mpeg for sendAudio, etc.).
- In sendDocument, sending by URL will currently only work for **.PDF** and **.ZIP** files.
- To use sendVoice, the file must have the type audio/ogg and be no more than 1MB in size. 1-20MB voice notes will be sent as files.
- Other configurations may work but we can't guarantee that they will.

Accent colors

Colors with identifiers 0 (red), 1 (orange), 2 (purple/violet), 3 (green), 4 (cyan), 5 (blue), 6 (pink) can be customized by app themes. Additionally, the following colors in RGB format are currently in use.

Color identifier	Light colors	Dark colors
7	E15052 F9AE63	FF9380 992F37
8	E0802B FAC534	ECB04E C35714
9	A05FF3 F48FFF	C697FF 5E31C8
10	27A910 A7DC57	A7EB6E 167E2D
11	27ACCE 82E8D6	40D8D0 045C7F
12	3391D4 7DD3F0	52BFFF 0B5494
13	DD4371 FFBE9F	FF86A6 8E366E
14	247BED F04856 FFFFFF	3FA2FE E5424F FFFFFF
15	D67722 1EA011 FFFFFF	FF905E 32A527 FFFFFF
16	179E42 E84A3F FFFFFF	66D364 D5444F FFFFFF
17	2894AF 6FC456 FFFFFF	22BCE2 3DA240 FFFFFF
18	0C9AB3 FFAD95 FFE6B5	22BCE2 FF9778 FFDA6B
19	7757D6 F79610 FFDE8E	9791FF F2731D FFDB59
20	1585CF F2AB1D FFFFFF	3DA6EB EEA51D FFFFFF

Profile accent colors

Currently, the following colors in RGB format are in use for profile backgrounds.

Color identifier	Light colors	Dark colors
0	BA5650	9C4540
1	C27C3E	945E2C
2	956AC8	715099
3	49A355	33713B
4	3E97AD	387E87
5	5A8FBB	477194
6	B85378	944763
7	7F8B95	435261
8	C9565D D97C57	994343 AC583E

Color identifier	Light colors	Dark colors
9	CF7244 CC9433	8F552F A17232
10	9662D4 B966B6	634691 9250A2
11	3D9755 89A650	296A43 5F8F44
12	3D95BA 50AD98	306C7C 3E987E
13	538BC2 4DA8BD	38618C 458BA1
14	B04F74 D1666D	884160 A65259
15	637482 7B8A97	53606E 384654

Inline mode objects

Objects and methods used in the inline mode are described in the Inline mode section.

Available methods

> All methods in the Bot API are case-insensitive. We support **GET** and **POST** HTTP methods. Use either URL query string or *application/json* or *application/x-www-form-urlencoded* or *multipart/form-data* for passing parameters in Bot API requests.
> On successful call, a JSON-object containing the result will be returned.

getMe

A simple method for testing your bot's authentication token. Requires no parameters. Returns basic information about the bot in form of a User object.

logOut

Use this method to log out from the cloud Bot API server before launching the bot locally. You **must** log out the bot before running it locally, otherwise there is no guarantee that the bot will receive updates. After a successful call, you can immediately log in on a local server, but will not be able to log in back to the cloud Bot API server for 10 minutes. Returns *True* on success. Requires no parameters.

close

Use this method to close the bot instance before moving it from one local server to another. You need to delete the webhook before calling this method to ensure that the bot isn't launched again after server restart. The method will return error 429 in the first 10 minutes after the bot is launched. Returns *True* on success. Requires no parameters.

sendMessage

Use this method to send text messages. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
text	String	Yes	Text of the message to be sent, 1-4096 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the message text. See formatting options for more details.
entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in message text, which can be specified instead of *parse_mode*
link_preview_options	LinkPreviewOptions	Optional	Link preview generation options for the message
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.

Parameter	Type	Required	Description
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

Formatting options

The Bot API supports basic formatting for messages. You can use bold, italic, underlined, strikethrough, spoiler text, block quotations as well as inline links and pre-formatted code in your bots' messages. Telegram clients will render them accordingly. You can specify text entities directly, or use markdown-style or HTML-style formatting.

Note that Telegram clients will display an **alert** to the user before opening an inline link ('Open this link?' together with the full URL).

Message entities can be nested, providing following restrictions are met:
- If two entities have common characters, then one of them is fully contained inside another.
- *bold*, *italic*, *underline*, *strikethrough*, and *spoiler* entities can contain and can be part of any other

entities, except *pre* and *code*.
- *blockquote* and *expandable_blockquote* entities can't be nested.
- All other entities can't contain each other.

Links `tg://user?id=<user_id>` can be used to mention a user by their identifier without using a username. Please note:

- These links will work **only** if they are used inside an inline link or in an inline keyboard button. For example, they will not work, when used in a message text.
- Unless the user is a member of the chat where they were mentioned, these mentions are only guaranteed to work if the user has contacted the bot in private in the past or has sent a callback query to the bot via an inline button and doesn't have Forwarded Messages privacy enabled for the bot.

You can find the list of programming and markup languages for which syntax highlighting is supported at libprisma#supported-languages.

Date-time entity formatting

Date-time entity formatting is specified by a format string, which must adhere to the following regular expression: `r|w?[dD]?[tT]?`.

If the format string is empty, the underlying text is displayed as-is; however, the user can still receive the underlying date in their local format. When populated, the format string determines the output based on the presence of the following control characters:

- `r`: Displays the time relative to the current time. Cannot be combined with any other control characters.
- `w`: Displays the day of the week in the user's localized language.
- `d`: Displays the date in short form (e.g., “17.03.22”).
- `D`: Displays the date in long form (e.g., “March 17, 2022”).
- `t`: Displays the time in short form (e.g., “22:45”).
- `T`: Displays the time in long form (e.g., “22:45:00”).

MarkdownV2 style

To use this mode, pass *MarkdownV2* in the *parse_mode* field. Use the following syntax in your message:

```
*bold \*text*
_italic \*text_
__underline__
~strikethrough~
||spoiler||
*bold _italic bold ~italic bold strikethrough ||italic bold strikethrough
spoiler||~ __underline italic bold___ bold*
[inline URL](http://www.example.com/)
[inline mention of a user](tg://user?id=123456789)
![👍](tg://emoji?id=5368324170671202286)
![22:45 tomorrow](tg://time?unix=1647531900&format=wDT)
![22:45 tomorrow](tg://time?unix=1647531900&format=t)
![22:45 tomorrow](tg://time?unix=1647531900&format=r)
```

```
![22:45 tomorrow](tg://time?unix=1647531900)
`inline fixed-width code`
```
pre-formatted fixed-width code block
```
```python
pre-formatted fixed-width code block written in the Python programming language
```
>Block quotation started
>Block quotation continued
>Block quotation continued
>Block quotation continued
>The last line of the block quotation
**>The expandable block quotation started right after the previous block quotation
>It is separated from the previous block quotation by an empty bold entity
>Expandable block quotation continued
>Hidden by default part of the expandable block quotation started
>Expandable block quotation continued
>The last line of the expandable block quotation with the expandability mark||
```

Please note:

- Any character with code between 1 and 126 inclusively can be escaped anywhere with a preceding '\' character, in which case it is treated as an ordinary character and not a part of the markup. This implies that '\' character usually must be escaped with a preceding '\' character.
- Inside `pre` and `code` entities, all '`' and '\' characters must be escaped with a preceding '\' character.
- Inside the `(...)` part of the inline link and custom emoji definition, all ')' and '\' must be escaped with a preceding '\' character.
- In all other places characters '_', '*', '[', ']', '(', ')', '~', '`', '>', '#', '+', '-', '=', '|', '{', '}', '.', '!' must be escaped with the preceding character '\'.
- In case of ambiguity between `italic` and `underline` entities `__` is always greedily treated from left to right as beginning or end of an `underline` entity, so instead of `___italic underline___` use `___italic underline_**__`, adding an empty bold entity as a separator.
- A valid emoji must be provided as an alternative value for the custom emoji. The emoji will be shown instead of the custom emoji in places where a custom emoji cannot be displayed (e.g., system notifications) or if the message is forwarded by a non-premium user. It is recommended to use the emoji from the **emoji** field of the custom emoji sticker.
- Custom emoji entities can only be used by bots that purchased additional usernames on Fragment or in the messages directly sent by the bot to private, group and supergroup chats if the owner of the bot has a Telegram Premium subscription.
- See date-time entity formatting for more details about supported date-time formats.

HTML style

To use this mode, pass *HTML* in the *parse_mode* field. The following tags are currently supported:

```
<b>bold</b>, <strong>bold</strong>
<i>italic</i>, <em>italic</em>
<u>underline</u>, <ins>underline</ins>
```

```
<s>strikethrough</s>, <strike>strikethrough</strike>, <del>strikethrough</del>
<span class="tg-spoiler">spoiler</span>, <tg-spoiler>spoiler</tg-spoiler>
<b>bold <i>italic bold <s>italic bold strikethrough <span class="tg-spoiler">italic
bold strikethrough spoiler</span></s> <u>underline italic bold</u></i> bold</b>
<a href="http://www.example.com/">inline URL</a>
<a href="tg://user?id=123456789">inline mention of a user</a>
<tg-emoji emoji-id="5368324170671202286">👍</tg-emoji>
<tg-time unix="1647531900" format="wDT">22:45 tomorrow</tg-time>
<tg-time unix="1647531900" format="t">22:45 tomorrow</tg-time>
<tg-time unix="1647531900" format="r">22:45 tomorrow</tg-time>
<tg-time unix="1647531900">22:45 tomorrow</tg-time>
<code>inline fixed-width code</code>
<pre>pre-formatted fixed-width code block</pre>
<pre><code class="language-python">pre-formatted fixed-width code block written in
the Python programming language</code></pre>
<blockquote>Block quotation started
Block quotation continued
The last line of the block quotation</blockquote>
<blockquote expandable>Expandable block quotation started
Expandable block quotation continued
Expandable block quotation continued
Hidden by default part of the block quotation started
Expandable block quotation continued
The last line of the block quotation</blockquote>
```

Please note:

- Only the tags mentioned above are currently supported.
- All `<`, `>` and `&` symbols that are not a part of a tag or an HTML entity must be replaced with the corresponding HTML entities (`<` with `<`, `>` with `>` and `&` with `&`).
- All numerical HTML entities are supported.
- The API currently supports only the following named HTML entities: `<`, `>`, `&` and `"`.
- Use nested `pre` and `code` tags, to define programming language for `pre` entity.
- Programming language can't be specified for standalone `code` tags.
- A valid emoji must be used as the content of the `tg-emoji` tag. The emoji will be shown instead of the custom emoji in places where a custom emoji cannot be displayed (e.g., system notifications) or if the message is forwarded by a non-premium user. It is recommended to use the emoji from the **emoji** field of the custom emoji sticker.
- Custom emoji entities can only be used by bots that purchased additional usernames on Fragment or in the messages directly sent by the bot to private, group and supergroup chats if the owner of the bot has a Telegram Premium subscription.
- See date-time entity formatting for more details about supported date-time formats.

Markdown style

This is a legacy mode, retained for backward compatibility. To use this mode, pass *Markdown* in the *parse_mode* field. Use the following syntax in your message:

```
*bold text*
_italic text_
[inline URL](http://www.example.com/)
[inline mention of a user](tg://user?id=123456789)
`inline fixed-width code`
```
pre-formatted fixed-width code block
```
```python
pre-formatted fixed-width code block written in the Python programming language
```
```

Please note:

- Entities must not be nested, use parse mode MarkdownV2 instead.
- There is no way to specify “underline”, “strikethrough”, “spoiler”, “blockquote”, “expandable_blockquote”, “custom_emoji”, and “date_time” entities, use parse mode MarkdownV2 instead.
- To escape characters '_', '*', '`', '[' outside of an entity, prepend the character '\' before them.
- Escaping inside entities is not allowed, so entity must be closed first and reopened again: use `_snake_\__case_` for italic `snake_case` and `*2*\**2=4*` for bold `2*2=4`.

Ephemeral Messages and Commands

Ephemeral interactions allow a bot and an individual member of a group or supergroup chat to communicate privately on the public timeline without cluttering the chat for other members. They may disappear automatically after some time, or if the app is restarted.

Ephemeral Commands (User to Bot)
Bots can declare ephemeral commands by setting the *is_ephemeral* field to *True* in the BotCommand class. A user can then send an ephemeral command that is received by the target bot but remains invisible to all members of the chat, including both users and other bots.

Ephemeral Messages
Bots can send an ephemeral message response back to a specific user designated by the *receiver_user_id* parameter. Other members of the group or supergroup chat will not see the message.

> It is **not guaranteed** that the ephemeral message will be received, especially if the user is offline.

Reply Targets and Conditions

- Any bot can send an ephemeral message to a user within **15 seconds** of the incoming eligible action. The message will be sent to the exact client application that triggered the action. For this the bot must provide either:
    - The *callback_query_id* from a received callback query, or

    - The *reply_parameters.ephemeral_message_id* from an incoming ephemeral message.
- If the bot is a chat administrator, it can send an ephemeral message to any non-bot member of the chat at any time without needing to specify a *callback_query_id* or *reply_parameters.ephemeral_message_id*. In this case, the message may be delivered across multiple active client applications of the user, but is regardless not guaranteed to be delivered to any of them.

Paid Broadcasts

By default, all bots are able to broadcast up to 30 messages per second to their users. Developers can increase this limit by enabling *Paid Broadcasts* in @BotFather - allowing their bot to broadcast **up to 1000 messages** per second.

Each message broadcasted over the free amount of 30 messages per second incurs a cost of 0.1 Stars per message, paid with Telegram Stars from the bot's balance. In order to use this feature, a bot must have at least *10,000 Stars* on its balance.

> Bots with increased limits are only charged for messages that are broadcasted successfully.

forwardMessage

Use this method to forward messages of any kind. Service messages and messages with protected content can't be forwarded. On success, the sent Message is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be forwarded; required if the message is forwarded to a direct messages chat
from_chat_id	Integer or String	Yes	Unique identifier for the chat where the original message was sent (or username of the target bot, supergroup, or channel in the format `@username`)
video_start_timestamp	Integer	Optional	New start timestamp for the forwarded video in the message
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.

Parameter	Type	Required	Description
protect_content	Boolean	Optional	Protects the contents of the forwarded message from forwarding and saving
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; only available when forwarding to private chats
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only
message_id	Integer	Yes	Message identifier in the chat specified in *from_chat_id*

forwardMessages

Use this method to forward multiple messages of any kind. If some of the specified messages can't be found or forwarded, they are skipped. Service messages and messages with protected content can't be forwarded. Album grouping is kept for forwarded messages. On success, an Array of MessageId of the sent messages is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the messages will be forwarded; required if the messages are forwarded to a direct messages chat
from_chat_id	Integer or String	Yes	Unique identifier for the chat where the original messages were sent (or username of the target bot, supergroup, or channel in the format `@username`)
message_ids	Array of Integer	Yes	A JSON-serialized list of 1-100 identifiers of messages in the chat *from_chat_id* to forward. The identifiers must be specified in a strictly increasing order.
disable_notification	Boolean	Optional	Sends the messages silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the forwarded messages from forwarding and saving

copyMessage

Use this method to copy messages of any kind. Service messages, paid media messages, giveaway messages, giveaway winners messages, invoice messages, and widget messages can't be copied. A quiz poll can be copied only if the value of the field *correct_option_ids* is known to the bot. The method is analogous to the method forwardMessage, but the copied message doesn't have a link to the original message. Returns the MessageId of the sent message on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
from_chat_id	Integer or String	Yes	Unique identifier for the chat where the original message was sent (or username of the target bot, supergroup, or channel in the format `@username`)
message_id	Integer	Yes	Message identifier in the chat specified in *from_chat_id*
video_start_timestamp	Integer	Optional	New start timestamp for the copied video in the message
caption	String	Optional	New caption for media, 0-1024 characters after entities parsing. If not specified, the original caption is kept.
parse_mode	String	Optional	Mode for parsing entities in the new caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the new caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media. Ignored if a new caption isn't specified.

Parameter	Type	Required	Description
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; only available when copying to private chats
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

copyMessages

Use this method to copy messages of any kind. If some of the specified messages can't be found or copied, they are skipped. Service messages, paid media messages, giveaway messages, giveaway winners messages, invoice messages, and widget messages can't be copied. A quiz poll can be copied only if the value of the field *correct_option_ids* is known to the bot. The method is analogous to the method forwardMessages, but the copied messages don't have a link to the original message. Album grouping is kept for copied messages. On success, an Array of MessageId of the sent messages is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the messages will be sent; required if the messages are sent to a direct messages chat
from_chat_id	Integer or String	Yes	Unique identifier for the chat where the original messages were sent (or username of the target bot, supergroup, or channel in the format `@username`)
message_ids	Array of Integer	Yes	A JSON-serialized list of 1-100 identifiers of messages in the chat *from_chat_id* to copy. The identifiers must be specified in a strictly increasing order.
disable_notification	Boolean	Optional	Sends the messages silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent messages from forwarding and saving
remove_caption	Boolean	Optional	Pass *True* to copy the messages without their captions

sendPhoto

Use this method to send photos. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only

Parameter	Type	Required	Description
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
photo	InputFile or String	Yes	Photo to send. Pass a file_id as String to send a photo that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get a photo from the Internet, or upload a new photo using multipart/form-data. The photo must be at most 10 MB in size. The photo's width and height must not exceed 10000 in total. Width and height ratio must be at most 20. More information on Sending Files »
caption	String	Optional	Photo caption (may also be used when resending photos by *file_id*), 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the photo caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	Optional	Pass *True* if the photo needs to be covered with a spoiler animation

Parameter	Type	Required	Description
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendLivePhoto

Use this method to send live photos. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent

Parameter	Type	Required	Description
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
live_photo	InputFile or String	Yes	Live photo video to send. The video must be no longer than 10 seconds and must not exceed 10 MB in size. Pass a file_id as String to send a video that exists on the Telegram servers (recommended) or upload a new video using multipart/form-data. More information on Sending Files ». Sending live photos by a URL is currently unsupported.
photo	InputFile or String	Yes	The static photo to send. Pass a file_id as String to send a photo that exists on the Telegram servers (recommended) or upload a new video using multipart/form-data. More information on Sending Files ». Sending live photos by a URL is currently unsupported.

Parameter	Type	Required	Description
caption	String	Optional	Video caption (may also be used when resending videos by *file_id*), 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the video caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	Optional	Pass *True* if the video needs to be covered with a spoiler animation
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only

Parameter	Type	Required	Description
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendAudio

Use this method to send audio files, if you want Telegram clients to display them in the music player. Your audio must be in the .MP3 or .M4A format. On success, the sent Message is returned. Bots can currently send audio files of up to 50 MB in size, this limit may be changed in the future.

For sending voice messages, use the sendVoice method instead.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only

Parameter	Type	Required	Description
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
audio	InputFile or String	Yes	Audio file to send. Pass a file_id as String to send an audio file that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get an audio file from the Internet, or upload a new one using multipart/form-data. More information on Sending Files »
caption	String	Optional	Audio caption, 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the audio caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
duration	Integer	Optional	Duration of the audio in seconds
performer	String	Optional	Performer
title	String	Optional	Track name

Parameter	Type	Required	Description
thumbnail	InputFile or String	Optional	Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to

Parameter	Type	Required	Description
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendDocument

Use this method to send general files. On success, the sent Message is returned. Bots can currently send files of any type of up to 50 MB in size, this limit may be changed in the future.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
document	InputFile or String	Yes	File to send. Pass a file_id as String to send a file that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get a file from the Internet, or upload a new one using multipart/form-data. More information on Sending Files »

Parameter	Type	Required	Description
thumbnail	InputFile or String	Optional	Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
caption	String	Optional	Document caption (may also be used when resending documents by *file_id*), 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the document caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
disable_content_type_detection	Boolean	Optional	Disables automatic server-side content type detection for files uploaded using multipart/form-data
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving

Parameter	Type	Required	Description
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendVideo

Use this method to send video files, Telegram clients support MPEG4 videos (other formats may be sent as Document). On success, the sent Message is returned. Bots can currently send video files of up to 50 MB in size, this limit may be changed in the future.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query

Parameter	Type	Required	Description
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
video	InputFile or String	Yes	Video to send. Pass a file_id as String to send a video that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get a video from the Internet, or upload a new video using multipart/form-data. More information on Sending Files »
duration	Integer	Optional	Duration of sent video in seconds
width	Integer	Optional	Video width
height	Integer	Optional	Video height
thumbnail	InputFile or String	Optional	Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »

Parameter	Type	Required	Description
cover	InputFile or String	Optional	Cover for the video in the message. Pass a file_id to send a file that exists on the Telegram servers (recommended), pass an HTTP URL for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new one using multipart/form-data under <file_attach_name> name. More information on Sending Files »
start_timestamp	Integer	Optional	Start timestamp for the video in the message
caption	String	Optional	Video caption (may also be used when resending videos by *file_id*), 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the video caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	Optional	Pass *True* if the video needs to be covered with a spoiler animation
supports_streaming	Boolean	Optional	Pass *True* if the uploaded video is suitable for streaming
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving

Parameter	Type	Required	Description
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendAnimation

Use this method to send animation files (GIF or H.264/MPEG-4 AVC video without sound). On success, the sent Message is returned. Bots can currently send animation files of up to 50 MB in size, this limit may be changed in the future.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query

Parameter	Type	Required	Description
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
animation	InputFile or String	Yes	Animation to send. Pass a file_id as String to send an animation that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get an animation from the Internet, or upload a new animation using multipart/form-data. More information on Sending Files »
duration	Integer	Optional	Duration of sent animation in seconds
width	Integer	Optional	Animation width
height	Integer	Optional	Animation height

Parameter	Type	Required	Description
thumbnail	InputFile or String	Optional	Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
caption	String	Optional	Animation caption (may also be used when resending animation by *file_id*), 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the animation caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media
has_spoiler	Boolean	Optional	Pass *True* if the animation needs to be covered with a spoiler animation
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving

Parameter	Type	Required	Description
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendVoice

Use this method to send audio files, if you want Telegram clients to display the file as a playable voice message. For this to work, your audio must be in an .OGG file encoded with OPUS, or in .MP3 format, or in .M4A format (other formats may be sent as Audio or Document). On success, the sent Message is returned. Bots can currently send voice messages of up to 50 MB in size, this limit may be changed in the future.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent

Parameter	Type	Required	Description
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
voice	InputFile or String	Yes	Audio file to send. Pass a file_id as String to send a file that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get a file from the Internet, or upload a new one using multipart/form-data. More information on Sending Files »
caption	String	Optional	Voice message caption, 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the voice message caption. See formatting options for more details.

Parameter	Type	Required	Description
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
duration	Integer	Optional	Duration of the voice message in seconds
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendVideoNote

Use this method to send a rounded square MPEG4 video of up to 1 minute long. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
video_note	InputFile or String	Yes	Video note to send. Pass a file_id as String to send a video note that exists on the Telegram servers (recommended) or upload a new video using multipart/form-data. More information on Sending Files ». Sending video notes by a URL is currently unsupported.
duration	Integer	Optional	Duration of sent video in seconds
length	Integer	Optional	Video width and height, i.e. diameter of the video message

Parameter	Type	Required	Description
thumbnail	InputFile or String	Optional	Thumbnail of the file sent; can be ignored if thumbnail generation for the file is supported server-side. The thumbnail should be in JPEG format and less than 200 kB in size. A thumbnail's width and height should not exceed 320. Ignored if the file is not uploaded using multipart/form-data. Thumbnails can't be reused and can be only uploaded as a new file, so you can pass “attach://<file_attach_name>” if the thumbnail was uploaded using multipart/form-data under <file_attach_name>. More information on Sending Files »
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to

Parameter	Type	Required	Description
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendWidget

Use this method to send a message with an HTML widget. Currently, in the production environment, widgets can be sent only to private chats, and sent widgets automatically have protected content and therefore can't be forwarded. On success, the sent Message is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
html	String	Yes	HTML code of the widget, 1-32768 bytes. The code must not exceed 4096 bytes when compressed by zlib with the maximum compression level.
width	Integer	Optional	Width of the widget in logical pixels; omit if the widget must occupy the whole width of the message bubble. The width is clamped by the app to the range from 200 to the maximum width of the message bubble.
height	Integer	Optional	Height of the widget in logical pixels; omit if the height of the widget isn't fixed. The height is clamped by the app to the range 50-600.
public_data	String	Optional	Public data of the widget, 0-1024 bytes, which will be available wherever the widget is shown. If the widget is shared, then all messages with it will have the same public data.
chat_data	String	Optional	Data of the widget, which is specific to the message, 0-1024 bytes
caption	String	Optional	Widget caption, 0-1024 characters after entities parsing

Parameter	Type	Required	Description
parse_mode	String	Optional	Mode for parsing entities in the widget caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the widget
allow_queries	Boolean	Optional	Pass *True* to allow the widget to send queries to the bot
is_shared	Boolean	Optional	Pass *True* to create a shared widget, which can be sent again using the method sendSharedWidget
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendSharedWidget

Use this method to send again a shared widget created by the bot. Currently, in the production environment, widgets can be sent only to private chats, and sent widgets automatically have protected content and therefore can't be forwarded. On success, the sent Message is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
widget_id	String	Yes	Unique identifier of the shared widget
chat_data	String	Optional	Data of the widget, which is specific to the message, 0-1024 bytes
caption	String	Optional	Widget caption, 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the widget caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the widget
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
reply_parameters	ReplyParameters	Optional	Description of the message to reply to

Parameter	Type	Required	Description
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendPaidMedia

Use this method to send paid media. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`. If the chat is a channel, all Telegram Star proceeds from this media will be credited to the chat's balance. Otherwise, they will be credited to the bot's balance.
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
star_count	Integer	Yes	The number of Telegram Stars that must be paid to buy access to the media; 1-25000
media	Array of InputPaidMedia	Yes	A JSON-serialized Array describing the media to be sent; up to 10 items
payload	String	Optional	Bot-defined paid media payload, 0-128 bytes. This will not be displayed to the user, use it for your internal processes.
caption	String	Optional	Media caption, 0-1024 characters after entities parsing

Parameter	Type	Required	Description
parse_mode	String	Optional	Mode for parsing entities in the media caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendMediaGroup

Use this method to send a group of photos, live photos, videos, documents or audios as an album. Documents and audio files can be only grouped in an album with messages of the same type. On success, an Array of Message objects that were sent is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the messages will be sent; required if the messages are sent to a direct messages chat
media	Array of InputMediaAudio, InputMediaDocument, InputMediaLivePhoto, InputMediaPhoto and InputMediaVideo	Yes	A JSON-serialized Array describing messages to be sent, must include 2-10 items
disable_notification	Boolean	Optional	Sends messages silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent messages from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
reply_parameters	ReplyParameters	Optional	Description of the message to reply to

sendLocation

Use this method to send point on the map. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
latitude	Float	Yes	Latitude of the location
longitude	Float	Yes	Longitude of the location
horizontal_accuracy	Float	Optional	The radius of uncertainty for the location, measured in meters; 0-1500
live_period	Integer	Optional	Period in seconds during which the location will be updated (see Live Locations), must be between 60 and 86400, or 0x7FFFFFFF for live locations that can be edited indefinitely. Must be 0 for ephemeral messages.

Parameter	Type	Required	Description
heading	Integer	Optional	For live locations, a direction in which the user is moving, in degrees. Must be between 1 and 360 if specified.
proximity_alert_radius	Integer	Optional	For live locations, a maximum distance for proximity alerts about approaching another chat member, in meters. Must be between 1 and 100000 if specified.
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to

Parameter	Type	Required	Description
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendVenue

Use this method to send information about a venue. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
latitude	Float	Yes	Latitude of the venue
longitude	Float	Yes	Longitude of the venue
title	String	Yes	Name of the venue
address	String	Yes	Address of the venue

Parameter	Type	Required	Description
foursquare_id	String	Optional	Foursquare identifier of the venue
foursquare_type	String	Optional	Foursquare type of the venue, if known. (For example, “arts_entertainment/default”, “arts_entertainment/aquarium” or “food/icecream”.)
google_place_id	String	Optional	Google Places identifier of the venue
google_place_type	String	Optional	Google Places type of the venue. (See supported types.)
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendContact

Use this method to send phone contacts. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
phone_number	String	Yes	Contact's phone number
first_name	String	Yes	Contact's first name
last_name	String	Optional	Contact's last name
vcard	String	Optional	Additional data about the contact in the form of a vCard, 0-2048 bytes
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving

Parameter	Type	Required	Description
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendPoll

Use this method to send a native poll. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`. Polls can't be sent to channel direct messages chats.

Parameter	Type	Required	Description
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
question	String	Yes	Poll question, 1-300 characters
question_parse_mode	String	Optional	Mode for parsing entities in the question. See formatting options for more details. Currently, only custom emoji entities are allowed.
question_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the poll question. It can be specified instead of *question_parse_mode*.
options	Array of InputPollOption	Yes	A JSON-serialized list of 1-12 answer options
is_anonymous	Boolean	Optional	*True*, if the poll needs to be anonymous, defaults to *True*
type	String	Optional	Poll type, “quiz” or “regular”, defaults to “regular”
allows_multiple_answers	Boolean	Optional	Pass *True* if the poll allows multiple answers, defaults to *False*
allows_revoting	Boolean	Optional	Pass *True* if the poll allows to change chosen answer options, defaults to *False* for quizzes and to *True* for regular polls
shuffle_options	Boolean	Optional	Pass *True* if the poll options must be shown in random order
allow_adding_options	Boolean	Optional	Pass *True* if answer options can be added to the poll after creation; not supported for anonymous polls and quizzes
hide_results_until_closes	Boolean	Optional	Pass *True* if poll results must be shown only after the poll closes
members_only	Boolean	Optional	Pass *True* if voting is limited to users who have been members of the chat where the poll is being sent for more than 24 hours; for channel chats only
country_codes	Array of String	Optional	A JSON-serialized list of 0-12 two-letter ISO 3166-1 alpha-2 country codes indicating the countries from which users can vote in the poll; for channel chats only. Use “FT” as a country code to allow users with anonymous numbers to vote. If omitted or empty, then users from any country can participate in the poll.

Parameter	Type	Required	Description
correct_option_ids	Array of Integer	Optional	A JSON-serialized list of monotonically increasing 0-based identifiers of the correct answer options, required for polls in quiz mode
explanation	String	Optional	Text that is shown when a user chooses an incorrect answer or taps on the lamp icon in a quiz-style poll, 0-200 characters with at most 2 line feeds after entities parsing
explanation_parse_mode	String	Optional	Mode for parsing entities in the explanation. See formatting options for more details.
explanation_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the poll explanation. It can be specified instead of *explanation_parse_mode*.
explanation_media	InputPollMedia	Optional	Media added to the quiz explanation
open_period	Integer	Optional	Amount of time in seconds the poll will be active after creation, 5-2628000. Can't be used together with *close_date*.
close_date	Integer	Optional	Point in time (Unix timestamp) when the poll will be automatically closed. Must be at least 5 and no more than 2628000 seconds in the future. Can't be used together with *open_period*.
is_closed	Boolean	Optional	Pass *True* if the poll needs to be immediately closed. This can be useful for poll preview.
description	String	Optional	Description of the poll to be sent, 0-1024 characters after entities parsing
description_parse_mode	String	Optional	Mode for parsing entities in the poll description. See formatting options for more details.
description_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the poll description, which can be specified instead of *description_parse_mode*
media	InputPollMedia	Optional	Media added to the poll description
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving

Parameter	Type	Required	Description
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendChecklist

Use this method to send a checklist on behalf of a connected business account. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot in the format `@username`
checklist	InputChecklist	Yes	A JSON-serialized object for the checklist to send
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message
reply_parameters	ReplyParameters	Optional	A JSON-serialized object for description of the message to reply to
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

sendDice

Use this method to send an animated emoji that will display a random value. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
emoji	String	Optional	Emoji on which the dice throw animation is based. Currently, must be one of “🎲”, “🎯”, “🏀”, “⚽”, “🎳”, or “🎰”. Dice can have values 1-6 for “🎲”, “🎯” and “🎳”, values 1-5 for “🏀” and “⚽”, and values 1-64 for “🎰”. Defaults to “🎲”.
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only

Parameter	Type	Required	Description
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendMessageDraft

Use this method to stream a partial message to a user while the message is being generated. Note that the streamed draft is ephemeral and acts as a temporary 30-second preview - once the output is finalized, you **must** call sendMessage with the complete message to persist it in the user's chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer	Yes	Unique identifier for the target private chat
message_thread_id	Integer	Optional	Unique identifier for the target message thread
draft_id	Integer	Yes	Unique identifier of the message draft; must be non-zero. Changes to drafts with the same identifier are animated. Otherwise, the draft is replaced without animation.
text	String	Optional	Text of the message to be sent, 0-4096 characters after entities parsing. Pass an empty text to show a “Thinking…” placeholder.
parse_mode	String	Optional	Mode for parsing entities in the message text. See formatting options for more details.
entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in message text, which can be specified instead of *parse_mode*
can_stop	Boolean	Optional	Pass *True* to show the user a button to stop further drafts. The bot will receive an Update “stopped_message_generation” if the user presses the button.

Parameter	Type	Required	Description
keep_on_stop	Boolean	Optional	Pass *True* to keep the draft in the chat when the button is pressed. The draft will still disappear after a short time or if the bot sends a message. To fully preserve the partial draft, the bot should send it as a new message.

sendChatAction

Use this method when you need to tell the user that something is happening on the bot's side. The status is set for 5 seconds or less (when a message arrives from your bot, Telegram clients clear its typing status). Returns *True* on success.

> Example: The ImageBot needs some time to process a request and upload the image. Instead of sending a text message along the lines of “Retrieving image, please wait…”, the bot may use sendChatAction with *action = upload_photo*. The user will see a “sending photo” status for the bot.

We only recommend using this method when a response from the bot will take a **noticeable** amount of time to arrive.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the action will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot or supergroup in the format `@username`. Channel chats and channel direct messages chats aren't supported.
message_thread_id	Integer	Optional	Unique identifier for the target message thread or topic of a forum; for supergroups and private chats of bots with forum topic mode enabled only
action	String	Yes	Type of action to broadcast. Choose one, depending on what the user is about to receive: *typing* for text messages, *upload_photo* for photos, *record_video* or *upload_video* for videos, *record_voice* or *upload_voice* for voice notes, *upload_document* for general files, *choose_sticker* for stickers, *find_location* for location data, *record_video_note* or *upload_video_note* for video notes.

setMessageReaction

Use this method to change the chosen reactions on a message. Service messages of some types can't be reacted to. Automatically forwarded messages from a channel to its discussion group have the same available reactions as messages in the channel. Bots can't use paid reactions. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_id	Integer	Yes	Identifier of the target message. If the message belongs to a media group, the reaction is set to the first non-deleted message in the group instead.
reaction	Array of ReactionType	Optional	A JSON-serialized list of reaction types to set on the message. Currently, as non-premium users, bots can set up to one reaction per message. A custom emoji reaction can be used if it is either already present on the message or explicitly allowed by chat administrators. Paid reactions can't be used by bots.
is_big	Boolean	Optional	Pass *True* to set the reaction with a big animation

getUserProfilePhotos

Use this method to get a list of profile pictures for a user. Returns a UserProfilePhotos object.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user
offset	Integer	Optional	Sequential number of the first photo to be returned. By default, all photos are returned.
limit	Integer	Optional	Limits the number of photos to be retrieved. Values between 1-100 are accepted. Defaults to 100.

getUserProfileAudios

Use this method to get a list of profile audios for a user. Returns a UserProfileAudios object.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user
offset	Integer	Optional	Sequential number of the first audio to be returned. By default, all audios are returned.
limit	Integer	Optional	Limits the number of audios to be retrieved. Values between 1-100 are accepted. Defaults to 100.

setUserEmojiStatus

Changes the emoji status for a given user that previously allowed the bot to manage their emoji status via the Mini App method requestEmojiStatusAccess. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user

Parameter	Type	Required	Description
emoji_status_custom_emoji_id	String	Optional	Custom emoji identifier of the emoji status to set. Pass an empty string to remove the status.
emoji_status_expiration_date	Integer	Optional	Expiration date of the emoji status, if any

getFile

Use this method to get basic information about a file and prepare it for downloading. For the moment, bots can download files of up to 20MB in size. On success, a File object is returned. The file can then be downloaded via the link `https://api.telegram.org/file/bot<token>/<file_path>`, where `<file_path>` is taken from the response. It is guaranteed that the link will be valid for at least 1 hour. When the link expires, a new one can be requested by calling getFile again.

Parameter	Type	Required	Description
file_id	String	Yes	File identifier to get information about

Note: This function may not preserve the original file name and MIME type. You should save the file's MIME type and name (if available) when the File object is received.

banChatMember

Use this method to ban a user in a group, a supergroup or a channel. In the case of supergroups and channels, the user will not be able to return to the chat on their own using invite links, etc., unless unbanned first. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target group or username of the target supergroup or channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
until_date	Integer	Optional	Date when the user will be unbanned; Unix time. If user is banned for more than 366 days or less than 30 seconds from the current time they are considered to be banned forever. Applied for supergroups and channels only.
revoke_messages	Boolean	Optional	Pass *True* to delete all messages from the chat for the user that is being removed. If *False*, the user will be able to see messages in the group that were sent before the user was removed. Always *True* for supergroups and channels.

unbanChatMember

Use this method to unban a previously banned user in a supergroup or channel. The user will **not** return to the group or channel automatically, but will be able to join via link, etc. The bot must be an administrator for this to work. By default, this method guarantees that after the call the user is not a member of the chat, but will be able to join it. So if the user is a member of the chat they will also be

removed from the chat. If you don't want this, use the parameter *only_if_banned*. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target group or username of the target supergroup or channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
only_if_banned	Boolean	Optional	Do nothing if the user is not banned

restrictChatMember

Use this method to restrict a user in a supergroup. The bot must be an administrator in the supergroup for this to work and must have the appropriate administrator rights. Pass *True* for all permissions to lift restrictions from a user. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
permissions	ChatPermissions	Yes	A JSON-serialized object for new user permissions
use_independent_chat_permissions	Boolean	Optional	Pass *True* if chat permissions are set independently. Otherwise, the *can_send_other_messages* and *can_add_web_page_previews* permissions will imply the *can_send_messages*, *can_send_audios*, *can_send_documents*, *can_send_photos*, *can_send_videos*, *can_send_video_notes*, and *can_send_voice_notes* permissions; the *can_send_polls* permission will imply the *can_send_messages* permission.
until_date	Integer	Optional	Date when restrictions will be lifted for the user; Unix time. If user is restricted for more than 366 days or less than 30 seconds from the current time, they are considered to be restricted forever.

promoteChatMember

Use this method to promote or demote a user in a supergroup or a channel. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Pass *False* for all boolean parameters to demote a user. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
is_anonymous	Boolean	Optional	Pass *True* if the administrator's presence in the chat is hidden
can_manage_chat	Boolean	Optional	Pass *True* if the administrator can access the chat event log, get boost list, see hidden supergroup and channel members, report spam messages, ignore slow mode, and send messages to the chat without paying Telegram Stars. Implied by any other administrator privilege.
can_delete_messages	Boolean	Optional	Pass *True* if the administrator can delete messages of other users
can_manage_video_chats	Boolean	Optional	Pass *True* if the administrator can manage video chats
can_restrict_members	Boolean	Optional	Pass *True* if the administrator can restrict, ban or unban chat members, or access supergroup statistics. For backward compatibility, defaults to *True* for promotions of channel administrators.
can_promote_members	Boolean	Optional	Pass *True* if the administrator can add new administrators with a subset of their own privileges or demote administrators that they have promoted, directly or indirectly (promoted by administrators that were appointed by him)
can_change_info	Boolean	Optional	Pass *True* if the administrator can change chat title, photo and other settings
can_invite_users	Boolean	Optional	Pass *True* if the administrator can invite new users to the chat
can_post_stories	Boolean	Optional	Pass *True* if the administrator can post stories to the chat
can_edit_stories	Boolean	Optional	Pass *True* if the administrator can edit stories posted by other users, post stories to the chat page, pin chat stories, and access the chat's story archive
can_delete_stories	Boolean	Optional	Pass *True* if the administrator can delete stories posted by other users
can_post_messages	Boolean	Optional	Pass *True* if the administrator can post messages in the channel, approve suggested posts, or access channel statistics; for channels only
can_edit_messages	Boolean	Optional	Pass *True* if the administrator can edit messages of other users and can pin messages; for channels only
can_pin_messages	Boolean	Optional	Pass *True* if the administrator can pin messages; for supergroups only

Parameter	Type	Required	Description
can_manage_topics	Boolean	Optional	Pass *True* if the user is allowed to create, rename, close, and reopen forum topics; for supergroups only
can_manage_direct_messages	Boolean	Optional	Pass *True* if the administrator can manage direct messages within the channel and decline suggested posts; for channels only
can_manage_tags	Boolean	Optional	Pass *True* if the administrator can edit the tags of regular members; for groups and supergroups only
can_send_welcome_messages	Boolean	Optional	Pass *True* if the administrator can manage chat welcome messages or directly send them in the case of bots

setChatAdministratorCustomTitle

Use this method to set a custom title for an administrator in a supergroup promoted by the bot. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
custom_title	String	Yes	New custom title for the administrator; 0-16 characters, emoji are not allowed

setChatMemberTag

Use this method to set a tag for a regular member in a group or a supergroup. The bot must be an administrator in the chat for this to work and must have the *can_manage_tags* administrator right. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user
tag	String	Optional	New tag for the member; 0-16 characters, emoji are not allowed

banChatSenderChat

Use this method to ban a channel chat in a supergroup or a channel. Until the chat is unbanned, the owner of the banned chat won't be able to send messages on behalf of **any of their channels**. The bot must be an administrator in the supergroup or channel for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
sender_chat_id	Integer	Yes	Unique identifier of the target sender chat

unbanChatSenderChat

Use this method to unban a previously banned channel chat in a supergroup or channel. The bot must be an administrator for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
sender_chat_id	Integer	Yes	Unique identifier of the target sender chat

setChatPermissions

Use this method to set default chat permissions for all members. The bot must be an administrator in the group or a supergroup for this to work and must have the *can_restrict_members* administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
permissions	ChatPermissions	Yes	A JSON-serialized object for new default chat permissions
use_independent_chat_permissions	Boolean	Optional	Pass *True* if chat permissions are set independently. Otherwise, the *can_send_other_messages* and *can_add_web_page_previews* permissions will imply the *can_send_messages*, *can_send_audios*, *can_send_documents*, *can_send_photos*, *can_send_videos*, *can_send_video_notes*, and *can_send_voice_notes* permissions; the *can_send_polls* permission will imply the *can_send_messages* permission.

exportChatInviteLink

Use this method to generate a new primary invite link for a chat; any previously generated primary link is revoked. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns the new invite link as *String* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`

> Note: Each administrator in a chat generates their own invite links. Bots can't use invite links generated by other administrators. If you want your bot to work with invite links, it will need to generate its own link using exportChatInviteLink or by calling the getChat method. If your bot needs to generate a new primary invite link replacing its previous one, use exportChatInviteLink again.

createChatInviteLink

Use this method to create an additional invite link for a chat. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. The link can be revoked using the method revokeChatInviteLink. Returns the new invite link as ChatInviteLink object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
name	String	Optional	Invite link name; 0-32 characters
expire_date	Integer	Optional	Point in time (Unix timestamp) when the link will expire
member_limit	Integer	Optional	The maximum number of users that can be members of the chat simultaneously after joining the chat via this invite link; 1-99999
creates_join_request	Boolean	Optional	*True*, if users joining the chat via the link need to be approved by chat administrators. If *True*, *member_limit* can't be specified.

editChatInviteLink

Use this method to edit a non-primary invite link created by the bot. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns the edited invite link as a ChatInviteLink object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
invite_link	String	Yes	The invite link to edit
name	String	Optional	Invite link name; 0-32 characters
expire_date	Integer	Optional	Point in time (Unix timestamp) when the link will expire

Parameter	Type	Required	Description
member_limit	Integer	Optional	The maximum number of users that can be members of the chat simultaneously after joining the chat via this invite link; 1-99999
creates_join_request	Boolean	Optional	*True*, if users joining the chat via the link need to be approved by chat administrators. If *True*, *member_limit* can't be specified.

createChatSubscriptionInviteLink

Use this method to create a subscription invite link for a channel chat. The bot must have the *can_invite_users* administrator rights. The link can be edited using the method editChatSubscriptionInviteLink or revoked using the method revokeChatInviteLink. Returns the new invite link as a ChatInviteLink object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target channel chat or username of the target channel in the format `@username`
name	String	Optional	Invite link name; 0-32 characters
subscription_period	Integer	Yes	The number of seconds the subscription will be active for before the next payment. Currently, it must always be 2592000 (30 days).
subscription_price	Integer	Yes	The amount of Telegram Stars a user must pay initially and after each subsequent subscription period to be a member of the chat; 1-10000

editChatSubscriptionInviteLink

Use this method to edit a subscription invite link created by the bot. The bot must have the *can_invite_users* administrator rights. Returns the edited invite link as a ChatInviteLink object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
invite_link	String	Yes	The invite link to edit
name	String	Optional	Invite link name; 0-32 characters

revokeChatInviteLink

Use this method to revoke an invite link created by the bot. If the primary link is revoked, a new link is automatically generated. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns the revoked invite link as ChatInviteLink object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier of the target chat or username of the target channel in the format `@username`
invite_link	String	Yes	The invite link to revoke

approveChatJoinRequest

Use this method to approve a chat join request. The bot must be an administrator in the chat for this to work and must have the *can_invite_users* administrator right. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user

declineChatJoinRequest

Use this method to decline a chat join request. The bot must be an administrator in the chat for this to work and must have the *can_invite_users* administrator right. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user

answerChatJoinRequestQuery

Use this method to process a received chat join request query. Returns *True* on success.

Parameter	Type	Required	Description
chat_join_request_query_id	String	Yes	Unique identifier of the join request query
result	String	Yes	Result of the query. Must be either “approve” to allow the user to join the chat, “decline” to disallow the user to join the chat, or “queue” to leave the decision to other administrators.

sendChatJoinRequestWebApp

Use this method to process a received chat join request query by showing a Mini App to the user before deciding the outcome. Call answerChatJoinRequestQuery to resolve the join request query based on the user interaction with the Mini App. Returns *True* on success.

Parameter	Type	Required	Description
chat_join_request_query_id	String	Yes	Unique identifier of the join request query

Parameter	Type	Required	Description
web_app_url	String	Yes	An HTTPS URL of a Web App to be opened with additional data as specified in Initializing Web Apps

setChatPhoto

Use this method to set a new profile photo for the chat. Photos can't be changed for private chats. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
photo	InputFile	Yes	New chat photo, uploaded using multipart/form-data

deleteChatPhoto

Use this method to delete a chat photo. Photos can't be changed for private chats. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`

setChatTitle

Use this method to change the title of a chat. Titles can't be changed for private chats. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
title	String	Yes	New chat title, 1-128 characters

setChatDescription

Use this method to change the description of a group, a supergroup or a channel. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
description	String	Optional	New chat description, 0-255 characters

pinChatMessage

Use this method to add a message to the list of pinned messages in a chat. In private chats and channel direct messages chats, all non-service messages can be pinned. Conversely, the bot must be an administrator with the 'can_pin_messages' right or the 'can_edit_messages' right to pin messages in groups and channels respectively. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be pinned
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
message_id	Integer	Yes	Identifier of a message to pin
disable_notification	Boolean	Optional	Pass *True* if it is not necessary to send a notification to all chat members about the new pinned message. Notifications are always disabled in channels and private chats.

unpinChatMessage

Use this method to remove a message from the list of pinned messages in a chat. In private chats and channel direct messages chats, all messages can be unpinned. Conversely, the bot must be an administrator with the 'can_pin_messages' right or the 'can_edit_messages' right to unpin messages in groups and channels respectively. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be unpinned
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
message_id	Integer	Optional	Identifier of the message to unpin. Required if *business_connection_id* is specified. If not specified, the most recent pinned message (by sending date) will be unpinned.

unpinAllChatMessages

Use this method to clear the list of pinned messages in a chat. In private chats and channel direct messages chats, no additional rights are required to unpin all pinned messages. Conversely, the bot must be an administrator with the 'can_pin_messages' right or the 'can_edit_messages' right to unpin all pinned messages in groups and channels respectively. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`

leaveChat

Use this method for your bot to leave a group, supergroup or channel. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup or channel in the format `@username`. Channel direct messages chats aren't supported; leave the corresponding channel instead.

getChat

Use this method to get up-to-date information about the chat. Returns a ChatFullInfo object on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup or channel in the format `@username`

getChatAdministrators

Use this method to get a list of administrators in a chat. Returns an Array of ChatMember objects.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup or channel in the format `@username`
return_bots	Boolean	Optional	Pass *True* to additionally receive all bots that are administrators of the chat. By default, bots other than the current bot are omitted.

getChatMemberCount

Use this method to get the number of members in a chat. Returns *Integer* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup or channel in the format `@username`

getChatMember

Use this method to get information about a member of a chat. The method is only guaranteed to work for other users if the bot is an administrator in the chat. Returns a ChatMember object on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup or channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user

getChatShareThemeBackdrops

Use this method to get the list of backdrops, which can be used as the background of a theme of the screen for sharing a link to a user or a chat. Requires no parameters. Returns a ChatShareThemeBackdrops object.

getUserPersonalChatMessages

Use this method to get the last messages from the personal chat (i.e., the chat currently added to their profile) of a given user. On success, an Array of Message objects is returned.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier for the target user
limit	Integer	Yes	The maximum number of messages to return; 1-20

setChatStickerSet

Use this method to set a new group sticker set for a supergroup. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Use the field *can_set_sticker_set* optionally returned in getChat requests to check if the bot can use this method. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
sticker_set_name	String	Yes	Name of the sticker set to be set as the group sticker set

deleteChatStickerSet

Use this method to delete a group sticker set from a supergroup. The bot must be an administrator in the chat for this to work and must have the appropriate administrator rights. Use the field *can_set_sticker_set* optionally returned in getChat requests to check if the bot can use this method. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

getForumTopicIconStickers

Use this method to get custom emoji stickers, which can be used as a forum topic icon by any user. Requires no parameters. Returns an Array of Sticker objects.

createForumTopic

Use this method to create a topic in a forum supergroup chat or a private chat with a user. In the case of a supergroup chat the bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator right. In the case of a private chat, a link to the topic can be created in the format `https://t.me/<bot_username>/<message_thread_id>`; the link opens the topic only for the user. Returns information about the created topic as a ForumTopic object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
name	String	Yes	Topic name, 1-128 characters
icon_color	Integer	Optional	Color of the topic icon in RGB format. Currently, must be one of 7322096 (0x6FB9F0), 16766590 (0xFFD67E), 13338331 (0xCB86DB), 9367192 (0x8EEE98), 16749490 (0xFF93B2), or 16478047 (0xFB6F5F).
icon_custom_emoji_id	String	Optional	Unique identifier of the custom emoji shown as the topic icon. Use getForumTopicIconStickers to get all allowed custom emoji identifiers.

editForumTopic

Use this method to edit name and icon of a topic in a forum supergroup chat or a private chat with a user. In the case of a supergroup chat the bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights, unless it is the creator of the topic. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Yes	Unique identifier for the target message thread of the forum topic
name	String	Optional	New topic name, 0-128 characters. If not specified or empty, the current name of the topic will be kept.
icon_custom_emoji_id	String	Optional	New unique identifier of the custom emoji shown as the topic icon. Use getForumTopicIconStickers to get all allowed custom emoji identifiers. Pass an empty string to remove the icon. If not specified, the current icon will be kept.

closeForumTopic

Use this method to close an open topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights, unless it is the creator of the topic. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Yes	Unique identifier for the target message thread of the forum topic

reopenForumTopic

Use this method to reopen a closed topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights, unless it is the creator of the topic. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Yes	Unique identifier for the target message thread of the forum topic

deleteForumTopic

Use this method to delete a forum topic along with all its messages in a forum supergroup chat or a private chat with a user. In the case of a supergroup chat the bot must be an administrator in the chat for this to work and must have the *can_delete_messages* administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Yes	Unique identifier for the target message thread of the forum topic

unpinAllForumTopicMessages

Use this method to clear the list of pinned messages in a forum topic in a forum supergroup chat or a private chat with a user. In the case of a supergroup chat the bot must be an administrator in the chat for this to work and must have the *can_pin_messages* administrator right in the supergroup. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_thread_id	Integer	Yes	Unique identifier for the target message thread of the forum topic

editGeneralForumTopic

Use this method to edit the name of the 'General' topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
name	String	Yes	New topic name, 1-128 characters

closeGeneralForumTopic

Use this method to close an open 'General' topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

reopenGeneralForumTopic

Use this method to reopen a closed 'General' topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights. The topic will be automatically unhidden if it was hidden. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

hideGeneralForumTopic

Use this method to hide the 'General' topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights. The topic will be automatically closed if it was open. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

unhideGeneralForumTopic

Use this method to unhide the 'General' topic in a forum supergroup chat. The bot must be an administrator in the chat for this to work and must have the *can_manage_topics* administrator rights. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

unpinAllGeneralForumTopicMessages

Use this method to clear the list of pinned messages in a General forum topic. The bot must be an administrator in the chat for this to work and must have the *can_pin_messages* administrator right in the supergroup. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`

answerCallbackQuery

Use this method to send answers to callback queries sent from inline keyboards. The answer will be displayed to the user as a notification at the top of the chat screen or as an alert. On success, *True* is returned.

> Alternatively, the user can be redirected to the specified Game URL. For this option to work, you must first create a game for your bot via @BotFather and accept the terms. Otherwise, you may use links like `t.me/your_bot?start=XXXX` that open your bot with a parameter.

Parameter	Type	Required	Description
callback_query_id	String	Yes	Unique identifier for the query to be answered
text	String	Optional	Text of the notification. If not specified, nothing will be shown to the user, 0-200 characters.
show_alert	Boolean	Optional	If *True*, an alert will be shown by the client instead of a notification at the top of the chat screen. Defaults to *False*.

Parameter	Type	Required	Description
url	String	Optional	URL that will be opened by the user's client. If you have created a Game and accepted the conditions via @BotFather, specify the URL that opens your game - note that this will only work if the query comes from a *callback_game* button. Otherwise, you may use links like `t.me/your_bot?start=XXXX` that open your bot with a parameter.
cache_time	Integer	Optional	The maximum amount of time in seconds that the result of the callback query may be cached client-side. Defaults to 0.

answerWidgetQuery

Use this method to send an answer to a query from a widget. The query must be answered within 15 seconds. The answer is received only by the widget that sent the query and doesn't change data of the widget; use editMessageWidgetData or editSharedWidgetData to change it. On success, *True* is returned.

Parameter	Type	Required	Description
widget_query_id	String	Yes	Unique identifier for the query to be answered
data	String	Optional	Data to be returned to the widget, 0-1024 bytes

answerGuestQuery

Use this method to reply to a received guest message. On success, a SentGuestMessage object is returned.

Parameter	Type	Required	Description
guest_query_id	String	Yes	Unique identifier for the query to be answered
result	InlineQueryResult	Yes	A JSON-serialized object describing the message to be sent

getUserChatBoosts

Use this method to get the list of boosts added to a chat by a user. Requires administrator rights in the chat. Returns a UserChatBoosts object.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the chat or username of the channel in the format `@username`
user_id	Integer	Yes	Unique identifier of the target user

getBusinessConnection

Use this method to get information about the connection of the bot with a business account. Returns a BusinessConnection object on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection

getManagedBotToken

Use this method to get the token of a managed bot. Returns the token as *String* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of the managed bot whose token will be returned

replaceManagedBotToken

Use this method to revoke the current token of a managed bot and generate a new one. Returns the new token as *String* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of the managed bot whose token will be replaced

getManagedBotAccessSettings

Use this method to get the access settings of a managed bot. Returns a BotAccessSettings object on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of the managed bot whose access settings will be returned

setManagedBotAccessSettings

Use this method to change the access settings of a managed bot. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of the managed bot whose access settings will be changed
is_access_restricted	Boolean	Yes	Pass *True* if only selected users can access the bot. The bot's owner can always access it.
added_user_ids	Array of Integer	Optional	A JSON-serialized list of up to 10 identifiers of users who will have access to the bot in addition to its owner. Ignored if *is_access_restricted* is *False*.

setMyCommands

Use this method to change the list of the bot's commands. See this manual for more details about bot commands. Returns *True* on success.

Parameter	Type	Required	Description
commands	Array of BotCommand	Yes	A JSON-serialized list of bot commands to be set as the list of the bot's commands. At most 100 commands can be specified.
scope	BotCommandScope	Optional	A JSON-serialized object, describing scope of users for which the commands are relevant. Defaults to BotCommandScopeDefault.
language_code	String	Optional	A two-letter ISO 639-1 language code. If empty, commands will be applied to all users from the given scope, for whose language there are no dedicated commands.

deleteMyCommands

Use this method to delete the list of the bot's commands for the given scope and user language. After deletion, higher level commands will be shown to affected users. Returns *True* on success.

Parameter	Type	Required	Description
scope	BotCommandScope	Optional	A JSON-serialized object, describing scope of users for which the commands are relevant. Defaults to BotCommandScopeDefault.
language_code	String	Optional	A two-letter ISO 639-1 language code. If empty, commands will be applied to all users from the given scope, for whose language there are no dedicated commands.

getMyCommands

Use this method to get the current list of the bot's commands for the given scope and user language. Returns an Array of BotCommand objects. If commands aren't set, an empty list is returned.

Parameter	Type	Required	Description
scope	BotCommandScope	Optional	A JSON-serialized object, describing scope of users. Defaults to BotCommandScopeDefault.
language_code	String	Optional	A two-letter ISO 639-1 language code or an empty string

setMyName

Use this method to change the bot's name. Returns *True* on success.

Parameter	Type	Required	Description
name	String	Optional	New bot name; 0-64 characters. Pass an empty string to remove the dedicated name for the given language.
language_code	String	Optional	A two-letter ISO 639-1 language code. If empty, the name will be shown to all users for whose language there is no dedicated name.

getMyName

Use this method to get the current bot name for the given user language. Returns BotName on success.

Parameter	Type	Required	Description
language_code	String	Optional	A two-letter ISO 639-1 language code or an empty string

setMyDescription

Use this method to change the bot's description, which is shown in the chat with the bot if the chat is empty. Returns *True* on success.

Parameter	Type	Required	Description
description	String	Optional	New bot description; 0-512 characters. Pass an empty string to remove the dedicated description for the given language.
language_code	String	Optional	A two-letter ISO 639-1 language code. If empty, the description will be applied to all users for whose language there is no dedicated description.

getMyDescription

Use this method to get the current bot description for the given user language. Returns BotDescription on success.

Parameter	Type	Required	Description
language_code	String	Optional	A two-letter ISO 639-1 language code or an empty string

setMyShortDescription

Use this method to change the bot's short description, which is shown on the bot's profile page and is sent together with the link when users share the bot. Returns *True* on success.

Parameter	Type	Required	Description
short_description	String	Optional	New short description for the bot; 0-120 characters. Pass an empty string to remove the dedicated short description for the given language.
language_code	String	Optional	A two-letter ISO 639-1 language code. If empty, the short description will be applied to all users for whose language there is no dedicated short description.

getMyShortDescription

Use this method to get the current bot short description for the given user language. Returns BotShortDescription on success.

Parameter	Type	Required	Description
language_code	String	Optional	A two-letter ISO 639-1 language code or an empty string

setMyProfilePhoto

Changes the profile photo of the bot. Returns *True* on success.

Parameter	Type	Required	Description
photo	InputProfilePhoto	Yes	The new profile photo to set

removeMyProfilePhoto

Removes the profile photo of the bot. Requires no parameters. Returns *True* on success.

setChatMenuButton

Use this method to change the bot's menu button in a private chat, or the default menu button. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer	Optional	Unique identifier for the target private chat. If not specified, the bot's default menu button will be changed.
menu_button	MenuButton	Optional	A JSON-serialized object for the bot's new menu button. Defaults to MenuButtonDefault.

getChatMenuButton

Use this method to get the current value of the bot's menu button in a private chat, or the default menu button. Returns MenuButton on success.

Parameter	Type	Required	Description
chat_id	Integer	Optional	Unique identifier for the target private chat. If not specified, the bot's default menu button will be returned.

setMyDefaultAdministratorRights

Use this method to change the default administrator rights requested by the bot when it's added as an administrator to groups or channels. These rights will be suggested to users, but they are free to modify the list before adding the bot. Returns *True* on success.

Parameter	Type	Required	Description
rights	ChatAdministratorRights	Optional	A JSON-serialized object describing new default administrator rights. If not specified, the default administrator rights will be cleared.
for_channels	Boolean	Optional	Pass *True* to change the default administrator rights of the bot in channels. Otherwise, the default administrator rights of the bot for groups and supergroups will be changed.

getMyDefaultAdministratorRights

Use this method to get the current default administrator rights of the bot. Returns ChatAdministratorRights on success.

Parameter	Type	Required	Description
for_channels	Boolean	Optional	Pass *True* to get default administrator rights of the bot in channels. Otherwise, default administrator rights of the bot for groups and supergroups will be returned.

getAvailableGifts

Returns the list of gifts that can be sent by the bot to users and channel chats. Requires no parameters. Returns a Gifts object.

sendGift

Sends a gift to the given user or channel chat. The gift can't be converted to Telegram Stars by the receiver. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Optional	Required if *chat_id* is not specified. Unique identifier of the target user who will receive the gift.
chat_id	Integer or String	Optional	Required if *user_id* is not specified. Unique identifier for the chat that will receive the gift or username of the channel in the format `@username`.
gift_id	String	Yes	Identifier of the gift; limited gifts can't be sent to channel chats
pay_for_upgrade	Boolean	Optional	Pass *True* to pay for the gift upgrade from the bot's balance, thereby making the upgrade free for the receiver
text	String	Optional	Text that will be shown along with the gift; 0-128 characters
text_parse_mode	String	Optional	Mode for parsing entities in the text. See formatting options for more details. Entities other than “bold”, “italic”, “underline”, “strikethrough”, “spoiler”, “custom_emoji”, and “date_time” are ignored.

Parameter	Type	Required	Description
text_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the gift text. It can be specified instead of *text_parse_mode*. Entities other than “bold”, “italic”, “underline”, “strikethrough”, “spoiler”, “custom_emoji”, and “date_time” are ignored.

giftPremiumSubscription

Gifts a Telegram Premium subscription to the given user. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user who will receive a Telegram Premium subscription
month_count	Integer	Yes	Number of months the Telegram Premium subscription will be active for the user; must be one of 3, 6, or 12
star_count	Integer	Yes	Number of Telegram Stars to pay for the Telegram Premium subscription; must be 1000 for 3 months, 1500 for 6 months, and 2500 for 12 months
text	String	Optional	Text that will be shown along with the service message about the subscription; 0-128 characters
text_parse_mode	String	Optional	Mode for parsing entities in the text. See formatting options for more details. Entities other than “bold”, “italic”, “underline”, “strikethrough”, “spoiler”, “custom_emoji”, and “date_time” are ignored.
text_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the gift text. It can be specified instead of *text_parse_mode*. Entities other than “bold”, “italic”, “underline”, “strikethrough”, “spoiler”, “custom_emoji”, and “date_time” are ignored.

verifyUser

Verifies a user on behalf of the organization which is represented by the bot. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user
custom_description	String	Optional	Custom description for the verification; 0-70 characters. Must be empty if the organization isn't allowed to provide a custom verification description.
custom_description_parse_mode	String	Optional	Mode for parsing entities in the custom description. See formatting options for more details. Only “text_link” entities are supported.

Parameter	Type	Required	Description
custom_description_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the custom description, which can be specified instead of *custom_description_parse_mode*. Only “text_link” entities are supported.

verifyChat

Verifies a chat on behalf of the organization which is represented by the bot. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`. Channel direct messages chats can't be verified.
custom_description	String	Optional	Custom description for the verification; 0-70 characters. Must be empty if the organization isn't allowed to provide a custom verification description.
custom_description_parse_mode	String	Optional	Mode for parsing entities in the custom description. See formatting options for more details. Only “text_link” entities are supported.
custom_description_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the custom description, which can be specified instead of *custom_description_parse_mode*. Only “text_link” entities are supported.

removeUserVerification

Removes verification from a user who is currently verified on behalf of the organization represented by the bot. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user

removeChatVerification

Removes verification from a chat that is currently verified on behalf of the organization represented by the bot. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot or channel in the format `@username`

readBusinessMessage

Marks incoming message as read on behalf of a business account. Requires the *can_read_messages* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection on behalf of which to read the message
chat_id	Integer	Yes	Unique identifier of the chat in which the message was received. The chat must have been active in the last 24 hours.
message_id	Integer	Yes	Unique identifier of the message to mark as read

deleteBusinessMessages

Delete messages on behalf of a business account. Requires the *can_delete_sent_messages* business bot right to delete messages sent by the bot itself, or the *can_delete_all_messages* business bot right to delete any message. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection on behalf of which to delete the messages
message_ids	Array of Integer	Yes	A JSON-serialized list of 1-100 identifiers of messages to delete. All messages must be from the same chat. See deleteMessage for limitations on which messages can be deleted.

setBusinessAccountName

Changes the first and last name of a managed business account. Requires the *can_change_name* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
first_name	String	Yes	The new value of the first name for the business account; 1-64 characters
last_name	String	Optional	The new value of the last name for the business account; 0-64 characters

setBusinessAccountUsername

Changes the username of a managed business account. Requires the *can_change_username* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
username	String	Optional	The new value of the username for the business account; 0-32 characters

setBusinessAccountBio

Changes the bio of a managed business account. Requires the *can_change_bio* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
bio	String	Optional	The new value of the bio for the business account; 0-140 characters

setBusinessAccountProfilePhoto

Changes the profile photo of a managed business account. Requires the *can_edit_profile_photo* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
photo	InputProfilePhoto	Yes	The new profile photo to set
is_public	Boolean	Optional	Pass *True* to set the public photo, which will be visible even if the main photo is hidden by the business account's privacy settings. An account can have only one public photo.

removeBusinessAccountProfilePhoto

Removes the current profile photo of a managed business account. Requires the *can_edit_profile_photo* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
is_public	Boolean	Optional	Pass *True* to remove the public photo, which is visible even if the main photo is hidden by the business account's privacy settings. After the main photo is removed, the previous profile photo (if present) becomes the main photo.

setBusinessAccountGiftSettings

Changes the privacy settings pertaining to incoming gifts in a managed business account. Requires the *can_change_gift_settings* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
show_gift_button	Boolean	Yes	Pass *True* if a button for sending a gift to the user or by the business account must always be shown in the input field
accepted_gift_types	AcceptedGiftTypes	Yes	Types of gifts accepted by the business account

getBusinessAccountStarBalance

Returns the amount of Telegram Stars owned by a managed business account. Requires the *can_view_gifts_and_stars* business bot right. Returns StarAmount on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection

transferBusinessAccountStars

Transfers Telegram Stars from the business account balance to the bot's balance. Requires the *can_transfer_stars* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
star_count	Integer	Yes	Number of Telegram Stars to transfer; 1-10000

getBusinessAccountGifts

Returns the gifts received and owned by a managed business account. Requires the *can_view_gifts_and_stars* business bot right. Returns OwnedGifts on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
exclude_unsaved	Boolean	Optional	Pass *True* to exclude gifts that aren't saved to the account's profile page
exclude_saved	Boolean	Optional	Pass *True* to exclude gifts that are saved to the account's profile page
exclude_unlimited	Boolean	Optional	Pass *True* to exclude gifts that can be purchased an unlimited number of times
exclude_limited_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can be upgraded to unique

Parameter	Type	Required	Description
exclude_limited_non_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can't be upgraded to unique
exclude_unique	Boolean	Optional	Pass *True* to exclude unique gifts
exclude_from_blockchain	Boolean	Optional	Pass *True* to exclude gifts that were assigned from the TON blockchain and can't be resold or transferred in Telegram
sort_by_price	Boolean	Optional	Pass *True* to sort results by gift price instead of send date. Sorting is applied before pagination.
offset	String	Optional	Offset of the first entry to return as received from the previous request; use empty string to get the first chunk of results
limit	Integer	Optional	The maximum number of gifts to be returned; 1-100. Defaults to 100.

getUserGifts

Returns the gifts owned and hosted by a user. Returns OwnedGifts on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the user
exclude_unlimited	Boolean	Optional	Pass *True* to exclude gifts that can be purchased an unlimited number of times
exclude_limited_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can be upgraded to unique
exclude_limited_non_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can't be upgraded to unique
exclude_from_blockchain	Boolean	Optional	Pass *True* to exclude gifts that were assigned from the TON blockchain and can't be resold or transferred in Telegram
exclude_unique	Boolean	Optional	Pass *True* to exclude unique gifts
sort_by_price	Boolean	Optional	Pass *True* to sort results by gift price instead of send date. Sorting is applied before pagination.
offset	String	Optional	Offset of the first entry to return as received from the previous request; use an empty string to get the first chunk of results
limit	Integer	Optional	The maximum number of gifts to be returned; 1-100. Defaults to 100.

getChatGifts

Returns the gifts owned by a chat. Returns OwnedGifts on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target channel in the format `@username`
exclude_unsaved	Boolean	Optional	Pass *True* to exclude gifts that aren't saved to the chat's profile page. Always *True*, unless the bot has the *can_post_messages* administrator right in the channel.
exclude_saved	Boolean	Optional	Pass *True* to exclude gifts that are saved to the chat's profile page. Always *False*, unless the bot has the *can_post_messages* administrator right in the channel.
exclude_unlimited	Boolean	Optional	Pass *True* to exclude gifts that can be purchased an unlimited number of times
exclude_limited_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can be upgraded to unique
exclude_limited_non_upgradable	Boolean	Optional	Pass *True* to exclude gifts that can be purchased a limited number of times and can't be upgraded to unique
exclude_from_blockchain	Boolean	Optional	Pass *True* to exclude gifts that were assigned from the TON blockchain and can't be resold or transferred in Telegram
exclude_unique	Boolean	Optional	Pass *True* to exclude unique gifts
sort_by_price	Boolean	Optional	Pass *True* to sort results by gift price instead of send date. Sorting is applied before pagination.
offset	String	Optional	Offset of the first entry to return as received from the previous request; use an empty string to get the first chunk of results
limit	Integer	Optional	The maximum number of gifts to be returned; 1-100. Defaults to 100.

getUniqueGift

Use this method to get information about a unique gift by its name. Returns a UniqueGift object on success.

Parameter	Type	Required	Description
name	String	Yes	Unique name of the gift

convertGiftToStars

Converts a given regular gift to Telegram Stars. Requires the *can_convert_gifts_to_stars* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
owned_gift_id	String	Yes	Unique identifier of the regular gift that should be converted to Telegram Stars

upgradeGift

Upgrades a given regular gift to a unique gift. Requires the *can_transfer_and_upgrade_gifts* business bot right. Additionally requires the *can_transfer_stars* business bot right if the upgrade is paid. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
owned_gift_id	String	Yes	Unique identifier of the regular gift that should be upgraded to a unique one
keep_original_details	Boolean	Optional	Pass *True* to keep the original gift text, sender and receiver in the upgraded gift
star_count	Integer	Optional	The amount of Telegram Stars that will be paid for the upgrade from the business account balance. If `gift.prepaid_upgrade_star_count > 0`, then pass 0, otherwise, the *can_transfer_stars* business bot right is required and `gift.upgrade_star_count` must be passed.

transferGift

Transfers an owned unique gift to another user. Requires the *can_transfer_and_upgrade_gifts* business bot right. Requires *can_transfer_stars* business bot right if the transfer is paid. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
owned_gift_id	String	Yes	Unique identifier of the regular gift that should be transferred
new_owner_chat_id	Integer	Yes	Unique identifier of the chat which will own the gift. The chat must be active in the last 24 hours.
star_count	Integer	Optional	The amount of Telegram Stars that will be paid for the transfer from the business account balance. If positive, then the *can_transfer_stars* business bot right is required.

deleteBusinessAccountGift

Deletes a regular gift owned by a managed business account without receiving Telegram Stars for it. Requires the *can_convert_gifts_to_stars* business bot right. The gift can be deleted only if it isn't limited (the field *total_count* of the Gift is omitted) and wasn't refunded. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
owned_gift_id	String	Yes	Unique identifier of the regular gift that should be deleted

postStory

Posts a story on behalf of a managed business account. Requires the *can_manage_stories* business bot right. Returns Story on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
content	InputStoryContent	Yes	Content of the story
active_period	Integer	Yes	Period after which the story is moved to the archive, in seconds; must be one of `6 * 3600`, `12 * 3600`, `86400`, or `2 * 86400`
caption	String	Optional	Caption of the story, 0-2048 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the story caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
areas	Array of StoryArea	Optional	A JSON-serialized list of clickable areas to be shown on the story
post_to_chat_page	Boolean	Optional	Pass *True* to keep the story accessible after it expires
protect_content	Boolean	Optional	Pass *True* if the content of the story must be protected from forwarding and screenshotting

repostStory

Reposts a story on behalf of a business account from another business account. Both business accounts must be managed by the same bot, and the story on the source account must have been posted (or reposted) by the bot. Requires the *can_manage_stories* business bot right for both business accounts. Returns Story on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
from_chat_id	Integer	Yes	Unique identifier of the chat which posted the story that should be reposted
from_story_id	Integer	Yes	Unique identifier of the story that should be reposted
active_period	Integer	Yes	Period after which the story is moved to the archive, in seconds; must be one of `6 * 3600`, `12 * 3600`, `86400`, or `2 * 86400`
post_to_chat_page	Boolean	Optional	Pass *True* to keep the story accessible after it expires
protect_content	Boolean	Optional	Pass *True* if the content of the story must be protected from forwarding and screenshotting

editStory

Edits a story previously posted by the bot on behalf of a managed business account. Requires the *can_manage_stories* business bot right. Returns Story on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
story_id	Integer	Yes	Unique identifier of the story to edit
content	InputStoryContent	Yes	Content of the story
caption	String	Optional	Caption of the story, 0-2048 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the story caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
areas	Array of StoryArea	Optional	A JSON-serialized list of clickable areas to be shown on the story

deleteStory

Deletes a story previously posted by the bot on behalf of a managed business account. Requires the *can_manage_stories* business bot right. Returns *True* on success.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection
story_id	Integer	Yes	Unique identifier of the story to delete

answerWebAppQuery

Use this method to set the result of an interaction with a Web App and send a corresponding message on behalf of the user to the chat from which the query originated. On success, a SentWebAppMessage object is returned.

Parameter	Type	Required	Description
web_app_query_id	String	Yes	Unique identifier for the query to be answered
result	InlineQueryResult	Yes	A JSON-serialized object describing the message to be sent

savePreparedInlineMessage

Stores a message that can be sent by a user of a Mini App. Returns a PreparedInlineMessage object.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user that can use the prepared message
result	InlineQueryResult	Yes	A JSON-serialized object describing the message to be sent
allow_user_chats	Boolean	Optional	Pass *True* if the message can be sent to private chats with users
allow_bot_chats	Boolean	Optional	Pass *True* if the message can be sent to private chats with bots
allow_group_chats	Boolean	Optional	Pass *True* if the message can be sent to group and supergroup chats
allow_channel_chats	Boolean	Optional	Pass *True* if the message can be sent to channel chats

savePreparedKeyboardButton

Stores a keyboard button that can be used by a user within a Mini App. Returns a PreparedKeyboardButton object.

Parameter	Type	Required	Description
user_id	Integer	Yes	Unique identifier of the target user that can use the button
button	KeyboardButton	Yes	A JSON-serialized object describing the button to be saved. The button must be of the type *request_users*, *request_chat*, or *request_managed_bot*.

Inline mode methods

Methods and objects used in the inline mode are described in the Inline mode section.

Updating messages

The following methods allow you to change an existing message in the message history instead of sending a new one with a result of an action. This is most useful for messages with inline keyboards using callback queries, but can also help reduce clutter in conversations with regular chat bots.

Please note, that it is currently only possible to edit messages without *reply_markup* or with inline keyboards.

editMessageText

Use this method to edit text, rich and game messages. On success, if the edited message is not an inline message, the edited Message is returned, otherwise *True* is returned. Note that business messages that were not sent by the bot and do not contain an inline keyboard can only be edited within **48 hours** from the time they were sent.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message to edit.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
text	String	Optional	New text of the message, 1-4096 characters after entity parsing; required if *rich_message* isn't specified
parse_mode	String	Optional	Mode for parsing entities in the message text. See formatting options for more details.
entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in message text, which can be specified instead of *parse_mode*
link_preview_options	LinkPreviewOptions	Optional	Link preview generation options for the message
rich_message	InputRichMessage	Optional	New rich content of the message; required if *text* isn't specified. Direct upload of new files and explicit upload of files by a URL isn't supported when an inline message is edited.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

editMessageCaption

Use this method to edit captions of messages. On success, if the edited message is not an inline message, the edited Message is returned, otherwise *True* is returned. Note that business messages that were not sent by the bot and do not contain an inline keyboard can only be edited within **48 hours** from the time they were sent.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message to edit.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
caption	String	Optional	New caption of the message, 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the message caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media. Supported only for animation, photo and video messages.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

editMessageMedia

Use this method to edit animation, audio, document, live photo, photo, or video messages, or to replace a text or a rich message with a media. If a message is part of a message album, then it can be edited only to an audio for audio albums, only to a document for document albums and to a photo, a live photo, or a video otherwise. When an inline message is edited, a new file can't be uploaded; use a previously uploaded file via its file_id or specify a URL. On success, if the edited message is not an inline message, the edited Message is returned, otherwise *True* is returned. Note that business messages that were not sent by the bot and do not contain an inline keyboard can only be edited within **48 hours** from the time they were sent.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message to edit.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
media	InputMedia	Yes	A JSON-serialized object for the new media content of the message
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for a new inline keyboard

editMessageLiveLocation

Use this method to edit live location messages. A location can be edited until its *live_period* expires or editing is explicitly disabled by a call to stopMessageLiveLocation. On success, if the edited message is not an inline message, the edited Message is returned, otherwise *True* is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message to edit.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
latitude	Float	Yes	Latitude of new location
longitude	Float	Yes	Longitude of new location

Parameter	Type	Required	Description
live_period	Integer	Optional	New period in seconds during which the location can be updated, starting from the message send date. If 0x7FFFFFFF is specified, then the location can be updated forever. Otherwise, the new value must not exceed the current *live_period* by more than a day, and the live location expiration date must remain within the next 90 days. If not specified, then *live_period* remains unchanged.
horizontal_accuracy	Float	Optional	The radius of uncertainty for the location, measured in meters; 0-1500
heading	Integer	Optional	Direction in which the user is moving, in degrees. Must be between 1 and 360 if specified.
proximity_alert_radius	Integer	Optional	The maximum distance for proximity alerts about approaching another chat member, in meters. Must be between 1 and 100000 if specified.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for a new inline keyboard

stopMessageLiveLocation

Use this method to stop updating a live location message before *live_period* expires. On success, if the message is not an inline message, the edited Message is returned, otherwise *True* is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message with live location to stop.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for a new inline keyboard

editMessageChecklist

Use this method to edit a checklist on behalf of a connected business account. On success, the edited Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Yes	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot in the format `@username`
message_id	Integer	Yes	Unique identifier for the target message
checklist	InputChecklist	Yes	A JSON-serialized object for the new checklist
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for the new inline keyboard for the message

editMessageWidgetData

Use this method to edit data of a widget in a message sent by the bot or via the bot (for inline bots). Public data of a shared widget can be changed at most once per second. On success, *True* is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target supergroup in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message with the widget.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
public_data	String	Optional	New public data of the widget, 0-1024 bytes; pass an empty string to remove the public data. If omitted, then the public data isn't changed.
chat_data	String	Optional	New data of the widget, which is specific to the message, 0-1024 bytes; pass an empty string to remove the data. If omitted, then the data isn't changed.

editSharedWidgetData

Use this method to edit public data of a shared widget created by the bot. A shared widget can be edited at most once per second. On success, *True* is returned.

Parameter	Type	Required	Description
widget_id	String	Yes	Unique identifier of the shared widget
public_data	String	Yes	New public data of the widget, 0-1024 bytes; pass an empty string to remove the public data

Parameter	Type	Required	Description
cache_time	Integer	Optional	The maximum amount of time in seconds that the public data of the widget may be cached client-side; 5-3600. If omitted, then the current value is kept, which is 30 by default.

editMessageReplyMarkup

Use this method to edit only the reply markup of messages. On success, if the edited message is not an inline message, the edited Message is returned, otherwise *True* is returned. Note that business messages that were not sent by the bot and do not contain an inline keyboard can only be edited within **48 hours** from the time they were sent.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the message to edit.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

stopPoll

Use this method to stop a poll which was sent by the bot. On success, the stopped Poll is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message to be edited was sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_id	Integer	Yes	Identifier of the original message with the poll
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for a new message inline keyboard

editEphemeralMessageText

Use this method to edit an ephemeral text or rich message. Note that it is not guaranteed that the user will receive the message edit event, especially if they are offline. On success, *True* is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
receiver_user_id	Integer	Yes	Identifier of the user who received the message
ephemeral_message_id	Integer	Yes	Identifier of the ephemeral message to edit
text	String	Optional	New text of the message, 1-4096 characters after entity parsing; required if *rich_message* isn't specified
parse_mode	String	Optional	Mode for parsing entities in the message text. See formatting options for more details.
entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in message text, which can be specified instead of *parse_mode*
rich_message	InputRichMessage	Optional	New rich content of the message; required if *text* isn't specified
link_preview_options	LinkPreviewOptions	Optional	Link preview generation options for the message
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

editEphemeralMessageMedia

Use this method to edit the media of an ephemeral message. Note that it is not guaranteed that the user will receive the message edit event, especially if they are offline. On success, *True* is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
receiver_user_id	Integer	Yes	Identifier of the user who received the message
ephemeral_message_id	Integer	Yes	Identifier of the ephemeral message to edit
media	InputMedia	Yes	A JSON-serialized object for the new media content of the message
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

editEphemeralMessageCaption

Use this method to edit the caption of an ephemeral message. Note that it is not guaranteed that the user will receive the message edit event, especially if they are offline. On success, *True* is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
receiver_user_id	Integer	Yes	Identifier of the user who received the message
ephemeral_message_id	Integer	Yes	Identifier of the ephemeral message to edit
caption	String	Optional	New caption of the message, 0-1024 characters after entities parsing
parse_mode	String	Optional	Mode for parsing entities in the message caption. See formatting options for more details.
caption_entities	Array of MessageEntity	Optional	A JSON-serialized list of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	Optional	Pass *True* if the caption must be shown above the message media. Supported only for animation, photo and video messages.
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

editEphemeralMessageReplyMarkup

Use this method to edit only the reply markup of an ephemeral message. Note that it is not guaranteed that the user will receive the message edit event, especially if they are offline. On success, *True* is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
receiver_user_id	Integer	Yes	Identifier of the user who received the message
ephemeral_message_id	Integer	Yes	Identifier of the ephemeral message to edit
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard

approveSuggestedPost

Use this method to approve a suggested post in a direct messages chat. The bot must have the 'can_post_messages' administrator right in the corresponding channel chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer	Yes	Unique identifier for the target direct messages chat

Parameter	Type	Required	Description
message_id	Integer	Yes	Identifier of a suggested post message to approve
send_date	Integer	Optional	Point in time (Unix timestamp) when the post is expected to be published; omit if the date has already been specified when the suggested post was created. If specified, then the date must be not more than 2678400 seconds (30 days) in the future.

declineSuggestedPost

Use this method to decline a suggested post in a direct messages chat. The bot must have the 'can_manage_direct_messages' administrator right in the corresponding channel chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer	Yes	Unique identifier for the target direct messages chat
message_id	Integer	Yes	Identifier of a suggested post message to decline
comment	String	Optional	Comment for the creator of the suggested post; 0-128 characters

deleteMessage

Use this method to delete a message, including service messages, with the following limitations:
- A message can only be deleted if it was sent less than 48 hours ago.
- Service messages about a supergroup, channel, or forum topic creation can't be deleted.
- A dice message in a private chat can only be deleted if it was sent more than 24 hours ago.
- Bots can delete outgoing messages in private chats, groups, and supergroups.
- Bots can delete incoming messages in private chats.
- Bots granted *can_post_messages* permissions can delete outgoing messages in channels.
- If the bot is an administrator of a group, it can delete any message there.
- If the bot has *can_delete_messages* administrator right in a supergroup or a channel, it can delete any message there.
- If the bot has *can_manage_direct_messages* administrator right in a channel, it can delete any message in the corresponding direct messages chat.
Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_id	Integer	Yes	Identifier of the message to delete

deleteMessages

Use this method to delete multiple messages simultaneously. If some of the specified messages can't be found, they are skipped. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_ids	Array of Integer	Yes	A JSON-serialized list of 1-100 identifiers of messages to delete. See deleteMessage for limitations on which messages can be deleted.

deleteEphemeralMessage

Use this method to delete an ephemeral message. Note that it is not guaranteed that the user will receive the message deletion event, especially if they are offline. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
receiver_user_id	Integer	Yes	Identifier of the user who received the message
ephemeral_message_id	Integer	Yes	Identifier of the ephemeral message to delete

deleteMessageReaction

Use this method to remove a reaction from a message in a group or a supergroup chat. The bot must have the 'can_delete_messages' administrator right in the chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
message_id	Integer	Yes	Identifier of the target message
user_id	Integer	Optional	Identifier of the user whose reaction will be removed, if the reaction was added by a user
actor_chat_id	Integer	Optional	Identifier of the chat whose reaction will be removed, if the reaction was added by a chat

deleteAllMessageReactions

Use this method to remove up to 10000 recent reactions in a group or a supergroup chat added by a given user or chat. The bot must have the 'can_delete_messages' administrator right in the chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target supergroup in the format `@username`
user_id	Integer	Optional	Identifier of the user whose reactions will be removed, if the reactions were added by a user

Parameter	Type	Required	Description
actor_chat_id	Integer	Optional	Identifier of the chat whose reactions will be removed, if the reactions were added by a chat

Stickers

The following methods and objects allow your bot to handle stickers and sticker sets.

Sticker

This object represents a sticker.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
type	String	Type of the sticker, currently one of “regular”, “mask”, “custom_emoji”. The type of the sticker is independent from its format, which is determined by the fields *is_animated* and *is_video*.
width	Integer	Sticker width
height	Integer	Sticker height
is_animated	Boolean	*True*, if the sticker is animated
is_video	Boolean	*True*, if the sticker is a video sticker
thumbnail	PhotoSize	*Optional*. Sticker thumbnail in the .WEBP or .JPG format
emoji	String	*Optional*. Emoji associated with the sticker
set_name	String	*Optional*. Name of the sticker set to which the sticker belongs
premium_animation	File	*Optional*. For premium regular stickers, premium animation for the sticker
mask_position	MaskPosition	*Optional*. For mask stickers, the position where the mask should be placed
custom_emoji_id	String	*Optional*. For custom emoji stickers, unique identifier of the custom emoji
needs_repainting	True	*Optional*. *True*, if the sticker must be repainted to a text color in messages, the color of the Telegram Premium badge in emoji status, white color on chat photos, or another appropriate color in other places
file_size	Integer	*Optional*. File size in bytes

StickerSet

This object represents a sticker set.

Field	Type	Description
name	String	Sticker set name
title	String	Sticker set title
sticker_type	String	Type of stickers in the set, currently one of “regular”, “mask”, “custom_emoji”
stickers	Array of Sticker	List of all set stickers
thumbnail	PhotoSize	*Optional*. Sticker set thumbnail in the .WEBP, .TGS, or .WEBM format

MaskPosition

This object describes the position on faces where a mask should be placed by default.

Field	Type	Description
point	String	The part of the face relative to which the mask should be placed. One of “forehead”, “eyes”, “mouth”, or “chin”.
x_shift	Float	Shift by X-axis measured in widths of the mask scaled to the face size, from left to right. For example, choosing -1.0 will place mask just to the left of the default mask position.
y_shift	Float	Shift by Y-axis measured in heights of the mask scaled to the face size, from top to bottom. For example, 1.0 will place the mask just below the default mask position.
scale	Float	Mask scaling coefficient. For example, 2.0 means double size.

InputSticker

This object describes a sticker to be added to a sticker set.

Field	Type	Description
sticker	String	The added sticker. Pass a *file_id* as a String to send a file that already exists on the Telegram servers, pass an HTTP URL as a String for Telegram to get a file from the Internet, or pass “attach://<file_attach_name>” to upload a new file using multipart/form-data under <file_attach_name> name. Animated and video stickers can't be uploaded via HTTP URL. More information on Sending Files »
format	String	Format of the added sticker, must be one of “static” for a **.WEBP** or **.PNG** image, “animated” for a **.TGS** animation, “video” for a **.WEBM** video
emoji_list	Array of String	List of 1-20 emoji associated with the sticker
mask_position	MaskPosition	*Optional*. Position where the mask should be placed on faces. For “mask” stickers only.
keywords	Array of String	*Optional*. List of 0-20 search keywords for the sticker with total length of up to 64 characters. For “regular” and “custom_emoji” stickers only.

sendSticker

Use this method to send static .WEBP, animated .TGS, or video .WEBM stickers. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
sticker	InputFile or String	Yes	Sticker to send. Pass a file_id as String to send a file that exists on the Telegram servers (recommended), pass an HTTP URL as a String for Telegram to get a .WEBP sticker from the Internet, or upload a new .WEBP, .TGS, or .WEBM sticker using multipart/form-data. More information on Sending Files ». Video and animated stickers can't be sent via an HTTP URL.

Parameter	Type	Required	Description
emoji	String	Optional	Emoji associated with the sticker; only for just uploaded stickers
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

getStickerSet

Use this method to get a sticker set. On success, a StickerSet object is returned.

Parameter	Type	Required	Description
name	String	Yes	Name of the sticker set

getCustomEmojiStickers

Use this method to get information about custom emoji stickers by their identifiers. Returns an Array of Sticker objects.

Parameter	Type	Required	Description
custom_emoji_ids	Array of String	Yes	A JSON-serialized list of custom emoji identifiers. At most 200 custom emoji identifiers can be specified.

uploadStickerFile

Use this method to upload a file with a sticker for later use in the createNewStickerSet, addStickerToSet, or replaceStickerInSet methods (the file can be used multiple times). Returns the uploaded File on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of sticker file owner
sticker	InputFile	Yes	A file with the sticker in .WEBP, .PNG, .TGS, or .WEBM format. See https://core.telegram.org/stickers for technical requirements. More information on Sending Files »
sticker_format	String	Yes	Format of the sticker, must be one of “static”, “animated”, “video”

createNewStickerSet

Use this method to create a new sticker set owned by a user. The bot will be able to edit the sticker set thus created. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of created sticker set owner
name	String	Yes	Short name of sticker set, to be used in `t.me/addstickers/` URLs (e.g., *animals*). Can contain only English letters, digits and underscores. Must begin with a letter, can't contain consecutive underscores and must end in `"_by_<bot_username>"`. `<bot_username>` is case insensitive. 1-64 characters.
title	String	Yes	Sticker set title, 1-64 characters
stickers	Array of InputSticker	Yes	A JSON-serialized list of 1-50 initial stickers to be added to the sticker set
sticker_type	String	Optional	Type of stickers in the set, pass “regular”, “mask”, or “custom_emoji”. By default, a regular sticker set is created.
needs_repainting	Boolean	Optional	Pass *True* if stickers in the sticker set must be repainted to the color of text when used in messages, the accent color if used as emoji status, white on chat photos, or another appropriate color based on context; for custom emoji sticker sets only

addStickerToSet

Use this method to add a new sticker to a set created by the bot. Emoji sticker sets can have up to 200 stickers. Other sticker sets can have up to 120 stickers. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of sticker set owner
name	String	Yes	Sticker set name
sticker	InputSticker	Yes	A JSON-serialized object with information about the added sticker. If exactly the same sticker had already been added to the set, then the set isn't changed.

setStickerPositionInSet

Use this method to move a sticker in a set created by the bot to a specific position. Returns *True* on success.

Parameter	Type	Required	Description
sticker	String	Yes	File identifier of the sticker
position	Integer	Yes	New sticker position in the set, zero-based

deleteStickerFromSet

Use this method to delete a sticker from a set created by the bot. Returns *True* on success.

Parameter	Type	Required	Description
sticker	String	Yes	File identifier of the sticker

replaceStickerInSet

Use this method to replace an existing sticker in a sticker set with a new one. The method is equivalent to calling deleteStickerFromSet, then addStickerToSet, then setStickerPositionInSet. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier of the sticker set owner
name	String	Yes	Sticker set name
old_sticker	String	Yes	File identifier of the replaced sticker
sticker	InputSticker	Yes	A JSON-serialized object with information about the added sticker. If exactly the same sticker had already been added to the set, then the set remains unchanged.

setStickerEmojiList

Use this method to change the list of emoji assigned to a regular or custom emoji sticker. The sticker must belong to a sticker set created by the bot. Returns *True* on success.

Parameter	Type	Required	Description
sticker	String	Yes	File identifier of the sticker
emoji_list	Array of String	Yes	A JSON-serialized list of 1-20 emoji associated with the sticker

setStickerKeywords

Use this method to change search keywords assigned to a regular or custom emoji sticker. The sticker must belong to a sticker set created by the bot. Returns *True* on success.

Parameter	Type	Required	Description
sticker	String	Yes	File identifier of the sticker
keywords	Array of String	Optional	A JSON-serialized list of 0-20 search keywords for the sticker with total length of up to 64 characters

setStickerMaskPosition

Use this method to change the mask position of a mask sticker. The sticker must belong to a sticker set that was created by the bot. Returns *True* on success.

Parameter	Type	Required	Description
sticker	String	Yes	File identifier of the sticker
mask_position	MaskPosition	Optional	A JSON-serialized object with the position where the mask should be placed on faces. Omit the parameter to remove the mask position.

setStickerSetTitle

Use this method to set the title of a created sticker set. Returns *True* on success.

Parameter	Type	Required	Description
name	String	Yes	Sticker set name
title	String	Yes	Sticker set title, 1-64 characters

setStickerSetThumbnail

Use this method to set the thumbnail of a regular or mask sticker set. The format of the thumbnail file must match the format of the stickers in the set. Returns *True* on success.

Parameter	Type	Required	Description
name	String	Yes	Sticker set name
user_id	Integer	Yes	User identifier of the sticker set owner
thumbnail	InputFile or String	Optional	A **.WEBP** or **.PNG** image with the thumbnail, must be up to 128 kilobytes in size and have a width and height of exactly 100px, or a **.TGS** animation with a thumbnail up to 32 kilobytes in size (see https://core.telegram.org/stickers#animation-requirements for animated sticker technical requirements), or a **.WEBM** video with the thumbnail up to 32 kilobytes in size; see https://core.telegram.org/stickers#video-requirements for video sticker technical requirements. Pass a *file_id* as a String to send a file that already exists on the Telegram servers, pass an HTTP URL as a String for Telegram to get a file from the Internet, or upload a new one using multipart/form-data. More information on Sending Files ». Animated and video sticker set thumbnails can't be uploaded via HTTP URL. If omitted, then the thumbnail is dropped and the first sticker is used as the thumbnail.
format	String	Yes	Format of the thumbnail, must be one of “static” for a **.WEBP** or **.PNG** image, “animated” for a **.TGS** animation, or “video” for a **.WEBM** video

setCustomEmojiStickerSetThumbnail

Use this method to set the thumbnail of a custom emoji sticker set. Returns *True* on success.

Parameter	Type	Required	Description
name	String	Yes	Sticker set name
custom_emoji_id	String	Optional	Custom emoji identifier of a sticker from the sticker set; pass an empty string to drop the thumbnail and use the first sticker as the thumbnail

deleteStickerSet

Use this method to delete a sticker set that was created by the bot. Returns *True* on success.

Parameter	Type	Required	Description
name	String	Yes	Sticker set name

Rich messages

The following methods and objects allow your bot to handle and send rich messages.

Rich Message Formatting Options

Rich messages support advanced structured formatting options like headings, lists, tables, media, block quotations, collapsible blocks, footnotes, and formulas. Telegram clients will render them accordingly. You can specify rich message content using Markdown-style or HTML-style formatting, or explicit blocks.

Plain URLs, e-mail addresses, username mentions, hashtags, cashtags, bot commands, phone numbers, and bank card numbers are detected automatically. To disable automatic entity detection, pass *True* in the *skip_entity_detection* field. Note that Telegram clients will display an alert to the user before opening an inline link ('Open this link?' together with the full URL).

When Markdown-style or HTML-style formatting is used, you can use links in the form `tg://photo?id=...`, `tg://video?id=...`, `tg://document?id=...`, and `tg://audio?id=...` instead of an HTTP URL to reuse previously uploaded files or upload a new file.

Rich Message Limits

Rich messages are subject to the following limits:

- Up to **32768 UTF-8 characters** in the rich message text, including custom emoji alternative text and formula source.
- Up to **500 blocks**, including nested blocks, list items, ordered list items, table rows, quotation blocks, and details blocks.
- Up to **16 levels** of nested formatting and blocks.
- Up to **50 media attachments** in total.
- Up to **20 columns** in a table.

Rich Markdown style

To use this mode, pass rich message content in the *markdown* field. Use the following syntax in your message:

```
**bold text**
__bold text__
*italic text*
_italic text_
~~strikethrough text~~
`inline fixed-width code`
==marked text==
||spoiler||

[inline URL](https://t.me/)
[inline e-mail](mailto:user@example.com)
[inline phone number](tel:+123456789)
[inline mention of a user](tg://user?id=123456789)
![👍](tg://emoji?id=5368324170671202286)
![22:45 tomorrow](tg://time?unix=1647531900&format=wDT)
$x^2 + y^2$
\#hashtag $USD +12345678901, card: 4242 4242 4242 4242, https://t.me t.me a@t.me
/command @username
all the text above was on the same line

# Heading 1
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
###### Heading 6
```

````
Paragraph text

```python
  print('pre-formatted fixed-width code block written in the Python programming
language')
```

---

- unordered list item
* unordered list item
+ unordered list item

1. ordered list item
2. ordered list item

- [ ] task list item
- [x] completed task list item

>Block quotation started
>
>Block quotation continued on the next line
>Block quotation continued on the same line
>
>The last line of the block quotation

![](https://telegram.org/example/photo.jpg)
![](https://telegram.org/example/video.mp4)
![](https://telegram.org/example/audio.mp3)
![](https://telegram.org/example/audio.ogg)
![](https://telegram.org/example/animation.gif)
![](https://telegram.org/example/document.zip)

![](https://telegram.org/example/photo.jpg "Photo caption")
![](https://telegram.org/example/video.mp4 "Video caption")
![](https://telegram.org/example/audio.mp3 "Audio caption")
![](https://telegram.org/example/audio.ogg "Voice note caption")
![](https://telegram.org/example/animation.gif "Animation caption")
![](https://telegram.org/example/document.zip "Document caption")

| Header 1 | Header 2 |
|:---------|:--------:|
| left     | center   |

Text with a reference[^id1] and another one[^id2].

[^id1]: Definition of the first footnote.
[^id2]: Definition of the second footnote.

$$E = mc^2$$
````

```math
E = mc^2
```

Example Nested Syntax Report for _Q1_
Intro with <u>underlined text</u>, ==marked text==, and $x^2 + y^2$.
Bold _italic <u>underlined italic bold</u> italic_ bold
<u>In inline tags, nested **markdown** is parsed</u>
>Quote with **bold text, ~~strikethrough, and <tg-spoiler>spoiler</tg-spoiler>~~**,
plus [a link](https://t.me/).

- List item with `code`, ^{superscript}, _{subscript}, and a
footnote[^note]
- Another item with **bold <tg-spoiler><code>spoiler code</code></tg-spoiler>**
- Another item with ~~strikethrough and <ins>inserted text</ins>~~

Metric	Value
Speed	**42** ^{ms}
Status	<tg-spoiler>ready</tg-spoiler>

[^note]: Footnote with _italic text_ and <u>HTML underline</u>.

Details blocks can contain Markdown content:

<details open><summary>Summary with **bold text**</summary>

Details heading
- List item with _italic text_
- List item with <tg-spoiler>spoiler</tg-spoiler>

</details>

Collages and slideshows can contain Markdown media blocks:

<tg-collage>

</tg-collage>

<tg-slideshow>


```
</tg-slideshow>
```

For formatting features that don't have Markdown syntax, use HTML tags:

```
<u>underlined text</u>, <ins>underlined text</ins>
<sub>subscript text</sub>
<sup>superscript text</sup>
<a name="chapter-1"></a>
<aside>Pull quote<cite>The Author</cite></aside>
<details open><summary>Title</summary>Content</details>
<tg-map lat="41.9" long="12.5" zoom="14"/>
<tg-collage><img src="https://telegram.org/example/photo.jpg"/>
<figcaption>Caption<cite>The Author</cite></figcaption></tg-collage>
<tg-slideshow><img src="https://telegram.org/example/photo.jpg"/><video
src="https://telegram.org/example/video.mp4"/><figcaption>Slideshow
caption<cite>The Author</cite></figcaption></tg-slideshow>
<p>Inline buttons:
  <tg-button type="url" style="success" url="https://t.me">url</tg-button>
  <tg-button type="url" url="tg://user?id=777000">user</tg-button>
  <tg-button type="callback_data" style="link" data="callback">callback with the
date <tg-time unix="1647531900" format="wDT">22:45 tomorrow</tg-time> and the
custom emoji <tg-emoji emoji-id="5368324170671202286">👍</tg-emoji></tg-button>
  <tg-button type="web_app" style="danger" url="https://telegram.org">Mini App
(private chats only)</tg-button>
  <tg-button type="login_url" url="https://t.me" forward-text="forward text"
request-write-access>login (requires domain set up via @BotFather)</tg-button>
  <tg-button type="switch_inline_query" style="primary" query="inline">inline</tg-
button>
  <tg-button type="switch_inline_query_current_chat" query="inline 2">inline 2</tg-
button>
  <tg-button type="switch_inline_query_chosen_chat" query="inline 3" allow-user-
chats allow-bot-chats allow-group-chats allow-channel-chats>inline 3</tg-button>
  <tg-button type="copy_text" text="...copy">Copy</tg-button>
  <tg-button type="disabled">Disabled</tg-button>
</p>
<tg-button-row align="left">
  <tg-button type="url" url="https://t.me">url</tg-button>
  <tg-button type="url" style="success" url="tg://user?id=777000">user</tg-button>
  <tg-button type="callback_data" style="link" data="callback">callback</tg-button>
</tg-button-row>
<tg-button-row align="center">
  <tg-button type="web_app" url="https://telegram.org">Mini App (private chats
only)</tg-button>
</tg-button-row>
<tg-button-row align="center">
  <tg-button type="login_url" style="danger" url="https://t.me" forward-
text="forward text" request-write-access>login (requires domain set up via
@BotFather)</tg-button>
</tg-button-row>
```

```
<tg-button-row align="right">
  <tg-button type="switch_inline_query" query="inline">inline</tg-button>
  <tg-button type="switch_inline_query_current_chat" query="inline 2">inline 2</tg-
button>
  <tg-button type="switch_inline_query_chosen_chat" query="inline 3" allow-user-
chats allow-group-chats allow-channel-chats>inline 3</tg-button>
</tg-button-row>
<tg-button-row>
  <tg-button type="copy_text" text="...copy">Copy</tg-button>
  <tg-button type="disabled" style="primary">Disabled</tg-button>
</tg-button-row>
```

Additionally, you can use the following tag in sendRichMessageDraft:

```
<tg-thinking>Thinking...</tg-thinking>
```

Please note:

- Rich Markdown is compatible with GitHub Flavored Markdown where possible and can contain arbitrary HTML. Supported rich message HTML tags are parsed as described in Rich HTML style.
- Media can be specified only as a separate block.
- Media blocks support only HTTP and HTTPS URLs.
- Media type is determined by the MIME type and the URL of the media.
- In media syntax, the optional title after the URL is used as the caption; for example, displays “Photo caption” under the media.
- Table cells can contain only inline formatting.
- Formula source is treated as raw LaTeX.
- Markdown isn't parsed inside block HTML tags other than <details>, <tg-collage> and <tg-slideshow>, therefore only HTML tags can be used there.
- See date-time entity formatting for more details about supported date-time formats.

Rich HTML style

To use this mode, pass rich message content in the *html* field. The following tags are currently supported:

```
<a name="chapter-0"></a>
<b>bold text</b>, <strong>bold text</strong>
<i>italic text</i>, <em>italic text</em>
<u>underlined text</u>, <ins>underlined text</ins>
<s>strikethrough text</s>, <strike>strikethrough text</strike>, <del>strikethrough
text</del>
<code>inline fixed-width code</code>
<mark>marked text</mark>
<sub>subscript text</sub>
<sup>superscript text</sup>
<tg-spoiler>spoiler</tg-spoiler>

<a href="#note-1">Reference</a>
```

```
<a href="https://t.me/">inline URL</a>
<a href="mailto:user@example.com">inline e-mail</a>
<a href="tel:+123456789">inline phone number</a>
<a href="tg://user?id=123456789">inline mention of a user</a>
<a href="#chapter-1">in-document link</a>
<a name="chapter-1"></a>

<tg-reference name="note-1">Referenced text</tg-reference>
<tg-emoji emoji-id="5368324170671202286">👍</tg-emoji>
<img src="tg://emoji?id=5368324170671202286" alt="👍"/>
<tg-time unix="1647531900" format="wDT">22:45 tomorrow</tg-time>
<tg-math>x^2 + y^2</tg-math>

#hashtag $USD +12345678901, card: 4242 4242 4242 4242, https://t.me t.me a@t.me
/command @username

all the text above was on the same line

<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>

<a name="chapter-2"></a>

<p>Paragraph text</p>
<pre>pre-formatted fixed-width code block</pre>
<pre><code class="language-python">  print('pre-formatted fixed-width code block
written in the Python programming language')</code></pre>
<footer>Footer text</footer>
<hr/>
<ul><li>unordered list item</li></ul>
<ol><li>ordered list item</li></ol>
<ol start="3" type="a" reversed><li>ordered list item</li></ol>
<ol><li value="7" type="i">ordered list item with explicit number</li></ol>
<ul>
<li><input type="checkbox" checked>Checked checkbox</li>
<li><input type="checkbox">Unchecked checkbox</li>
</ul>

<blockquote>Block quotation started<br>Block quotation continued<br>The last line
of the block quotation<cite>The Author</cite></blockquote>
<blockquote expandable>Expandable block quotation started<br>Expandable block
quotation continued<br>Expandable block quotation continued<br>Expandable block
quotation continued<br>The last line of the expandable block quotation<cite>The
Author</cite></blockquote>
<aside>Pull quote<cite>The Author</cite></aside>
```

```
<img src="https://telegram.org/example/photo.jpg"/>
<video src="https://telegram.org/example/video.mp4"></video>
<audio src="https://telegram.org/example/audio.mp3"></audio>
<audio src="https://telegram.org/example/audio.ogg"></audio>
<video src="https://telegram.org/example/animation.gif"></video>
<tg-document src="https://telegram.org/example/document.zip"></tg-document>

<figure><img src="https://telegram.org/example/photo.jpg" tg-spoiler/>
<figcaption>Photo caption<cite>Photo credit</cite></figcaption></figure>
<figure><video src="https://telegram.org/example/video.mp4" tg-spoiler></video>
<figcaption>Video caption</figcaption></figure>
<figure><audio src="https://telegram.org/example/audio.mp3"></audio>
<figcaption>Audio caption</figcaption></figure>
<figure><audio src="https://telegram.org/example/audio.ogg"></audio>
<figcaption>Voice note caption</figcaption></figure>
<figure><video src="https://telegram.org/example/animation.gif" tg-spoiler></video>
<figcaption>Animation caption</figcaption></figure>
<figure><tg-document src="https://telegram.org/example/document.zip"></tg-document>
<figcaption>Document caption</figcaption></figure>

<tg-map lat="41.9" long="12.5" zoom="14"/>
<figure><tg-map lat="41.9" long="12.5" zoom="14"/><figcaption>Map
caption</figcaption></figure>

<tg-collage><img src="https://telegram.org/example/photo.jpg"/><video
src="https://telegram.org/example/video.mp4"/></tg-collage>
<tg-collage><video src="https://telegram.org/example/video.mp4"/><img
src="https://telegram.org/example/photo.jpg"/><figcaption>Collage
caption</figcaption></tg-collage>
<tg-slideshow><img src="https://telegram.org/example/photo.jpg"/><video
src="https://telegram.org/example/video.mp4"/></tg-slideshow>
<tg-slideshow><video src="https://telegram.org/example/video.mp4"/><img
src="https://telegram.org/example/photo.jpg"/><figcaption>Slideshow
caption</figcaption></tg-slideshow>

<table><tr><th>Header 1</th><th>Header 2</th></tr><tr><td>Value 1</td><td>Value
2</td></tr></table>
<table bordered striped compact><caption>Table caption</caption>
<tr><td colspan="2" rowspan="2" align="left">Value</td><td
align="center">Value2</td><td align="right">Value3</td></tr>
<tr><td valign="top">Value4</td><td valign="middle">Value5</td><td
valign="bottom">Value6</td></tr>
<tr><td>Value7</td></tr></table>

<details><summary>Title</summary>Content</details>
<details open><summary>Title</summary>Content</details>
<tg-math-block>E = mc^2</tg-math-block>
<p>Inline buttons:
  <tg-button type="url" style="success" url="https://t.me">url</tg-button>
  <tg-button type="url" url="tg://user?id=777000">user</tg-button>
  <tg-button type="callback_data" style="link" data="callback">callback with the
```

```
date <tg-time unix="1647531900" format="wDT">22:45 tomorrow</tg-time> and the
custom emoji <tg-emoji emoji-id="5368324170671202286">👍</tg-emoji></tg-button>
  <tg-button type="web_app" style="danger" url="https://telegram.org">Mini App
(private chats only)</tg-button>
  <tg-button type="login_url" url="https://t.me" forward-text="forward text"
request-write-access>login (requires domain set up via @BotFather)</tg-button>
  <tg-button type="switch_inline_query" style="primary" query="inline">inline</tg-
button>
  <tg-button type="switch_inline_query_current_chat" query="inline 2">inline 2</tg-
button>
  <tg-button type="switch_inline_query_chosen_chat" query="inline 3" allow-user-
chats allow-bot-chats allow-group-chats allow-channel-chats>inline 3</tg-button>
  <tg-button type="copy_text" text="...copy">Copy</tg-button>
  <tg-button type="disabled">Disabled</tg-button>
</p>
<tg-button-row align="left">
  <tg-button type="url" url="https://t.me">url</tg-button>
  <tg-button type="url" style="success" url="tg://user?id=777000">user</tg-button>
  <tg-button type="callback_data" style="link" data="callback">callback</tg-button>
</tg-button-row>
<tg-button-row align="center">
  <tg-button type="web_app" url="https://telegram.org">Mini App (private chats
only)</tg-button>
</tg-button-row>
<tg-button-row align="center">
  <tg-button type="login_url" style="danger" url="https://t.me" forward-
text="forward text" request-write-access>login (requires domain set up via
@BotFather)</tg-button>
</tg-button-row>
<tg-button-row align="right">
  <tg-button type="switch_inline_query" query="inline">inline</tg-button>
  <tg-button type="switch_inline_query_current_chat" query="inline 2">inline 2</tg-
button>
  <tg-button type="switch_inline_query_chosen_chat" query="inline 3" allow-user-
chats allow-group-chats allow-channel-chats>inline 3</tg-button>
</tg-button-row>
<tg-button-row>
  <tg-button type="copy_text" text="...copy">Copy</tg-button>
  <tg-button type="disabled" style="primary">Disabled</tg-button>
</tg-button-row>
```

Additionally, you can use the following tag in sendRichMessageDraft:

```
<tg-thinking>Thinking...</tg-thinking>
```

Please note:

- Only the tags mentioned above are currently supported.
- All numerical HTML entities are supported.

- The API currently supports only the following named HTML entities: `<`, `>`, `&`, `"`, `'`, ` `, `…`, `—`, `–`, `‘`, `’`, `“` and `”`.
- Use nested `pre` and `code` tags to define the programming language for a pre-formatted block.
- Programming language can't be specified for standalone `code` tags.
- Links `mailto:...`, `tel:...`, and `tg://user?id=...` are rendered as e-mail links, phone links, and inline mentions respectively. Other supported links are rendered as regular inline links.
- Images, videos, and audio files can be specified only as separate media blocks.
- Media blocks support only HTTP and HTTPS URLs.
- An empty `<a name="..."></a>` on its own creates an anchor that can be linked to with `<a href="#...">...</a>`.
- In `<figcaption>`, you can use `<cite>` tags to specify caption credit.
- Use `<tg-reference name="...">...</tg-reference>` to define referenced text that can be linked to with `<a href="#...">...</a>`.
- The body of a `<details>` tag can contain rich message content. If the `open` attribute is specified, the block is expanded by default.
- Formula source is treated as raw LaTeX.
- See date-time entity formatting for more details about supported date-time formats.

RichMessage

Rich formatted message.

Field	Type	Description
blocks	Array of RichBlock	Content of the message
is_rtl	Boolean	*Optional*. *True*, if the rich message must be shown right-to-left

InputRichMessage

Describes a rich message to be sent. Exactly **one** of the fields *html*, *markdown*, or *blocks* must be used.

Field	Type	Description
blocks	Array of InputRichBlock	*Optional*. Content of the rich message to send described as a list of blocks
html	String	*Optional*. Content of the rich message to send described using HTML formatting. See rich message formatting options for more details. Use *media* field to specify the media used in the message.
markdown	String	*Optional*. Content of the rich message to send described using Markdown formatting. See rich message formatting options for more details. Use *media* field to specify the media used in the message.
media	Array of InputRichMessageMedia	*Optional*. List of media that are specified in the *markdown* or *html* fields using `tg://photo?id=`, `tg://video?id=`, `tg://document?id=`, and `tg://audio?id=` links

Field	Type	Description
is_rtl	Boolean	*Optional*. Pass *True* if the rich message must be shown right-to-left
skip_entity_detection	Boolean	*Optional*. Pass *True* to skip automatic detection of entities (e.g., URLs, email addresses, username mentions, hashtags, cashtags, bot commands, or phone numbers) in the text

InputRichMessageMedia

Describes a media element embedded in an outgoing rich message.

Field	Type	Description
id	String	Unique identifier of the media used in a `tg://photo?id=`, `tg://video?id=`, `tg://document?id=`, or `tg://audio?id=` link. 1-64 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed.
media	InputMediaAnimation or InputMediaAudio or InputMediaDocument or InputMediaPhoto or InputMediaVideo or InputMediaVoiceNote	The media to be sent. Everything except the media itself and its properties is ignored.

sendRichMessage

Use this method to send rich messages. If the message contains a block with a media element, then the bot must have the right to send the media to the chat. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent. Bot can send rich messages on behalf of a business account only if the corresponding user can send rich messages.
chat_id	Integer or String	Optional	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`; required unless the message is sent as an ephemeral reply to a callback query

Parameter	Type	Required	Description
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
ephemeral_message_parameters	EphemeralMessageParameters	Optional	A JSON-serialized object containing the parameters of the ephemeral message to send
rich_message	InputRichMessage	Yes	The message to be sent
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.

Parameter	Type	Required	Description
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup or ReplyKeyboardMarkup or ReplyKeyboardRemove or ForceReply	Optional	Additional interface options. A JSON-serialized object for an inline keyboard, custom reply keyboard, instructions to remove a reply keyboard or to force a reply from the user.

sendRichMessageDraft

Use this method to stream a partial rich message to a user while the message is being generated. Note that the streamed draft is ephemeral and acts as a temporary 30-second preview - once the output is finalized, you **must** call sendRichMessage with the complete message to persist it in the user's chat. Returns *True* on success.

Parameter	Type	Required	Description
chat_id	Integer	Yes	Unique identifier for the target private chat
message_thread_id	Integer	Optional	Unique identifier for the target message thread
draft_id	Integer	Yes	Unique identifier of the message draft; must be non-zero. Changes to drafts with the same identifier are animated. Otherwise, the draft is replaced without animation.
rich_message	InputRichMessage	Yes	The partial message to be streamed. Direct upload of new files and explicit upload of files by a URL isn't supported.
can_stop	Boolean	Optional	Pass *True* to show the user a button to stop further drafts. The bot will receive an Update “stopped_message_generation” if the user presses the button.
keep_on_stop	Boolean	Optional	Pass *True* to keep the draft in the chat when the button is pressed. The draft will still disappear after a short time or if the bot sends a message. To fully preserve the partial draft, the bot should send it as a new message.

RichMessageButton

This object represents a button in a RichMessage. Exactly one of the fields other than *text* and *style* must be used to specify the type of the button.

Field	Type	Description
text	RichText	Text of the button. May contain only plain text, RichTextCustomEmoji and RichTextDateTime entities.
style	String	*Optional*. Style of the button. Must be one of “danger”, “success”, “primary”, or “link” (the button is shown as a regular link without borders). Apps may use theme-specific colors for the button background and text based on the style. The style “link” is allowed only for callback buttons.
url	String	*Optional*. HTTP or tg:// URL to be opened when the button is pressed. Links `tg://user?id=<user_id>` can be used to mention a user by their identifier without using a username, if this is allowed by their privacy settings.
callback_data	String	*Optional*. Data to be sent in a callback query to the bot when the button is pressed, 1-64 bytes
web_app	WebAppInfo	*Optional*. Description of the Web App that will be launched when the user presses the button. The Web App will be able to send an arbitrary message on behalf of the user using the method answerWebAppQuery. Available only in private chats between a user and the bot. Not supported for messages sent on behalf of a business account.
login_url	LoginUrl	*Optional*. An HTTPS URL used to automatically authorize the user. Can be used as a replacement for the Telegram Login Widget. Not supported for ephemeral messages.
switch_inline_query	String	*Optional*. If set, pressing the button will prompt the user to select one of their chats, open that chat and insert the bot's username and the specified inline query in the input field. May be empty, in which case just the bot's username will be inserted. Not supported for messages sent in channel direct messages chats and on behalf of a business account.

Field	Type	Description
switch_inline_query_current_chat	String	*Optional*. If set, pressing the button will insert the bot's username and the specified inline query in the current chat's input field. May be empty, in which case only the bot's username will be inserted. Not supported in channels and for messages sent in channel direct messages chats and on behalf of a business account.
switch_inline_query_chosen_chat	SwitchInlineQueryChosenChat	*Optional*. If set, pressing the button will prompt the user to select one of their chats of the specified type, open that chat and insert the bot's username and the specified inline query in the input field. Not supported for messages sent in channel direct messages chats and on behalf of a business account.
copy_text	CopyTextButton	*Optional*. A button that copies the specified text to the clipboard
disabled	DisabledButton	*Optional*. If set, then the button is disabled and does nothing

RichText

This object represents a rich formatted text. Currently, it can be either a String for plain text, an Array of RichText, or any of the following types:

- RichTextBold
- RichTextItalic
- RichTextUnderline
- RichTextStrikethrough
- RichTextSpoiler
- RichTextDateTime
- RichTextTextMention
- RichTextSubscript
- RichTextSuperscript
- RichTextMarked
- RichTextCode
- RichTextCustomEmoji
- RichTextMathematicalExpression
- RichTextUrl
- RichTextEmailAddress
- RichTextPhoneNumber
- RichTextBankCardNumber
- RichTextTonAddress
- RichTextMention
- RichTextHashtag
- RichTextCashtag

- RichTextBotCommand
- RichTextButton
- RichTextAnchor
- RichTextAnchorLink
- RichTextReference
- RichTextReferenceLink

RichTextBold

A bold text.

Field	Type	Description
type	String	Type of the rich text, always “bold”
text	RichText	The text

RichTextItalic

An italicized text.

Field	Type	Description
type	String	Type of the rich text, always “italic”
text	RichText	The text

RichTextUnderline

An underlined text.

Field	Type	Description
type	String	Type of the rich text, always “underline”
text	RichText	The text

RichTextStrikethrough

A strikethrough text.

Field	Type	Description
type	String	Type of the rich text, always “strikethrough”
text	RichText	The text

RichTextSpoiler

A text covered by a spoiler.

Field	Type	Description
type	String	Type of the rich text, always “spoiler”
text	RichText	The text

RichTextDateTime

Formatted date and time.

Field	Type	Description
type	String	Type of the rich text, always “date_time”
text	RichText	The text
unix_time	Integer	The Unix time associated with the entity
date_time_format	String	The string that defines the formatting of the date and time. See date-time entity formatting for more details.

RichTextTextMention

A mention of a Telegram user by their identifier.

Field	Type	Description
type	String	Type of the rich text, always “text_mention”
text	RichText	The text
user	User	The mentioned user

RichTextSubscript

A subscript text.

Field	Type	Description
type	String	Type of the rich text, always “subscript”
text	RichText	The text

RichTextSuperscript

A superscript text.

Field	Type	Description
type	String	Type of the rich text, always “superscript”
text	RichText	The text

RichTextMarked

A marked text.

Field	Type	Description
type	String	Type of the rich text, always “marked”
text	RichText	The text

RichTextCode

A monowidth text.

Field	Type	Description
type	String	Type of the rich text, always “code”
text	RichText	The text

RichTextCustomEmoji

A custom emoji.

Field	Type	Description
type	String	Type of the rich text, always “custom_emoji”
custom_emoji_id	String	Unique identifier of the custom emoji. Use getCustomEmojiStickers to get full information about the sticker.
alternative_text	String	Alternative emoji for the custom emoji

RichTextMathematicalExpression

A mathematical expression.

Field	Type	Description
type	String	Type of the rich text, always “mathematical_expression”
expression	String	The expression in LaTeX format

RichTextUrl

A text with a link.

Field	Type	Description
type	String	Type of the rich text, always “url”
text	RichText	The text

Field	Type	Description
url	String	URL of the link

RichTextEmailAddress

A text with an email address.

Field	Type	Description
type	String	Type of the rich text, always “email_address”
text	RichText	The text
email_address	String	The email address

RichTextPhoneNumber

A text with a phone number.

Field	Type	Description
type	String	Type of the rich text, always “phone_number”
text	RichText	The text
phone_number	String	The phone number

RichTextBankCardNumber

A text with a bank card number.

Field	Type	Description
type	String	Type of the rich text, always “bank_card_number”
text	RichText	The text
bank_card_number	String	The bank card number

RichTextTonAddress

An address in the TON blockchain.

Field	Type	Description
type	String	Type of the rich text, always “ton_address”
text	RichText	Text of the address
address	String	The address

RichTextMention

A mention by a username.

Field	Type	Description
type	String	Type of the rich text, always “mention”
text	RichText	The text
username	String	The username

RichTextHashtag

A hashtag.

Field	Type	Description
type	String	Type of the rich text, always “hashtag”
text	RichText	The text
hashtag	String	The hashtag

RichTextCashtag

A cashtag.

Field	Type	Description
type	String	Type of the rich text, always “cashtag”
text	RichText	The text
cashtag	String	The cashtag

RichTextBotCommand

A bot command.

Field	Type	Description
type	String	Type of the rich text, always “bot_command”
text	RichText	The text
bot_command	String	The bot command

RichTextButton

A button.

Field	Type	Description
type	String	Type of the rich text, always “button”
button	RichMessageButton	The button

RichTextAnchor

An anchor.

Field	Type	Description
type	String	Type of the rich text, always “anchor”
name	String	The name of the anchor

RichTextAnchorLink

A link to an anchor.

Field	Type	Description
type	String	Type of the rich text, always “anchor_link”
text	RichText	The link text
anchor_name	String	The name of the anchor. If the name is empty, then the link brings back to the top of the message.

RichTextReference

A reference.

Field	Type	Description
type	String	Type of the rich text, always “reference”
text	RichText	Text of the reference
name	String	The name of the reference

RichTextReferenceLink

A link to a reference.

Field	Type	Description
type	String	Type of the rich text, always “reference_link”
text	RichText	The link text
reference_name	String	The name of the reference

RichBlockCaption

Caption of a rich formatted block.

Field	Type	Description
text	RichText	Block caption
credit	RichText	*Optional*. Block credit which corresponds to the HTML tag <cite>

RichBlockTableCell

Cell in a table.

Field	Type	Description
text	RichText	*Optional*. Text in the cell. If omitted, then the cell is invisible.
is_header	True	*Optional*. *True*, if the cell is a header cell
colspan	Integer	*Optional*. The number of columns the cell spans if it is bigger than 1
rowspan	Integer	*Optional*. The number of rows the cell spans if it is bigger than 1
align	String	Horizontal cell content alignment. Currently, must be one of “left”, “center”, or “right”.
valign	String	Vertical cell content alignment. Currently, must be one of “top”, “middle”, or “bottom”.

RichBlockListItem

An item of a list.

Field	Type	Description
label	String	Label of the item
blocks	Array of RichBlock	The content of the item
has_checkbox	True	*Optional*. *True*, if the item has a checkbox
is_checked	True	*Optional*. *True*, if the item has a checked checkbox
value	Integer	*Optional*. For ordered lists, the numeric value of the item label
type	String	*Optional*. For ordered lists, the type of the item label; must be one of “a” for lowercase letters, “A” for uppercase letters, “i” for lowercase Roman numerals, “I” for uppercase Roman numerals, or “1” for decimal numbers

RichBlock

This object represents a block in a rich formatted message. Currently, it can be any of the following types:

- RichBlockParagraph

- RichBlockSectionHeading
- RichBlockPreformatted
- RichBlockFooter
- RichBlockDivider
- RichBlockMathematicalExpression
- RichBlockAnchor
- RichBlockList
- RichBlockBlockQuotation
- RichBlockExpandableBlockQuotation
- RichBlockPullQuotation
- RichBlockCollage
- RichBlockSlideshow
- RichBlockTable
- RichBlockDetails
- RichBlockMap
- RichBlockButtons
- RichBlockAnimation
- RichBlockAudio
- RichBlockDocument
- RichBlockPhoto
- RichBlockVideo
- RichBlockVoiceNote
- RichBlockThinking

RichBlockParagraph

A text paragraph, corresponding to the HTML tag `<p>`.

Field	Type	Description
type	String	Type of the block, always “paragraph”
text	RichText	Text of the block

RichBlockSectionHeading

A section heading, corresponding to the HTML tags `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, or `<h6>`.

Field	Type	Description
type	String	Type of the block, always “heading”
text	RichText	Text of the block
size	Integer	Relative size of the text font; 1-6, 1 is the largest, 6 is the smallest

RichBlockPreformatted

A preformatted text block, corresponding to the nested HTML tags `<pre>` and `<code>`.

Field	Type	Description
type	String	Type of the block, always “pre”
text	RichText	Text of the block
language	String	*Optional*. The programming language of the text

RichBlockFooter

A footer, corresponding to the HTML tag `<footer>`.

Field	Type	Description
type	String	Type of the block, always “footer”
text	RichText	Text of the block

RichBlockDivider

A divider, corresponding to the HTML tag `<hr/>`.

Field	Type	Description
type	String	Type of the block, always “divider”

RichBlockMathematicalExpression

A block with a mathematical expression in LaTeX format, corresponding to the custom HTML tag `<tg-math-block>`.

Field	Type	Description
type	String	Type of the block, always “mathematical_expression”
expression	String	The mathematical expression in LaTeX format

RichBlockAnchor

A block with an anchor, corresponding to the HTML tag `<a>` with the attribute `name`.

Field	Type	Description
type	String	Type of the block, always “anchor”
name	String	The name of the anchor

RichBlockList

A list of blocks, corresponding to the HTML tag `<ul>` or `<ol>` with multiple nested tags `<li>`.

Field	Type	Description
type	String	Type of the block, always “list”
items	Array of RichBlockListItem	Items of the list

RichBlockBlockQuotation

A block quotation, corresponding to the HTML tag `<blockquote>`.

Field	Type	Description
type	String	Type of the block, always “blockquote”
blocks	Array of RichBlock	Content of the block
credit	RichText	*Optional*. Credit of the block

RichBlockExpandableBlockQuotation

A block quotation, corresponding to the HTML tag `<blockquote>` with custom attribute `"expandable"`.

Field	Type	Description
type	String	Type of the block, always “expandable_blockquote”
text	RichText	Content of the block
credit	RichText	*Optional*. Credit of the block

RichBlockPullQuotation

A quotation with centered text, loosely corresponding to the HTML tag `<aside>`.

Field	Type	Description
type	String	Type of the block, always “pullquote”
text	RichText	Text of the block
credit	RichText	*Optional*. Credit of the block

RichBlockCollage

A collage, corresponding to the custom HTML tag `<tg-collage>`.

Field	Type	Description
type	String	Type of the block, always “collage”
blocks	Array of RichBlock	Elements of the collage
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockSlideshow

A slideshow, corresponding to the custom HTML tag `<tg-slideshow>`.

Field	Type	Description
type	String	Type of the block, always “slideshow”
blocks	Array of RichBlock	Elements of the slideshow
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockTable

A table, corresponding to the HTML tag `<table>`.

Field	Type	Description
type	String	Type of the block, always “table”
cells	Array of Array of RichBlockTableCell	Cells of the table
is_bordered	True	*Optional*. *True*, if the table has borders
is_striped	True	*Optional*. *True*, if the table is striped
is_compact	True	*Optional*. *True*, if table cells have smaller indents
caption	RichText	*Optional*. Caption of the table

RichBlockDetails

An expandable block for details disclosure, corresponding to the HTML tag `<details>`.

Field	Type	Description
type	String	Type of the block, always “details”
summary	RichText	Always shown summary of the block
blocks	Array of RichBlock	Content of the block
is_open	True	*Optional*. *True*, if the content of the block is visible by default

RichBlockMap

A block with a map, corresponding to the custom HTML tag `<tg-map>`.

Field	Type	Description
type	String	Type of the block, always “map”
location	Location	Location of the center of the map
zoom	Integer	Map zoom level

Field	Type	Description
width	Integer	Expected width of the map
height	Integer	Expected height of the map
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockButtons

A block containing a list of buttons that are shown in one row, corresponding to the custom HTML tag `<tg-button-row>`.

Field	Type	Description
type	String	Type of the block, always “buttons”
buttons	Array of RichMessageButton	The buttons
align	String	*Optional*. Horizontal alignment of the buttons. Currently, must be one of “left”, “center”, or “right”.

RichBlockAnimation

A block with an animation, corresponding to the HTML tag `<video>`.

Field	Type	Description
type	String	Type of the block, always “animation”
animation	Animation	The animation
has_spoiler	True	*Optional*. *True*, if the media preview is covered by a spoiler animation
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockAudio

A block with a music file, corresponding to the HTML tag `<audio>`.

Field	Type	Description
type	String	Type of the block, always “audio”
audio	Audio	The audio
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockDocument

A block with a general file, corresponding to the custom HTML tag `<tg-document>`.

Field	Type	Description
type	String	Type of the block, always “document”
document	Document	The document
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockPhoto

A block with a photo, corresponding to the HTML tag `<img>`.

Field	Type	Description
type	String	Type of the block, always “photo”
photo	Array of PhotoSize	Available sizes of the photo
has_spoiler	True	*Optional*. *True*, if the media preview is covered by a spoiler animation
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockVideo

A block with a video, corresponding to the HTML tag `<video>`.

Field	Type	Description
type	String	Type of the block, always “video”
video	Video	The video
has_spoiler	True	*Optional*. *True*, if the media preview is covered by a spoiler animation
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockVoiceNote

A block with a voice note, corresponding to the HTML tag `<audio>`.

Field	Type	Description
type	String	Type of the block, always “voice_note”
voice_note	Voice	The voice note
caption	RichBlockCaption	*Optional*. Caption of the block

RichBlockThinking

A block with a “Thinking…” placeholder, corresponding to the custom HTML tag `<tg-thinking>`. The block may be used only in sendRichMessageDraft, therefore it can't be received in messages. See https://t.me/addemoji/AIActions for examples of custom emoji that are recommended for usage in the block.

Field	Type	Description
type	String	Type of the block, always “thinking”
text	RichText	Text of the block. See https://t.me/addemoji/AIActions for examples of custom emoji that are recommended for usage in the block.

InputRichBlockListItem

An item of a list to be sent.

Field	Type	Description
blocks	Array of InputRichBlock	The content of the item
has_checkbox	True	*Optional*. Pass *True* if the item has a checkbox
is_checked	True	*Optional*. Pass *True* if the item has a checked checkbox
value	Integer	*Optional*. For ordered lists, the numeric value of the item label
type	String	*Optional*. For ordered lists, the type of the item label; must be one of “a” for lowercase letters, “A” for uppercase letters, “i” for lowercase Roman numerals, “I” for uppercase Roman numerals, or “1” for decimal numbers

InputRichBlock

This object represents a block in a rich formatted message to be sent. Currently, it can be any of the following types:

- InputRichBlockParagraph
- InputRichBlockSectionHeading
- InputRichBlockPreformatted
- InputRichBlockFooter
- InputRichBlockDivider
- InputRichBlockMathematicalExpression
- InputRichBlockAnchor
- InputRichBlockList
- InputRichBlockBlockQuotation
- InputRichBlockExpandableBlockQuotation
- InputRichBlockPullQuotation
- InputRichBlockCollage
- InputRichBlockSlideshow
- InputRichBlockTable
- InputRichBlockDetails
- InputRichBlockMap
- InputRichBlockButtons
- InputRichBlockAnimation
- InputRichBlockAudio
- InputRichBlockDocument
- InputRichBlockPhoto

- InputRichBlockVideo
- InputRichBlockVoiceNote
- InputRichBlockThinking

InputRichBlockParagraph

A text paragraph, corresponding to the HTML tag `<p>`.

Field	Type	Description
type	String	Type of the block, always “paragraph”
text	RichText	Text of the block

InputRichBlockSectionHeading

A section heading, corresponding to the HTML tags `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, or `<h6>`.

Field	Type	Description
type	String	Type of the block, always “heading”
text	RichText	Text of the block
size	Integer	Relative size of the text font; 1-6, 1 is the largest, 6 is the smallest

InputRichBlockPreformatted

A preformatted text block, corresponding to the nested HTML tags `<pre>` and `<code>`.

Field	Type	Description
type	String	Type of the block, always “pre”
text	RichText	Text of the block
language	String	*Optional*. The programming language of the text

InputRichBlockFooter

A footer, corresponding to the HTML tag `<footer>`.

Field	Type	Description
type	String	Type of the block, always “footer”
text	RichText	Text of the block

InputRichBlockDivider

A divider, corresponding to the HTML tag `<hr/>`.

Field	Type	Description
type	String	Type of the block, always “divider”

InputRichBlockMathematicalExpression

A block with a mathematical expression in LaTeX format, corresponding to the custom HTML tag `<tg-math-block>`.

Field	Type	Description
type	String	Type of the block, always “mathematical_expression”
expression	String	The mathematical expression in LaTeX format

InputRichBlockAnchor

A block with an anchor, corresponding to the HTML tag `<a>` with the attribute `name`.

Field	Type	Description
type	String	Type of the block, always “anchor”
name	String	The name of the anchor

InputRichBlockList

A list of blocks, corresponding to the HTML tag `<ul>` or `<ol>` with multiple nested tags `<li>`.

Field	Type	Description
type	String	Type of the block, always “list”
items	Array of InputRichBlockListItem	Items of the list

InputRichBlockBlockQuotation

A block quotation, corresponding to the HTML tag `<blockquote>`.

Field	Type	Description
type	String	Type of the block, always “blockquote”
blocks	Array of InputRichBlock	Content of the block
credit	RichText	*Optional*. Credit of the block

InputRichBlockExpandableBlockQuotation

A block quotation, corresponding to the HTML tag `<blockquote>` with custom attribute `"expandable"`.

Field	Type	Description
type	String	Type of the block, always “expandable_blockquote”
text	RichText	Content of the block
credit	RichText	*Optional*. Credit of the block

InputRichBlockPullQuotation

A quotation with centered text, loosely corresponding to the HTML tag `<aside>`.

Field	Type	Description
type	String	Type of the block, always “pullquote”
text	RichText	Text of the block
credit	RichText	*Optional*. Credit of the block

InputRichBlockCollage

A collage, corresponding to the custom HTML tag `<tg-collage>`.

Field	Type	Description
type	String	Type of the block, always “collage”
blocks	Array of InputRichBlock	Elements of the collage
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockSlideshow

A slideshow, corresponding to the custom HTML tag `<tg-slideshow>`.

Field	Type	Description
type	String	Type of the block, always “slideshow”
blocks	Array of InputRichBlock	Elements of the slideshow
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockTable

A table, corresponding to the HTML tag `<table>`.

Field	Type	Description
type	String	Type of the block, always “table”
cells	Array of Array of RichBlockTableCell	Cells of the table

Field	Type	Description
is_bordered	True	*Optional*. Pass *True* if the table has borders
is_striped	True	*Optional*. Pass *True* if the table is striped
is_compact	True	*Optional*. Pass *True* if table cells must have smaller indents
caption	RichText	*Optional*. Caption of the table

InputRichBlockDetails

An expandable block for details disclosure, corresponding to the HTML tag `<details>`.

Field	Type	Description
type	String	Type of the block, always “details”
summary	RichText	Always shown summary of the block
blocks	Array of InputRichBlock	Content of the block
is_open	True	*Optional*. Pass *True* if the content of the block is visible by default

InputRichBlockMap

A block with a map, corresponding to the custom HTML tag `<tg-map>`. The map's width and height must not exceed 10000 in total. The width and height ratio must be at most 20.

Field	Type	Description
type	String	Type of the block, always “map”
location	Location	Location of the center of the map
zoom	Integer	*Optional*. Map zoom level; 0-24
width	Integer	*Optional*. Map width; 0-10000
height	Integer	*Optional*. Map height; 0-10000
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockButtons

A block containing a list of buttons that are shown in one row, corresponding to the custom HTML tag `<tg-button-row>`.

Field	Type	Description
type	String	Type of the block, always “buttons”
buttons	Array of RichMessageButton	List of 1-8 buttons to send

Field	Type	Description
align	String	*Optional*. Horizontal alignment of the buttons. Currently, must be one of “left”, “center”, or “right”.

InputRichBlockAnimation

A block with an animation, corresponding to the HTML tag `<video>`.

Field	Type	Description
type	String	Type of the block, always “animation”
animation	InputMediaAnimation	The animation. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockAudio

A block with a music file, corresponding to the HTML tag `<audio>`.

Field	Type	Description
type	String	Type of the block, always “audio”
audio	InputMediaAudio	The audio. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockDocument

A block with a general file, corresponding to the custom HTML tag `<tg-document>`.

Field	Type	Description
type	String	Type of the block, always “document”
document	InputMediaDocument	The document. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockPhoto

A block with a photo, corresponding to the HTML tag `<img>`.

Field	Type	Description
type	String	Type of the block, always “photo”
photo	InputMediaPhoto	The photo. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockVideo

A block with a video, corresponding to the HTML tag `<video>`.

Field	Type	Description
type	String	Type of the block, always “video”
video	InputMediaVideo	The video. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockVoiceNote

A block with a voice note, corresponding to the HTML tag `<audio>`.

Field	Type	Description
type	String	Type of the block, always “voice_note”
voice_note	InputMediaVoiceNote	The voice note. Caption is ignored.
caption	RichBlockCaption	*Optional*. Caption of the block

InputRichBlockThinking

A block with a “Thinking…” placeholder, corresponding to the custom HTML tag `<tg-thinking>`. The block may be used only in sendRichMessageDraft, therefore it can't be received in messages. See https://t.me/addemoji/AIActions for examples of custom emoji that are recommended for usage in the block.

Field	Type	Description
type	String	Type of the block, always “thinking”
text	RichText	Text of the block. See https://t.me/addemoji/AIActions for examples of custom emoji that are recommended for usage in the block.

Inline mode

The following methods and objects allow your bot to work in inline mode.
Please see our Introduction to Inline bots for more details.

To enable this option, send the `/setinline` command to @BotFather and provide the placeholder text that the user will see in the input field after typing your bot's name.

InlineQuery

This object represents an incoming inline query. When the user sends an empty query, your bot could return some default or trending results.

Field	Type	Description
id	String	Unique identifier for this query
from	User	Sender
query	String	Text of the query (up to 256 characters)
offset	String	Offset of the results to be returned, can be controlled by the bot
chat_type	String	*Optional*. Type of the chat from which the inline query was sent. Can be either “sender” for a private chat with the inline query sender, “private”, “group”, “supergroup”, or “channel”. The chat type should be always known for requests sent from official clients and most third-party clients, unless the request was sent from a secret chat.
location	Location	*Optional*. Sender location, only for bots that request user location

answerInlineQuery

Use this method to send answers to an inline query. On success, *True* is returned.
No more than **50** results per query are allowed.

Parameter	Type	Required	Description
inline_query_id	String	Yes	Unique identifier for the answered query
results	Array of InlineQueryResult	Yes	A JSON-serialized Array of results for the inline query
cache_time	Integer	Optional	The maximum amount of time in seconds that the result of the inline query may be cached on the server. Defaults to 300.
is_personal	Boolean	Optional	Pass *True* if results may be cached on the server side only for the user that sent the query. By default, results may be returned to any user who sends the same query.
next_offset	String	Optional	Pass the offset that a client should send in the next query with the same text to receive more results. Pass an empty string if there are no more results or if you don't support pagination. Offset length can't exceed 64 bytes.
button	InlineQueryResultsButton	Optional	A JSON-serialized object describing a button to be shown above inline query results

InlineQueryResultsButton

This object represents a button to be shown above inline query results. You **must** use exactly one of the optional fields.

Field	Type	Description
text	String	Label text on the button

Field	Type	Description
web_app	WebAppInfo	*Optional*. Description of the Web App that will be launched when the user presses the button. The Web App will be able to switch back to the inline mode using the method switchInlineQuery inside the Web App.
start_parameter	String	*Optional*. Deep-linking parameter for the /start message sent to the bot when a user presses the button. 1-64 characters, only `A-Z`, `a-z`, `0-9`, `_` and `-` are allowed. *Example:* An inline bot that sends YouTube videos can ask the user to connect the bot to their YouTube account to adapt search results accordingly. To do this, it displays a 'Connect your YouTube account' button above the results, or even before showing any. The user presses the button, switches to a private chat with the bot and, in doing so, passes a start parameter that instructs the bot to return an OAuth link. Once done, the bot can offer a *switch_inline* button so that the user can easily return to the chat where they wanted to use the bot's inline capabilities.

InlineQueryResult

This object represents one result of an inline query. Telegram clients currently support results of the following 20 types:

- InlineQueryResultCachedAudio
- InlineQueryResultCachedDocument
- InlineQueryResultCachedGif
- InlineQueryResultCachedMpeg4Gif
- InlineQueryResultCachedPhoto
- InlineQueryResultCachedSticker
- InlineQueryResultCachedVideo
- InlineQueryResultCachedVoice
- InlineQueryResultArticle
- InlineQueryResultAudio
- InlineQueryResultContact
- InlineQueryResultGame
- InlineQueryResultDocument
- InlineQueryResultGif
- InlineQueryResultLocation
- InlineQueryResultMpeg4Gif
- InlineQueryResultPhoto
- InlineQueryResultVenue
- InlineQueryResultVideo
- InlineQueryResultVoice

Note: All URLs passed in inline query results will be available to end users and therefore must be assumed to be **public**.

InlineQueryResultArticle

Represents a link to an article or web page.

Field	Type	Description
type	String	Type of the result, must be *article*
id	String	Unique identifier for this result, 1-64 Bytes
title	String	Title of the result
input_message_content	InputMessageContent	Content of the message to be sent
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
url	String	*Optional*. URL of the result
description	String	*Optional*. Short description of the result
thumbnail_url	String	*Optional*. Url of the thumbnail for the result
thumbnail_width	Integer	*Optional*. Thumbnail width
thumbnail_height	Integer	*Optional*. Thumbnail height

InlineQueryResultPhoto

Represents a link to a photo. By default, this photo will be sent by the user with optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the photo.

Field	Type	Description
type	String	Type of the result, must be *photo*
id	String	Unique identifier for this result, 1-64 bytes
photo_url	String	A valid URL of the photo. Photo must be in **JPEG** format. Photo size must not exceed 5MB.
thumbnail_url	String	URL of the thumbnail for the photo
photo_width	Integer	*Optional*. Width of the photo
photo_height	Integer	*Optional*. Height of the photo
title	String	*Optional*. Title for the result
description	String	*Optional*. Short description of the result
caption	String	*Optional*. Caption of the photo to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the photo caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*

Field	Type	Description
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the photo

InlineQueryResultGif

Represents a link to an animated GIF file. By default, this animated GIF file will be sent by the user with optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the animation.

Field	Type	Description
type	String	Type of the result, must be *gif*
id	String	Unique identifier for this result, 1-64 bytes
gif_url	String	A valid URL for the GIF file
gif_width	Integer	*Optional*. Width of the GIF
gif_height	Integer	*Optional*. Height of the GIF
gif_duration	Integer	*Optional*. Duration of the GIF in seconds
thumbnail_url	String	URL of the static (JPEG or GIF) or animated (MPEG4) thumbnail for the result
thumbnail_mime_type	String	*Optional*. MIME type of the thumbnail, must be one of “image/jpeg”, “image/gif”, or “video/mp4”. Defaults to “image/jpeg”.
title	String	*Optional*. Title for the result
caption	String	*Optional*. Caption of the GIF file to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the GIF animation

InlineQueryResultMpeg4Gif

Represents a link to a video animation (H.264/MPEG-4 AVC video without sound). By default, this animated MPEG-4 file will be sent by the user with optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the animation.

Field	Type	Description
type	String	Type of the result, must be *mpeg4_gif*
id	String	Unique identifier for this result, 1-64 bytes
mpeg4_url	String	A valid URL for the MPEG4 file
mpeg4_width	Integer	*Optional*. Video width
mpeg4_height	Integer	*Optional*. Video height
mpeg4_duration	Integer	*Optional*. Video duration in seconds
thumbnail_url	String	URL of the static (JPEG or GIF) or animated (MPEG4) thumbnail for the result
thumbnail_mime_type	String	*Optional*. MIME type of the thumbnail, must be one of “image/jpeg”, “image/gif”, or “video/mp4”. Defaults to “image/jpeg”.
title	String	*Optional*. Title for the result
caption	String	*Optional*. Caption of the MPEG-4 file to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the video animation

InlineQueryResultVideo

Represents a link to a page containing an embedded video player or a video file. By default, this video file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the video.

> If an InlineQueryResultVideo message contains an embedded video (e.g., YouTube), you **must** replace its content using *input_message_content*.

Field	Type	Description
type	String	Type of the result, must be *video*
id	String	Unique identifier for this result, 1-64 bytes
video_url	String	A valid URL for the embedded video player or video file
mime_type	String	MIME type of the content of the video URL, “text/html” or “video/mp4”
thumbnail_url	String	URL of the thumbnail (JPEG only) for the video
title	String	Title for the result
caption	String	*Optional*. Caption of the video to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the video caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
video_width	Integer	*Optional*. Video width
video_height	Integer	*Optional*. Video height
video_duration	Integer	*Optional*. Video duration in seconds
description	String	*Optional*. Short description of the result
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the video. This field is **required** if InlineQueryResultVideo is used to send an HTML-page as a result (e.g., a YouTube video).

InlineQueryResultAudio

Represents a link to an MP3 audio file. By default, this audio file will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the audio.

Field	Type	Description
type	String	Type of the result, must be *audio*
id	String	Unique identifier for this result, 1-64 bytes
audio_url	String	A valid URL for the audio file
title	String	Title

Field	Type	Description
caption	String	*Optional*. Caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the audio caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
performer	String	*Optional*. Performer
audio_duration	Integer	*Optional*. Audio duration in seconds
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the audio

InlineQueryResultVoice

Represents a link to a voice recording in an .OGG container encoded with OPUS. By default, this voice recording will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the the voice message.

Field	Type	Description
type	String	Type of the result, must be *voice*
id	String	Unique identifier for this result, 1-64 bytes
voice_url	String	A valid URL for the voice recording
title	String	Recording title
caption	String	*Optional*. Caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the voice message caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
voice_duration	Integer	*Optional*. Recording duration in seconds
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the voice recording

InlineQueryResultDocument

Represents a link to a file. By default, this file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the file. Currently, only **.PDF** and **.ZIP** files can be sent using this method.

Field	Type	Description
type	String	Type of the result, must be *document*
id	String	Unique identifier for this result, 1-64 bytes
title	String	Title for the result
caption	String	*Optional*. Caption of the document to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the document caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
document_url	String	A valid URL for the file
mime_type	String	MIME type of the content of the file, either “application/pdf” or “application/zip”
description	String	*Optional*. Short description of the result
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the file
thumbnail_url	String	*Optional*. URL of the thumbnail (JPEG only) for the file
thumbnail_width	Integer	*Optional*. Thumbnail width
thumbnail_height	Integer	*Optional*. Thumbnail height

InlineQueryResultLocation

Represents a location on a map. By default, the location will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the location.

Field	Type	Description
type	String	Type of the result, must be *location*
id	String	Unique identifier for this result, 1-64 Bytes
latitude	Float	Location latitude in degrees
longitude	Float	Location longitude in degrees
title	String	Location title
horizontal_accuracy	Float	*Optional*. The radius of uncertainty for the location, measured in meters; 0-1500
live_period	Integer	*Optional*. Period in seconds during which the location can be updated, must be between 60 and 86400, or 0x7FFFFFFF for live locations that can be edited indefinitely

Field	Type	Description
heading	Integer	*Optional*. For live locations, a direction in which the user is moving, in degrees. Must be between 1 and 360 if specified.
proximity_alert_radius	Integer	*Optional*. For live locations, a maximum distance for proximity alerts about approaching another chat member, in meters. Must be between 1 and 100000 if specified.
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the location
thumbnail_url	String	*Optional*. Url of the thumbnail for the result
thumbnail_width	Integer	*Optional*. Thumbnail width
thumbnail_height	Integer	*Optional*. Thumbnail height

InlineQueryResultVenue

Represents a venue. By default, the venue will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the venue.

Field	Type	Description
type	String	Type of the result, must be *venue*
id	String	Unique identifier for this result, 1-64 Bytes
latitude	Float	Latitude of the venue location in degrees
longitude	Float	Longitude of the venue location in degrees
title	String	Title of the venue
address	String	Address of the venue
foursquare_id	String	*Optional*. Foursquare identifier of the venue if known
foursquare_type	String	*Optional*. Foursquare type of the venue, if known. (For example, “arts_entertainment/default”, “arts_entertainment/aquarium” or “food/icecream”.)
google_place_id	String	*Optional*. Google Places identifier of the venue
google_place_type	String	*Optional*. Google Places type of the venue. (See supported types.)
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the venue
thumbnail_url	String	*Optional*. Url of the thumbnail for the result

Field	Type	Description
thumbnail_width	Integer	*Optional*. Thumbnail width
thumbnail_height	Integer	*Optional*. Thumbnail height

InlineQueryResultContact

Represents a contact with a phone number. By default, this contact will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the contact.

Field	Type	Description
type	String	Type of the result, must be *contact*
id	String	Unique identifier for this result, 1-64 Bytes
phone_number	String	Contact's phone number
first_name	String	Contact's first name
last_name	String	*Optional*. Contact's last name
vcard	String	*Optional*. Additional data about the contact in the form of a vCard, 0-2048 bytes
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the contact
thumbnail_url	String	*Optional*. Url of the thumbnail for the result
thumbnail_width	Integer	*Optional*. Thumbnail width
thumbnail_height	Integer	*Optional*. Thumbnail height

InlineQueryResultGame

Represents a Game.

Field	Type	Description
type	String	Type of the result, must be *game*
id	String	Unique identifier for this result, 1-64 bytes
game_short_name	String	Short name of the game
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message

InlineQueryResultCachedPhoto

Represents a link to a photo stored on the Telegram servers. By default, this photo will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the photo.

Field	Type	Description
type	String	Type of the result, must be *photo*
id	String	Unique identifier for this result, 1-64 bytes
photo_file_id	String	A valid file identifier of the photo
title	String	*Optional*. Title for the result
description	String	*Optional*. Short description of the result
caption	String	*Optional*. Caption of the photo to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the photo caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the photo

InlineQueryResultCachedGif

Represents a link to an animated GIF file stored on the Telegram servers. By default, this animated GIF file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with specified content instead of the animation.

Field	Type	Description
type	String	Type of the result, must be *gif*
id	String	Unique identifier for this result, 1-64 bytes
gif_file_id	String	A valid file identifier for the GIF file
title	String	*Optional*. Title for the result
caption	String	*Optional*. Caption of the GIF file to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the caption. See formatting options for more details.

Field	Type	Description
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the GIF animation

InlineQueryResultCachedMpeg4Gif

Represents a link to a video animation (H.264/MPEG-4 AVC video without sound) stored on the Telegram servers. By default, this animated MPEG-4 file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the animation.

Field	Type	Description
type	String	Type of the result, must be *mpeg4_gif*
id	String	Unique identifier for this result, 1-64 bytes
mpeg4_file_id	String	A valid file identifier for the MPEG4 file
title	String	*Optional*. Title for the result
caption	String	*Optional*. Caption of the MPEG-4 file to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the video animation

InlineQueryResultCachedSticker

Represents a link to a sticker stored on the Telegram servers. By default, this sticker will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the sticker.

Field	Type	Description
type	String	Type of the result, must be *sticker*
id	String	Unique identifier for this result, 1-64 bytes
sticker_file_id	String	A valid file identifier of the sticker
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the sticker

InlineQueryResultCachedDocument

Represents a link to a file stored on the Telegram servers. By default, this file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the file.

Field	Type	Description
type	String	Type of the result, must be *document*
id	String	Unique identifier for this result, 1-64 bytes
title	String	Title for the result
document_file_id	String	A valid file identifier for the file
description	String	*Optional*. Short description of the result
caption	String	*Optional*. Caption of the document to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the document caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the file

InlineQueryResultCachedVideo

Represents a link to a video file stored on the Telegram servers. By default, this video file will be sent by the user with an optional caption. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the video.

Field	Type	Description
type	String	Type of the result, must be *video*
id	String	Unique identifier for this result, 1-64 bytes

Field	Type	Description
video_file_id	String	A valid file identifier for the video file
title	String	Title for the result
description	String	*Optional*. Short description of the result
caption	String	*Optional*. Caption of the video to be sent, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the video caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the message media
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the video

InlineQueryResultCachedVoice

Represents a link to a voice message stored on the Telegram servers. By default, this voice message will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the voice message.

Field	Type	Description
type	String	Type of the result, must be *voice*
id	String	Unique identifier for this result, 1-64 bytes
voice_file_id	String	A valid file identifier for the voice message
title	String	Voice message title
caption	String	*Optional*. Caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the voice message caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the voice message

InlineQueryResultCachedAudio

Represents a link to an MP3 audio file stored on the Telegram servers. By default, this audio file will be sent by the user. Alternatively, you can use *input_message_content* to send a message with the specified content instead of the audio.

Field	Type	Description
type	String	Type of the result, must be *audio*
id	String	Unique identifier for this result, 1-64 bytes
audio_file_id	String	A valid file identifier for the audio file
caption	String	*Optional*. Caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the audio caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
reply_markup	InlineKeyboardMarkup	*Optional*. Inline keyboard attached to the message
input_message_content	InputMessageContent	*Optional*. Content of the message to be sent instead of the audio

InputMessageContent

This object represents the content of a message to be sent as a result of an inline query. Telegram clients currently support the following types:

- InputTextMessageContent
- InputRichMessageContent
- InputLocationMessageContent
- InputVenueMessageContent
- InputContactMessageContent
- InputInvoiceMessageContent
- InputWidgetMessageContent
- InputSharedWidgetMessageContent

InputTextMessageContent

Represents the content of a text message to be sent as the result of an inline query.

Field	Type	Description
message_text	String	Text of the message to be sent, 1-4096 characters
parse_mode	String	*Optional*. Mode for parsing entities in the message text. See formatting options for more details.
entities	Array of MessageEntity	*Optional*. List of special entities that appear in message text, which can be specified instead of *parse_mode*

Field	Type	Description
link_preview_options	LinkPreviewOptions	*Optional*. Link preview generation options for the message

InputRichMessageContent

Represents the content of a rich message to be sent as the result of an inline query.

Field	Type	Description
rich_message	InputRichMessage	The message to be sent. Only previously uploaded files may be used in the message.

InputLocationMessageContent

Represents the content of a location message to be sent as the result of an inline query.

Field	Type	Description
latitude	Float	Latitude of the location in degrees
longitude	Float	Longitude of the location in degrees
horizontal_accuracy	Float	*Optional*. The radius of uncertainty for the location, measured in meters; 0-1500
live_period	Integer	*Optional*. Period in seconds during which the location can be updated, must be between 60 and 86400, or 0x7FFFFFFF for live locations that can be edited indefinitely
heading	Integer	*Optional*. For live locations, a direction in which the user is moving, in degrees. Must be between 1 and 360 if specified.
proximity_alert_radius	Integer	*Optional*. For live locations, a maximum distance for proximity alerts about approaching another chat member, in meters. Must be between 1 and 100000 if specified.

InputVenueMessageContent

Represents the content of a venue message to be sent as the result of an inline query.

Field	Type	Description
latitude	Float	Latitude of the venue in degrees
longitude	Float	Longitude of the venue in degrees
title	String	Name of the venue
address	String	Address of the venue
foursquare_id	String	*Optional*. Foursquare identifier of the venue, if known
foursquare_type	String	*Optional*. Foursquare type of the venue, if known. (For example, “arts_entertainment/default”, “arts_entertainment/aquarium” or “food/icecream”.)

Field	Type	Description
google_place_id	String	*Optional*. Google Places identifier of the venue
google_place_type	String	*Optional*. Google Places type of the venue. (See supported types.)

InputContactMessageContent

Represents the content of a contact message to be sent as the result of an inline query.

Field	Type	Description
phone_number	String	Contact's phone number
first_name	String	Contact's first name
last_name	String	*Optional*. Contact's last name
vcard	String	*Optional*. Additional data about the contact in the form of a vCard, 0-2048 bytes

InputInvoiceMessageContent

Represents the content of an invoice message to be sent as the result of an inline query.

Field	Type	Description
title	String	Product name, 1-32 characters
description	String	Product description, 1-255 characters
payload	String	Bot-defined invoice payload, 1-128 bytes. This will not be displayed to the user, use it for your internal processes.
provider_token	String	*Optional*. Payment provider token, obtained via @BotFather. Pass an empty string for payments in Telegram Stars.
currency	String	Three-letter ISO 4217 currency code, see more on currencies. Pass “XTR” for payments in Telegram Stars.
prices	Array of LabeledPrice	Price breakdown, a JSON-serialized list of components (e.g. product price, tax, discount, delivery cost, delivery tax, bonus, etc.). Must contain exactly one item for payments in Telegram Stars.
max_tip_amount	Integer	*Optional*. The maximum accepted amount for tips in the *smallest units* of the currency (integer, **not** float/double). For example, for a maximum tip of `US$ 1.45` pass `max_tip_amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies). Defaults to 0. Not supported for payments in Telegram Stars.

Field	Type	Description
suggested_tip_amounts	Array of Integer	*Optional*. A JSON-serialized Array of suggested amounts of tip in the *smallest units* of the currency (integer, **not** float/double). At most 4 suggested tip amounts can be specified. The suggested tip amounts must be positive, passed in a strictly increased order and must not exceed *max_tip_amount*.
provider_data	String	*Optional*. A JSON-serialized object for data about the invoice, which will be shared with the payment provider. A detailed description of the required fields should be provided by the payment provider.
photo_url	String	*Optional*. URL of the product photo for the invoice. Can be a photo of the goods or a marketing image for a service.
photo_size	Integer	*Optional*. Photo size in bytes
photo_width	Integer	*Optional*. Photo width
photo_height	Integer	*Optional*. Photo height
need_name	Boolean	*Optional*. Pass *True* if you require the user's full name to complete the order. Ignored for payments in Telegram Stars.
need_phone_number	Boolean	*Optional*. Pass *True* if you require the user's phone number to complete the order. Ignored for payments in Telegram Stars.
need_email	Boolean	*Optional*. Pass *True* if you require the user's email address to complete the order. Ignored for payments in Telegram Stars.
need_shipping_address	Boolean	*Optional*. Pass *True* if you require the user's shipping address to complete the order. Ignored for payments in Telegram Stars.
send_phone_number_to_provider	Boolean	*Optional*. Pass *True* if the user's phone number should be sent to the provider. Ignored for payments in Telegram Stars.
send_email_to_provider	Boolean	*Optional*. Pass *True* if the user's email address should be sent to the provider. Ignored for payments in Telegram Stars.
is_flexible	Boolean	*Optional*. Pass *True* if the final price depends on the shipping method. Ignored for payments in Telegram Stars.

InputWidgetMessageContent

Represents the content of a message with an HTML widget to be sent as the result of an inline query. Currently, widgets can't be sent in response to inline queries in the production environment.

Field	Type	Description
html	String	HTML code of the widget, 1-32768 bytes. The code must not exceed 4096 bytes when compressed by zlib with the maximum compression level.

Field	Type	Description
width	Integer	*Optional*. Width of the widget in logical pixels; omit if the widget must occupy the whole width of the message bubble. The width is clamped by the app to the range from 200 to the maximum width of the message bubble.
height	Integer	*Optional*. Height of the widget in logical pixels; omit if the height of the widget isn't fixed. The height is clamped by the app to the range 50-600.
public_data	String	*Optional*. Public data of the widget, 0-1024 bytes, which will be available wherever the widget is shown. If the widget is shared, then all messages with it will have the same public data.
chat_data	String	*Optional*. Data of the widget, which is specific to the message, 0-1024 bytes
caption	String	*Optional*. Widget caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the widget caption. See formatting options for more details.
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the widget
allow_queries	Boolean	*Optional*. Pass *True* to allow the widget to send queries to the bot
is_shared	Boolean	*Optional*. Pass *True* to create a shared widget. Each message sent from the result creates a separate shared widget, whose identifier is received by the bot only in the field *widget_id* of queries from the widget. To send an existing shared widget in response to an inline query, use InputSharedWidgetMessageContent instead.

InputSharedWidgetMessageContent

Represents the content of a message with a shared widget to be sent as the result of an inline query. Currently, widgets can't be sent in response to inline queries in the production environment.

Field	Type	Description
widget_id	String	Unique identifier of the shared widget created by the bot
chat_data	String	*Optional*. Data of the widget, which is specific to the message, 0-1024 bytes
caption	String	*Optional*. Widget caption, 0-1024 characters after entities parsing
parse_mode	String	*Optional*. Mode for parsing entities in the widget caption. See formatting options for more details.

Field	Type	Description
caption_entities	Array of MessageEntity	*Optional*. List of special entities that appear in the caption, which can be specified instead of *parse_mode*
show_caption_above_media	Boolean	*Optional*. Pass *True* if the caption must be shown above the widget

ChosenInlineResult

Represents a result of an inline query that was chosen by the user and sent to their chat partner.

Field	Type	Description
result_id	String	The unique identifier for the result that was chosen
from	User	The user that chose the result
location	Location	*Optional*. Sender location, only for bots that require user location
inline_message_id	String	*Optional*. Identifier of the sent inline message. Available only if there is an inline keyboard attached to the message. Will be also received in callback queries and can be used to edit the message.
query	String	The query that was used to obtain the result

Note: It is necessary to enable inline feedback via @BotFather in order to receive these objects in updates.

Payments

Your bot can accept payments from Telegram users. Please see the introduction to payments for more details on the process and how to set up payments for your bot.

sendInvoice

Use this method to send invoices. On success, the sent Message is returned.

Parameter	Type	Required	Description
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot, supergroup, or channel in the format `@username`
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only

Parameter	Type	Required	Description
direct_messages_topic_id	Integer	Optional	Identifier of the direct messages topic to which the message will be sent; required if the message is sent to a direct messages chat
title	String	Yes	Product name, 1-32 characters
description	String	Yes	Product description, 1-255 characters
payload	String	Yes	Bot-defined invoice payload, 1-128 bytes. This will not be displayed to the user, use it for your internal processes.
provider_token	String	Optional	Payment provider token, obtained via @BotFather. Pass an empty string for payments in Telegram Stars.
currency	String	Yes	Three-letter ISO 4217 currency code, see more on currencies. Pass “XTR” for payments in Telegram Stars.
prices	Array of LabeledPrice	Yes	Price breakdown, a JSON-serialized list of components (e.g. product price, tax, discount, delivery cost, delivery tax, bonus, etc.). Must contain exactly one item for payments in Telegram Stars.
max_tip_amount	Integer	Optional	The maximum accepted amount for tips in the *smallest units* of the currency (integer, **not** float/double). For example, for a maximum tip of `US$ 1.45` pass `max_tip_amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies). Defaults to 0. Not supported for payments in Telegram Stars.
suggested_tip_amounts	Array of Integer	Optional	A JSON-serialized Array of suggested amounts of tips in the *smallest units* of the currency (integer, **not** float/double). At most 4 suggested tip amounts can be specified. The suggested tip amounts must be positive, passed in a strictly increased order and must not exceed *max_tip_amount*.

Parameter	Type	Required	Description
start_parameter	String	Optional	Unique deep-linking parameter. If left empty, **forwarded copies** of the sent message will have a *Pay* button, allowing multiple users to pay directly from the forwarded message, using the same invoice. If non-empty, forwarded copies of the sent message will have a *URL* button with a deep link to the bot (instead of a *Pay* button), with the value used as the start parameter.
provider_data	String	Optional	JSON-serialized data about the invoice, which will be shared with the payment provider. A detailed description of required fields should be provided by the payment provider.
photo_url	String	Optional	URL of the product photo for the invoice. Can be a photo of the goods or a marketing image for a service. People like it better when they see what they are paying for.
photo_size	Integer	Optional	Photo size in bytes
photo_width	Integer	Optional	Photo width
photo_height	Integer	Optional	Photo height
need_name	Boolean	Optional	Pass *True* if you require the user's full name to complete the order. Ignored for payments in Telegram Stars.
need_phone_number	Boolean	Optional	Pass *True* if you require the user's phone number to complete the order. Ignored for payments in Telegram Stars.
need_email	Boolean	Optional	Pass *True* if you require the user's email address to complete the order. Ignored for payments in Telegram Stars.
need_shipping_address	Boolean	Optional	Pass *True* if you require the user's shipping address to complete the order. Ignored for payments in Telegram Stars.
send_phone_number_to_provider	Boolean	Optional	Pass *True* if the user's phone number should be sent to the provider. Ignored for payments in Telegram Stars.

Parameter	Type	Required	Description
send_email_to_provider	Boolean	Optional	Pass *True* if the user's email address should be sent to the provider. Ignored for payments in Telegram Stars.
is_flexible	Boolean	Optional	Pass *True* if the final price depends on the shipping method. Ignored for payments in Telegram Stars.
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
suggested_post_parameters	SuggestedPostParameters	Optional	A JSON-serialized object containing the parameters of the suggested post to send; for direct messages chats only. If the message is sent as a reply to another suggested post, then that suggested post is automatically declined.
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard. If empty, one 'Pay `total price`' button will be shown. If not empty, the first button must be a Pay button.

createInvoiceLink

Use this method to create a link for an invoice. Returns the created invoice link as *String* on success.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the link will be created. For payments in Telegram Stars only.

Parameter	Type	Required	Description
title	String	Yes	Product name, 1-32 characters
description	String	Yes	Product description, 1-255 characters
payload	String	Yes	Bot-defined invoice payload, 1-128 bytes. This will not be displayed to the user, use it for your internal processes.
provider_token	String	Optional	Payment provider token, obtained via @BotFather. Pass an empty string for payments in Telegram Stars.
currency	String	Yes	Three-letter ISO 4217 currency code, see more on currencies. Pass “XTR” for payments in Telegram Stars.
prices	Array of LabeledPrice	Yes	Price breakdown, a JSON-serialized list of components (e.g. product price, tax, discount, delivery cost, delivery tax, bonus, etc.). Must contain exactly one item for payments in Telegram Stars.
subscription_period	Integer	Optional	The number of seconds the subscription will be active for before the next payment. The currency must be set to “XTR” (Telegram Stars) if the parameter is used. Currently, it must always be 2592000 (30 days) if specified. Any number of subscriptions can be active for a given bot at the same time, including multiple concurrent subscriptions from the same user. Subscription price must no exceed 10000 Telegram Stars.
max_tip_amount	Integer	Optional	The maximum accepted amount for tips in the *smallest units* of the currency (integer, **not** float/double). For example, for a maximum tip of `US$ 1.45` pass `max_tip_amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies). Defaults to 0. Not supported for payments in Telegram Stars.
suggested_tip_amounts	Array of Integer	Optional	A JSON-serialized Array of suggested amounts of tips in the *smallest units* of the currency (integer, **not** float/double). At most 4 suggested tip amounts can be specified. The suggested tip amounts must be positive, passed in a strictly increased order and must not exceed *max_tip_amount*.
provider_data	String	Optional	JSON-serialized data about the invoice, which will be shared with the payment provider. A detailed description of required fields should be provided by the payment provider.

Parameter	Type	Required	Description
photo_url	String	Optional	URL of the product photo for the invoice. Can be a photo of the goods or a marketing image for a service.
photo_size	Integer	Optional	Photo size in bytes
photo_width	Integer	Optional	Photo width
photo_height	Integer	Optional	Photo height
need_name	Boolean	Optional	Pass *True* if you require the user's full name to complete the order. Ignored for payments in Telegram Stars.
need_phone_number	Boolean	Optional	Pass *True* if you require the user's phone number to complete the order. Ignored for payments in Telegram Stars.
need_email	Boolean	Optional	Pass *True* if you require the user's email address to complete the order. Ignored for payments in Telegram Stars.
need_shipping_address	Boolean	Optional	Pass *True* if you require the user's shipping address to complete the order. Ignored for payments in Telegram Stars.
send_phone_number_to_provider	Boolean	Optional	Pass *True* if the user's phone number should be sent to the provider. Ignored for payments in Telegram Stars.
send_email_to_provider	Boolean	Optional	Pass *True* if the user's email address should be sent to the provider. Ignored for payments in Telegram Stars.
is_flexible	Boolean	Optional	Pass *True* if the final price depends on the shipping method. Ignored for payments in Telegram Stars.

answerShippingQuery

If you sent an invoice requesting a shipping address and the parameter *is_flexible* was specified, the Bot API will send an Update with a *shipping_query* field to the bot. Use this method to reply to shipping queries. On success, *True* is returned.

Parameter	Type	Required	Description
shipping_query_id	String	Yes	Unique identifier for the query to be answered
ok	Boolean	Yes	Pass *True* if delivery to the specified address is possible and *False* if there are any problems (for example, if delivery to the specified address is not possible)
shipping_options	Array of ShippingOption	Optional	Required if *ok* is *True*. A JSON-serialized Array of available shipping options.

Parameter	Type	Required	Description
error_message	String	Optional	Required if *ok* is *False*. Error message in human readable form that explains why it is impossible to complete the order (e.g. “Sorry, delivery to your desired address is unavailable”). Telegram will display this message to the user.

answerPreCheckoutQuery

Once the user has confirmed their payment and shipping details, the Bot API sends the final confirmation in the form of an Update with the field *pre_checkout_query*. Use this method to respond to such pre-checkout queries. On success, *True* is returned. **Note:** The Bot API must receive an answer within 10 seconds after the pre-checkout query was sent.

Parameter	Type	Required	Description
pre_checkout_query_id	String	Yes	Unique identifier for the query to be answered
ok	Boolean	Yes	Specify *True* if everything is alright (goods are available, etc.) and the bot is ready to proceed with the order. Use *False* if there are any problems.
error_message	String	Optional	Required if *ok* is *False*. Error message in human readable form that explains the reason for failure to proceed with the checkout (e.g. "Sorry, somebody just bought the last of our amazing black T-shirts while you were busy filling out your payment details. Please choose a different color or garment!"). Telegram will display this message to the user.

getMyStarBalance

A method to get the current Telegram Stars balance of the bot. Requires no parameters. On success, returns a StarAmount object.

getStarTransactions

Returns the bot's Telegram Star transactions in chronological order. On success, returns a StarTransactions object.

Parameter	Type	Required	Description
offset	Integer	Optional	Number of transactions to skip in the response
limit	Integer	Optional	The maximum number of transactions to be retrieved. Values between 1-100 are accepted. Defaults to 100.

refundStarPayment

Refunds a successful payment in Telegram Stars. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Identifier of the user whose payment will be refunded
telegram_payment_charge_id	String	Yes	Telegram payment identifier

editUserStarSubscription

Allows the bot to cancel or re-enable extension of a subscription paid in Telegram Stars. Returns *True* on success.

Parameter	Type	Required	Description
user_id	Integer	Yes	Identifier of the user whose subscription will be edited
telegram_payment_charge_id	String	Yes	Telegram payment identifier for the subscription
is_canceled	Boolean	Yes	Pass *True* to cancel extension of the user subscription; the subscription must be active up to the end of the current subscription period. Pass *False* to allow the user to re-enable a subscription that was previously canceled by the bot.

LabeledPrice

This object represents a portion of the price for goods or services.

Field	Type	Description
label	String	Portion label
amount	Integer	Price of the product in the *smallest units* of the currency (integer, **not** float/double). For example, for a price of `US$ 1.45` pass `amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies).

Invoice

This object contains basic information about an invoice.

Field	Type	Description
title	String	Product name
description	String	Product description
start_parameter	String	Unique bot deep-linking parameter that can be used to generate this invoice
currency	String	Three-letter ISO 4217 currency code, or “XTR” for payments in Telegram Stars
total_amount	Integer	Total price in the *smallest units* of the currency (integer, **not** float/double). For example, for a price of `US$ 1.45` pass `amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies).

ShippingAddress

This object represents a shipping address.

Field	Type	Description
country_code	String	Two-letter ISO 3166-1 alpha-2 country code
state	String	State, if applicable
city	String	City
street_line1	String	First line for the address
street_line2	String	Second line for the address
post_code	String	Address post code

OrderInfo

This object represents information about an order.

Field	Type	Description
name	String	*Optional*. User name
phone_number	String	*Optional*. User's phone number
email	String	*Optional*. User email
shipping_address	ShippingAddress	*Optional*. User shipping address

ShippingOption

This object represents one shipping option.

Field	Type	Description
id	String	Shipping option identifier
title	String	Option title
prices	Array of LabeledPrice	List of price portions

SuccessfulPayment

This object contains basic information about a successful payment. Note that if the buyer initiates a chargeback with the relevant payment provider following this transaction, the funds may be debited from your balance. This is outside of Telegram's control.

Field	Type	Description
currency	String	Three-letter ISO 4217 currency code, or “XTR” for payments in Telegram Stars

Field	Type	Description
total_amount	Integer	Total price in the *smallest units* of the currency (integer, **not** float/double). For example, for a price of `US$ 1.45` pass `amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies).
invoice_payload	String	Bot-specified invoice payload
subscription_expiration_date	Integer	*Optional*. Expiration date of the subscription, in Unix time; for recurring payments only
is_recurring	True	*Optional*. *True*, if the payment is a recurring payment for a subscription
is_first_recurring	True	*Optional*. *True*, if the payment is the first payment for a subscription
shipping_option_id	String	*Optional*. Identifier of the shipping option chosen by the user
order_info	OrderInfo	*Optional*. Order information provided by the user
telegram_payment_charge_id	String	Telegram payment identifier
provider_payment_charge_id	String	Provider payment identifier

RefundedPayment

This object contains basic information about a refunded payment.

Field	Type	Description
currency	String	Three-letter ISO 4217 currency code, or “XTR” for payments in Telegram Stars. Currently, always “XTR”.
total_amount	Integer	Total refunded price in the *smallest units* of the currency (integer, **not** float/double). For example, for a price of `US$ 1.45`, `total_amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies).
invoice_payload	String	Bot-specified invoice payload
telegram_payment_charge_id	String	Telegram payment identifier
provider_payment_charge_id	String	*Optional*. Provider payment identifier

ShippingQuery

This object contains information about an incoming shipping query.

Field	Type	Description
id	String	Unique query identifier
from	User	User who sent the query

Field	Type	Description
invoice_payload	String	Bot-specified invoice payload
shipping_address	ShippingAddress	User specified shipping address

PreCheckoutQuery

This object contains information about an incoming pre-checkout query.

Field	Type	Description
id	String	Unique query identifier
from	User	User who sent the query
currency	String	Three-letter ISO 4217 currency code, or “XTR” for payments in Telegram Stars
total_amount	Integer	Total price in the *smallest units* of the currency (integer, **not** float/double). For example, for a price of `US$ 1.45` pass `amount = 145`. See the *exp* parameter in currencies.json, it shows the number of digits past the decimal point for each currency (2 for the majority of currencies).
invoice_payload	String	Bot-specified invoice payload
shipping_option_id	String	*Optional*. Identifier of the shipping option chosen by the user
order_info	OrderInfo	*Optional*. Order information provided by the user

PaidMediaPurchased

This object contains information about a paid media purchase.

Field	Type	Description
from	User	User who purchased the media
paid_media_payload	String	Bot-specified paid media payload

RevenueWithdrawalState

This object describes the state of a revenue withdrawal operation. Currently, it can be one of

- RevenueWithdrawalStatePending
- RevenueWithdrawalStateSucceeded
- RevenueWithdrawalStateFailed

RevenueWithdrawalStatePending

The withdrawal is in progress.

Field	Type	Description
type	String	Type of the state, always “pending”

RevenueWithdrawalStateSucceeded

The withdrawal succeeded.

Field	Type	Description
type	String	Type of the state, always “succeeded”
date	Integer	Date the withdrawal was completed in Unix time
url	String	An HTTPS URL that can be used to see transaction details

RevenueWithdrawalStateFailed

The withdrawal failed and the transaction was refunded.

Field	Type	Description
type	String	Type of the state, always “failed”

AffiliateInfo

Contains information about the affiliate that received a commission via this transaction.

Field	Type	Description
affiliate_user	User	*Optional*. The bot or the user that received an affiliate commission if it was received by a bot or a user
affiliate_chat	Chat	*Optional*. The chat that received an affiliate commission if it was received by a chat
commission_per_mille	Integer	The number of Telegram Stars received by the affiliate for each 1000 Telegram Stars received by the bot from referred users
amount	Integer	Integer amount of Telegram Stars received by the affiliate from the transaction, rounded to 0; can be negative for refunds
nanostar_amount	Integer	*Optional*. The number of 1/1000000000 shares of Telegram Stars received by the affiliate; from -999999999 to 999999999; can be negative for refunds

TransactionPartner

This object describes the source of a transaction, or its recipient for outgoing transactions. Currently, it can be one of

- TransactionPartnerUser
- TransactionPartnerChat
- TransactionPartnerAffiliateProgram
- TransactionPartnerFragment
- TransactionPartnerTelegramAds
- TransactionPartnerTelegramApi
- TransactionPartnerOther

TransactionPartnerUser

Describes a transaction with a user.

Field	Type	Description
type	String	Type of the transaction partner, always “user”
transaction_type	String	Type of the transaction, currently one of “invoice_payment” for payments via invoices, “paid_media_payment” for payments for paid media, “gift_purchase” for gifts sent by the bot, “premium_purchase” for Telegram Premium subscriptions gifted by the bot, “business_account_transfer” for direct transfers from managed business accounts
user	User	Information about the user
affiliate	AffiliateInfo	*Optional*. Information about the affiliate that received a commission via this transaction. Can be available only for “invoice_payment” and “paid_media_payment” transactions.
invoice_payload	String	*Optional*. Bot-specified invoice payload. Can be available only for “invoice_payment” transactions.
subscription_period	Integer	*Optional*. The duration of the paid subscription. Can be available only for “invoice_payment” transactions.
paid_media	Array of PaidMedia	*Optional*. Information about the paid media bought by the user; for “paid_media_payment” transactions only
paid_media_payload	String	*Optional*. Bot-specified paid media payload. Can be available only for “paid_media_payment” transactions.
gift	Gift	*Optional*. The gift sent to the user by the bot; for “gift_purchase” transactions only
premium_subscription_duration	Integer	*Optional*. Number of months the gifted Telegram Premium subscription will be active for; for “premium_purchase” transactions only

TransactionPartnerChat

Describes a transaction with a chat.

Field	Type	Description
type	String	Type of the transaction partner, always “chat”
chat	Chat	Information about the chat
gift	Gift	*Optional*. The gift sent to the chat by the bot

TransactionPartnerAffiliateProgram

Describes the affiliate program that issued the affiliate commission received via this transaction.

Field	Type	Description
type	String	Type of the transaction partner, always “affiliate_program”
sponsor_user	User	*Optional*. Information about the bot that sponsored the affiliate program
commission_per_mille	Integer	The number of Telegram Stars received by the bot for each 1000 Telegram Stars received by the affiliate program sponsor from referred users

TransactionPartnerFragment

Describes a withdrawal transaction with Fragment.

Field	Type	Description
type	String	Type of the transaction partner, always “fragment”
withdrawal_state	RevenueWithdrawalState	*Optional*. State of the transaction if the transaction is outgoing

TransactionPartnerTelegramAds

Describes a withdrawal transaction to the Telegram Ads platform.

Field	Type	Description
type	String	Type of the transaction partner, always “telegram_ads”

TransactionPartnerTelegramApi

Describes a transaction with payment for paid broadcasting.

Field	Type	Description
type	String	Type of the transaction partner, always “telegram_api”
request_count	Integer	The number of successful requests that exceeded regular limits and were therefore billed

TransactionPartnerOther

Describes a transaction with an unknown source or recipient.

Field	Type	Description
type	String	Type of the transaction partner, always “other”

StarTransaction

Describes a Telegram Star transaction. Note that if the buyer initiates a chargeback with the payment provider from whom they acquired Stars (e.g., Apple, Google) following this transaction, the refunded Stars will be deducted from the bot's balance. This is outside of Telegram's control.

Field	Type	Description
id	String	Unique identifier of the transaction. Coincides with the identifier of the original transaction for refund transactions. Coincides with *SuccessfulPayment.telegram_payment_charge_id* for successful incoming payments from users.
amount	Integer	Integer amount of Telegram Stars transferred by the transaction
nanostar_amount	Integer	*Optional*. The number of 1/1000000000 shares of Telegram Stars transferred by the transaction; from 0 to 999999999
date	Integer	Date the transaction was created in Unix time
source	TransactionPartner	*Optional*. Source of an incoming transaction (e.g., a user purchasing goods or services, Fragment refunding a failed withdrawal). Only for incoming transactions.
receiver	TransactionPartner	*Optional*. Receiver of an outgoing transaction (e.g., a user for a purchase refund, Fragment for a withdrawal). Only for outgoing transactions.

StarTransactions

Contains a list of Telegram Star transactions.

Field	Type	Description
transactions	Array of StarTransaction	The list of transactions

Telegram Passport

Telegram Passport is a unified authorization method for services that require personal identification. Users can upload their documents once, then instantly share their data with services that require real-world ID (finance, ICOs, etc.). Please see the manual for details.

PassportData

Describes Telegram Passport data shared with the bot by the user.

Field	Type	Description
data	Array of EncryptedPassportElement	Array with information about documents and other Telegram Passport elements that was shared with the bot
credentials	EncryptedCredentials	Encrypted credentials required to decrypt the data

PassportFile

This object represents a file uploaded to Telegram Passport. Currently all Telegram Passport files are in JPEG format when decrypted and don't exceed 10MB.

Field	Type	Description
file_id	String	Identifier for this file, which can be used to download or reuse the file
file_unique_id	String	Unique identifier for this file, which is supposed to be the same over time and for different bots. Can't be used to download or reuse the file.
file_size	Integer	File size in bytes
file_date	Integer	Unix time when the file was uploaded

EncryptedPassportElement

Describes documents or other Telegram Passport elements shared with the bot by the user.

Field	Type	Description
type	String	Element type. One of “personal_details”, “passport”, “driver_license”, “identity_card”, “internal_passport”, “address”, “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration”, “temporary_registration”, “phone_number”, “email”.
data	String	*Optional*. Base64-encoded encrypted Telegram Passport element data provided by the user; available only for “personal_details”, “passport”, “driver_license”, “identity_card”, “internal_passport” and “address” types. Can be decrypted and verified using the accompanying EncryptedCredentials.
phone_number	String	*Optional*. User's verified phone number; available only for “phone_number” type
email	String	*Optional*. User's verified email address; available only for “email” type
files	Array of PassportFile	*Optional*. Array of encrypted files with documents provided by the user; available only for “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration” and “temporary_registration” types. Files can be decrypted and verified using the accompanying EncryptedCredentials.
front_side	PassportFile	*Optional*. Encrypted file with the front side of the document, provided by the user; available only for “passport”, “driver_license”, “identity_card” and “internal_passport”. The file can be decrypted and verified using the accompanying EncryptedCredentials.
reverse_side	PassportFile	*Optional*. Encrypted file with the reverse side of the document, provided by the user; available only for “driver_license” and “identity_card”. The file can be decrypted and verified using the accompanying EncryptedCredentials.
selfie	PassportFile	*Optional*. Encrypted file with the selfie of the user holding a document, provided by the user; available if requested for “passport”, “driver_license”, “identity_card” and “internal_passport”. The file can be decrypted and verified using the accompanying EncryptedCredentials.
translation	Array of PassportFile	*Optional*. Array of encrypted files with translated versions of documents provided by the user; available if requested for “passport”, “driver_license”, “identity_card”, “internal_passport”, “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration” and “temporary_registration” types. Files can be decrypted and verified using the accompanying EncryptedCredentials.

Field	Type	Description
hash	String	Base64-encoded element hash for using in PassportElementErrorUnspecified

EncryptedCredentials

Describes data required for decrypting and authenticating EncryptedPassportElement. See the Telegram Passport Documentation for a complete description of the data decryption and authentication processes.

Field	Type	Description
data	String	Base64-encoded encrypted JSON-serialized data with unique user's payload, data hashes and secrets required for EncryptedPassportElement decryption and authentication
hash	String	Base64-encoded data hash for data authentication
secret	String	Base64-encoded secret, encrypted with the bot's public RSA key, required for data decryption

setPassportDataErrors

Informs a user that some of the Telegram Passport elements they provided contains errors. The user will not be able to re-submit their Passport to you until the errors are fixed (the contents of the field for which you returned the error must change). Returns *True* on success.

Use this if the data submitted by the user doesn't satisfy the standards your service requires for any reason. For example, if a birthday date seems invalid, a submitted document is blurry, a scan shows evidence of tampering, etc. Supply some details in the error message to make sure the user knows how to correct the issues.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier
errors	Array of PassportElementError	Yes	A JSON-serialized Array describing the errors

PassportElementError

This object represents an error in the Telegram Passport element which was submitted that should be resolved by the user. It should be one of:

- PassportElementErrorDataField
- PassportElementErrorFrontSide
- PassportElementErrorReverseSide
- PassportElementErrorSelfie
- PassportElementErrorFile
- PassportElementErrorFiles
- PassportElementErrorTranslationFile
- PassportElementErrorTranslationFiles
- PassportElementErrorUnspecified

PassportElementErrorDataField

Represents an issue in one of the data fields that was provided by the user. The error is considered resolved when the field's value changes.

Field	Type	Description
source	String	Error source, must be *data*
type	String	The section of the user's Telegram Passport which has the error, one of “personal_details”, “passport”, “driver_license”, “identity_card”, “internal_passport”, “address”
field_name	String	Name of the data field which has the error
data_hash	String	Base64-encoded data hash
message	String	Error message

PassportElementErrorFrontSide

Represents an issue with the front side of a document. The error is considered resolved when the file with the front side of the document changes.

Field	Type	Description
source	String	Error source, must be *front_side*
type	String	The section of the user's Telegram Passport which has the issue, one of “passport”, “driver_license”, “identity_card”, “internal_passport”
file_hash	String	Base64-encoded hash of the file with the front side of the document
message	String	Error message

PassportElementErrorReverseSide

Represents an issue with the reverse side of a document. The error is considered resolved when the file with reverse side of the document changes.

Field	Type	Description
source	String	Error source, must be *reverse_side*
type	String	The section of the user's Telegram Passport which has the issue, one of “driver_license”, “identity_card”
file_hash	String	Base64-encoded hash of the file with the reverse side of the document
message	String	Error message

PassportElementErrorSelfie

Represents an issue with the selfie with a document. The error is considered resolved when the file with the selfie changes.

Field	Type	Description
source	String	Error source, must be *selfie*
type	String	The section of the user's Telegram Passport which has the issue, one of “passport”, “driver_license”, “identity_card”, “internal_passport”
file_hash	String	Base64-encoded hash of the file with the selfie
message	String	Error message

PassportElementErrorFile

Represents an issue with a document scan. The error is considered resolved when the file with the document scan changes.

Field	Type	Description
source	String	Error source, must be *file*
type	String	The section of the user's Telegram Passport which has the issue, one of “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration”, “temporary_registration”
file_hash	String	Base64-encoded file hash
message	String	Error message

PassportElementErrorFiles

Represents an issue with a list of scans. The error is considered resolved when the list of files containing the scans changes.

Field	Type	Description
source	String	Error source, must be *files*
type	String	The section of the user's Telegram Passport which has the issue, one of “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration”, “temporary_registration”
file_hashes	Array of String	List of base64-encoded file hashes
message	String	Error message

PassportElementErrorTranslationFile

Represents an issue with one of the files that constitute the translation of a document. The error is considered resolved when the file changes.

Field	Type	Description
source	String	Error source, must be *translation_file*

Field	Type	Description
type	String	Type of element of the user's Telegram Passport which has the issue, one of “passport”, “driver_license”, “identity_card”, “internal_passport”, “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration”, “temporary_registration”
file_hash	String	Base64-encoded file hash
message	String	Error message

PassportElementErrorTranslationFiles

Represents an issue with the translated version of a document. The error is considered resolved when a file with the document translation change.

Field	Type	Description
source	String	Error source, must be *translation_files*
type	String	Type of element of the user's Telegram Passport which has the issue, one of “passport”, “driver_license”, “identity_card”, “internal_passport”, “utility_bill”, “bank_statement”, “rental_agreement”, “passport_registration”, “temporary_registration”
file_hashes	Array of String	List of base64-encoded file hashes
message	String	Error message

PassportElementErrorUnspecified

Represents an issue in an unspecified place. The error is considered resolved when new data is added.

Field	Type	Description
source	String	Error source, must be *unspecified*
type	String	Type of element of the user's Telegram Passport which has the issue
element_hash	String	Base64-encoded element hash
message	String	Error message

Games

Your bot can offer users **HTML5 games** to play solo or to compete against each other in groups and one-on-one chats. Create games via @BotFather using the */newgame* command. Please note that this kind of power requires responsibility: you will need to accept the terms for each game that your bots will be offering.

- Games are a new type of content on Telegram, represented by the Game and InlineQueryResultGame objects.

- Once you've created a game via BotFather, you can send games to chats as regular messages using the sendGame method, or use inline mode with InlineQueryResultGame.
- If you send the game message without any buttons, it will automatically have a 'Play *GameName*' button. When this button is pressed, your bot gets a CallbackQuery with the *game_short_name* of the requested game. You provide the correct URL for this particular user and the app opens the game in the in-app browser.
- You can manually add multiple buttons to your game message. Please note that the first button in the first row **must always** launch the game, using the field *callback_game* in InlineKeyboardButton. You can add extra buttons according to taste: e.g., for a description of the rules, or to open the game's official community.
- To make your game more attractive, you can upload a GIF animation that demonstrates the game to the users via BotFather (see Lumberjack for example).
- A game message will also display high scores for the current chat. Use setGameScore to post high scores to the chat with the game, add the *disable_edit_message* parameter to disable automatic update of the message with the current scoreboard.
- Use getGameHighScores to get data for in-game high score tables.
- You can also add an extra sharing button for users to share their best score to different chats.
- For examples of what can be done using this new stuff, check the @gamebot and @gamee bots.

sendGame

Use this method to send a game. On success, the sent Message is returned.

Parameter	Type	Required	Description
business_connection_id	String	Optional	Unique identifier of the business connection on behalf of which the message will be sent
chat_id	Integer or String	Yes	Unique identifier for the target chat or username of the target bot in the format `@username`. Games can't be sent to channel direct messages chats and channel chats.
message_thread_id	Integer	Optional	Unique identifier for the target message thread (topic) of a forum; for forum supergroups and private chats of bots with forum topic mode enabled only
game_short_name	String	Yes	Short name of the game, serves as the unique identifier for the game. Set up your games via @BotFather.
disable_notification	Boolean	Optional	Sends the message silently. Users will receive a notification with no sound.
protect_content	Boolean	Optional	Protects the contents of the sent message from forwarding and saving
allow_paid_broadcast	Boolean	Optional	Pass *True* to allow up to 1000 messages per second, ignoring broadcasting limits for a fee of 0.1 Telegram Stars per message. The relevant Stars will be withdrawn from the bot's balance.

Parameter	Type	Required	Description
message_effect_id	String	Optional	Unique identifier of the message effect to be added to the message; for private chats only
reply_parameters	ReplyParameters	Optional	Description of the message to reply to
reply_markup	InlineKeyboardMarkup	Optional	A JSON-serialized object for an inline keyboard. If empty, one 'Play game_title' button will be shown. If not empty, the first button must launch the game.

Game

This object represents a game. Use BotFather to create and edit games, their short names will act as unique identifiers.

Field	Type	Description
title	String	Title of the game
description	String	Description of the game
photo	Array of PhotoSize	Photo that will be displayed in the game message in chats
text	String	*Optional*. Brief description of the game or high scores included in the game message. Can be automatically edited to include current high scores for the game when the bot calls setGameScore, or manually edited using editMessageText. 0-4096 characters.
text_entities	Array of MessageEntity	*Optional*. Special entities that appear in *text*, such as usernames, URLs, bot commands, etc.
animation	Animation	*Optional*. Animation that will be displayed in the game message in chats. Upload via BotFather.

CallbackGame

A placeholder, currently holds no information. Use BotFather to set up your game.

setGameScore

Use this method to set the score of the specified user in a game message. On success, if the message is not an inline message, the Message is returned, otherwise *True* is returned. Returns an error, if the new score is not greater than the user's current score in the chat and *force* is *False*.

Parameter	Type	Required	Description
user_id	Integer	Yes	User identifier
score	Integer	Yes	New score, must be non-negative

Parameter	Type	Required	Description
force	Boolean	Optional	Pass *True* if the high score is allowed to decrease. This can be useful when fixing mistakes or banning cheaters.
disable_edit_message	Boolean	Optional	Pass *True* if the game message should not be automatically edited to include the current scoreboard
chat_id	Integer	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the sent message.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.

getGameHighScores

Use this method to get data for high score tables. Will return the score of the specified user and several of their neighbors in a game. Returns an Array of GameHighScore objects.

> This method will currently return scores for the target user, plus two of their closest neighbors on each side. Will also return the top three users if the user and their neighbors are not among them. Please note that this behavior is subject to change.

Parameter	Type	Required	Description
user_id	Integer	Yes	Target user id
chat_id	Integer	Optional	Required if *inline_message_id* is not specified. Unique identifier for the target chat.
message_id	Integer	Optional	Required if *inline_message_id* is not specified. Identifier of the sent message.
inline_message_id	String	Optional	Required if *chat_id* and *message_id* are not specified. Identifier of the inline message.

GameHighScore

This object represents one row of the high scores table for a game.

Field	Type	Description
position	Integer	Position in high score table for the game
user	User	User
score	Integer	Score

And that's about all we've got for now.
If you've got any questions, please check out our **Bot FAQ »**